

УНИВЕРЗИТЕТ У БЕОГРАДУ
ФАКУЛТЕТ ОРГАНИЗАЦИОНИХ НАУКА

ЗАВРШНИ РАД

**Софтверски систем за продавницу
чоколаде у Јава окружењу**

Ментор

**Др Синиша Влајић,
Редовни професор**

Студент

Јелена Чавић, 2018/0233

Београд, 2026. године

Садржај:

1.	Увод	3
2.	Упрошћена Ларманова метода развоја софтвера	4
2.1.	Основни принципи Ларманове методе развоја софтвера.....	4
2.2.	Фазе упрошћене Ларманове методе развоја софтвера	4
2.2.1.	Прикупљање корисничких захтева	5
2.2.2.	Анализа система.....	6
2.2.3.	Пројектовање (дизајн)	9
2.2.4.	Имплементација	10
2.2.5.	Тестирање система.....	10
3.	Јава технологије	11
3.1.	Објектно оријентисано програмирање	11
3.2.	Графички кориснички интерфејс (GUI).....	15
3.3.	Нити	18
3.3.1.	Нити – теоријско објашњење.....	18
3.3.2.	Примена нити у развоју софтверског система студијског примера	19
3.4.	Мрежна комуникација – сокети	22
3.4.1.	Сокети – теоријско објашњење	22
3.4.2.	Примена сокета у развоју софтверског система студијског примера	23
3.5.	Базе података – JDBC	26
4.	Студијски пример	30
4.1.	Фаза прикупљања корисничких захтева.....	30
4.1.1.	Вербални опис	30
4.1.2.	Случајеви коришћења.....	31
4.2.	Фаза анализе.....	41
4.2.1.	Понашање софтверског система.....	41
4.2.2.	Резултирајуће системске операције	68
4.2.3.	Дефинисање уговора о системским операцијама.....	69
4.2.4.	Структура софтверског система - Концептуални (доменски) модел	72
4.2.5.	Структура софтверског система- Релациони модел	73
4.3.	Фаза пројектовања	78
4.3.1.	Пројектовање корисничког интерфејса	79
4.3.2.	Пројектовање апликационе логике	109
4.4.	Фаза имплементације	127
4.5.	Тестирање.....	130
4.5.1.	JUnit оквир за јединично тестирање.....	130
4.5.2.	Тестирање системске операције SOAddCokolada	131
4.5.3.	Тестирање системске операције SOUpdatePorudzbina.....	133
5.	Закључак	136
6.	Коришћена литература	137

1. Увод

Савремено пословање карактерише интензивна употреба софтверских решења која омогућавају брже, прецизније и транспарентније обављање пословних процеса. Како трговински сектор све више усваја дигиталне технологије, јавља се потреба за развојем специјализованих апликација које подржавају ефикасно управљање артиклима, купцима, поруџбинама и продајним токовима. У том контексту, циљ овог рада је развој софтверског система намењеног продавници чоколаде, који омогућава аутоматизацију кључних активности и унапређење укупног квалитета пословања.

Развој поузданог софтверског система подразумева методолошки приступ, јасну структуру и систематичну анализу захтева корисника. Посебан значај има избор адекватне методе развоја софтвера, јер она одређује ток активности, фазе које се спроводе, артефакте који се добијају и начин на који се успоставља јасна веза између захтева, анализе, пројектовања и имплементације. У настави на Факултету организационих наука примењује се **упрошћена Ларманова метода развоја софтвера**, која почива на итеративном приступу и објектно-оријентисаном размишљању, уз јасно разграничење структура и понашања софтверског система. [1] Ова метода омогућава постепено унапређивање модела система, проверу исправности у свакој фази и лакшу комуникацију између развојног тима и корисника. За реализацију развијеног решења изабрано је **Јава окружење**, јер Јава представља робустан, безбедан и мултиплатформски језик погодан за изградњу стабилних пословних апликација. [2] Богат екосистем библиотека и подршка за објектно-оријентисану парадигму омогућавају јасну имплементацију пословне логике, развој корисничког интерфејса и интеграцију са складиштем података.

У овом раду прво се представља упрошћена Ларманова метода развоја софтвера као теоријска основа развоја софтверског система. Након тога следи приказ Јава технологија релевантних за имплементацију, а затим и студијски пример који приказује примену методе на конкретан систем за продавницу чоколаде. Кроз фазе прикупљања захтева, анализе, пројектовања, имплементације и тестирања добија се потпуно заокружено софтверско решење, чија примена може да подигне ниво дигитализације и ефикасности пословања у продавници чоколаде.

2. Упрощена Ларманова метода развоја софтвера

Упрощена Ларманова метода развоја софтвера представља модел развоја софтверског система који се заснива на идејама Крејга Лармана о итеративном, инкременталном и објектно-оријентисаном развоју софтвера. Метода задржава суштину Лармановог приступа, али је прилагођена мањим пројектима и академском контексту. Кључна вредност методе је јасно раздвајање **структуре софтверског система** (концепти и њихови атрибути) и **понашања система** (системске операције), што омогућава доследност у преласку са захтева на дизајн и имплементацију.

2.1. Основни принципи Ларманове методе развоја софтвера

Ларманова метода заснива се на више кључних начела која усмеравају целокупан развој:

1. Итеративни и инкрементални приступ

Развој се реализује кроз низ малих итерација. Свака итерација производи функционалну целину која може бити тестирана и унапређена. Овим се смањује ризик, а грешке се откривају у раним фазама.

2. Објектно-оријентисано моделирање

Систем се сагледава кроз концепте реалног домена (класе) и њихову интеракцију. Структура и понашање су структурисани тако да олакшају каснију имплементацију у објектно-оријентисаном програмском језику, као што је Јава.

3. Усмереност на корисничке захтеве

Захтеви се формулишу на основу стварних потреба корисника. Основни алат за њихово моделирање јесу случајеви коришћења (Use Cases), који представљају жељене интеракције са системом.

4. Наглашена улога пословне логике

Структура система (концептуални модел) и понашање (системске операције) чине суштину пословне логике. Управо ова два елемента воде ка правилном пројектовању апликационе логике и складишта података.

5. Раздвајање модела и приказа

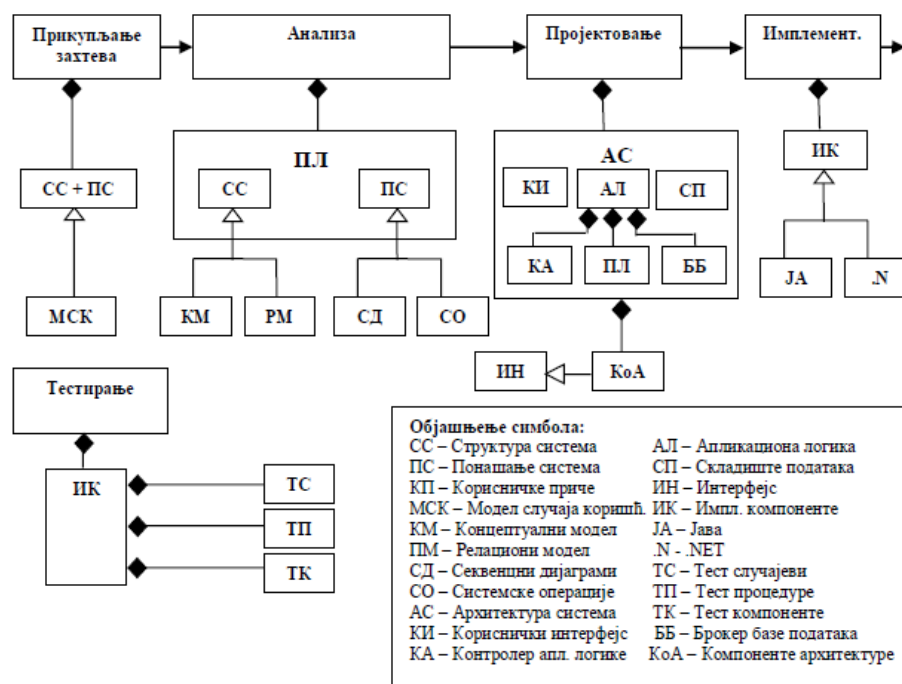
Према принципу „Model - View Separation“ апликациона логика је независна од корисничког интерфејса, што омогућава да исти систем има више различитих интерфејса (нпр. desktop, web) без промене логике.

2.2. Фазе упрощене Ларманове методе развоја софтвера

У свом упрощеном облику метода обухвата пет фаза:

1. Прикупљање корисничких захтева
2. Анализа
3. Пројектовање
4. Имплементација
5. Тестирање

Ове фазе нису стриктно секвенцијалне — резултати једне фазе могу да доведу до корекција у претходним фазама, што одражава итеративну природу методе.



Слика 1: Развој софтверског система [1]

2.2.1. Прикупљање корисничких захтева

Прикупљање корисничких захтева представља почетну и једну од најважнијих фаза у развоју софтверског система, јер се у овој фази дефинише **шта систем треба да ради и под којим условима**. Кориснички захтеви описују потребе, очекивања и ограничења која корисници имају у односу на софтверски систем и служе као основа за све наредне фазе развоја.

Кориснички захтев је својство или услов који систем мора да задовољи. Они се најчешће деле на **функционалне** и **нефункционалне** захтеве.

Функционални захтеви описују **конкретне функционалности система**, односно шта систем треба да омогући кориснику (на пример: додавање, измена или брисање података).

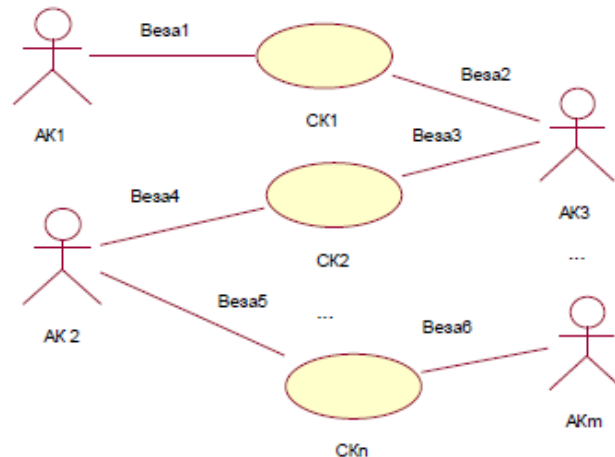
Нефункционални захтеви се односе на **квалитативне аспекте система**, као што су поузданост, безбедност, перформансе, употребљивост и ограничења технологије.

У овој фази развоја, кориснички захтеви се формализују коришћењем **UML модела случајева коришћења**, који омогућава јасан и разумљив приказ интеракције између корисника и система. Модел случаја коришћења састоји се од:

- **актора**, који представљају спољне кориснике или системе који комуницирају са апликацијом,
- **случајева коришћења**, који описују функционалности које систем пружа,
- **веза** између актора и случајева коришћења, које приказују на који начин корисници користе систем.

Један модел случајева коришћења може садржати више актора и више случајева коришћења, при чему један актор може учествовати у више случајева коришћења, као и један случај коришћења може бити повезан са више актора.

Сваки случај коришћења описује се кроз **скуп сценарија**, који представљају различите токове интеракције између актора и система. Сценарији се разлажу на акције актора (АПУСО¹, АНСО², АПСО³) и одговоре система (ИА⁴), што у каснијој фази омогућава једноставан прелаз ка системским дијаграмима секвенци. Основни (главни) сценарио описује типичан и очекиван ток извршавања, док алтернативни сценарији обухватају могуће изузетке и варијације у понашању система. На тај начин се у раној фази развоја добија **конкретна и прегледна слика начина на који актори комуницирају са апликацијом**, као и увид у понашање система у различитим ситуацијама.



Слика 2: Модел случаја коришћења [1]

2.2.2. Анализа система

Фаза анализе система представља фазу у којој се, на основу прикупљених корисничких захтева, врши **детаљан опис понашања и структуре система**, без разматрања конкретних техничких решења и имплементационих детаља. У овој фази се дефинише шта систем треба да ради, на који начин реагује на захтеве корисника и каквим подацима располаже, како би се створила јасна основа за каснију фазу пројектовања.

Анализа система обухвата два основна аспекта:

- **анализу понашања система**, која описује динамику система и начин на који систем обрађује корисничке захтеве,
- **анализу структуре система**, која описује податке којима систем управља и односе између њих.

Анализа понашања система описује динамички аспект система, односно начин на који систем комуницира са актерима и како обрађује њихове захтеве.

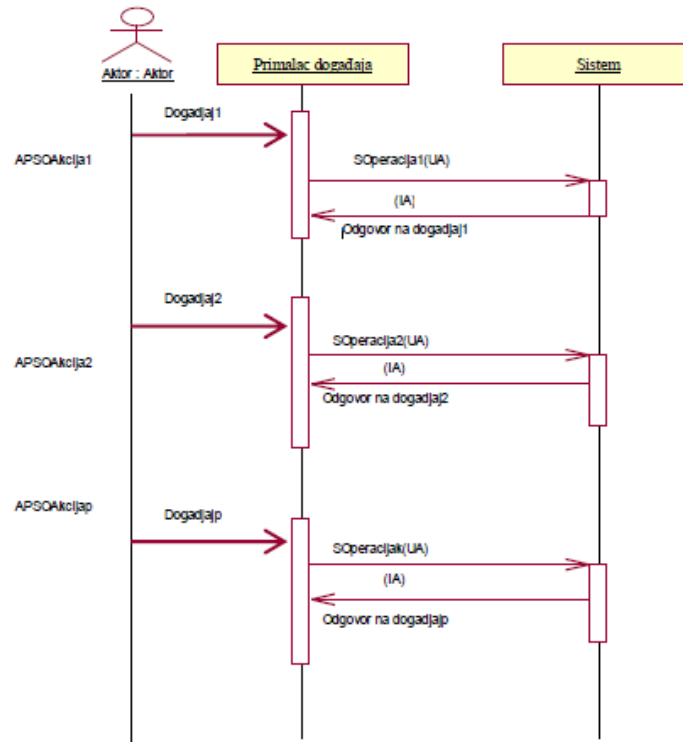
¹ Актор Припрема Улаз (Улазне Аргументе) за Системску Операцију (АПУСО).

² Актор извршава НеСистемску Операцију (АНСО).

³ Актор Позива систем да изврши Системску Операцију (АПСО).

⁴ Излазни аргументи (ИА)

Дијаграми секвенци користе се за графички приказ интеракције између актера и система током извршавања појединих случајева коришћења. Они приказују редослед порука које се размењују између објеката и омогућавају јасно разумевање тока извршавања система у времену.



Слика 3: Дијаграм секвенци [1]

На основу дијаграма секвенци дефинишу се **системске операције**, које представљају функционалности које систем пружа кориснику. Системске операције описују комуникацију између клијентске и серверске стране система и представљају основне јединице пословне логике. Свака системска операција садржи назив методе и њене улазне и излазне параметре.

За сваку системску операцију дефинисани су **уговори системских операција**, који формално описују понашање система. Уговори системских операција садрже назив операције, улазне параметре, имена СК у којима се позива системска операција, предуслове који морају бити испуњени пре извршавања операције, као и постуслове који описују стање система након успешног извршавања операције.

У оквиру анализе понашања система дефинисана су и **вредносна и структурна ограничења** при извршењу системских операција. Вредносна ограничења односе се на правила која дефинишу дозвољене вредности појединих атрибута, као што су обавезност уноса података или дозвољени опсези вредности. Структурна ограничења дефинишу правила која се односе на односе између ентитета, као што су кардиналности и услови постојања веза.

Табела Racun		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Atributi	Име	Тип атрибута	Вредност атрибута	Међузав. атрибута једне табеле	Међузав. атрибута више табела	INSERT /
	BrojRacuna	String	not null			UPDATE
	NazivPartnera	String				CASCADES
	UkupnaVrednost	Double	(default:0)		UkupnaVrednost= SUM (StavkaRacuna.ProdajnaVrednost)	StavkeRacuna
	Obradjen	Boolean	(default: false)			DELETE CASCADES
	Storniran	Boolean	(default: false)			StavkeRacuna

Слика 4: Пример табеле вредносних и структурних ограничења [1]

Анализа структуре система бави се статичким аспектом система, односно дефинисањем података које систем користи и односа који постоје између њих. Ова анализа обухвата концептуални модел и релациони модел система.

Концептуални модел представља апстрактан приказ доменских појмова и њихових међусобних односа. Он описује реалне ентитете из проблемског домена, њихове атрибуте и везе, без уласка у детаље начина чувања података.



Слика 5: Пример концептуалног модела [1]

На основу концептуалног модела креиран је **релациони модел**, који представља логичку структуру базе података. Релациони модел дефинише табеле, њихове атрибуте, примарне и спољне кључеве, као и односе између табела. Овај модел представља директну основу за пројектовање и имплементацију базе података система.

Racun(BrojRacuna, NazivPartnera, UkupnaVrednost, Obradjen, Storniran);

StavkaRacuna(BrojRacuna, RB, SifraProizvoda, Kolicina, ProdajnaCena, ProdajnaVrednost)

Слика 6: Пример релационог модела [1]

2.2.3. Пројектовање (дизајн)

Фаза пројектовања дефинише **физичку архитектуру** софтверског система. Примењује се тронивојска архитектура која се састоји од:

1. **Корисничког интерфејса**

- екранске форме и контролери корисничког интерфејса.

2. **Апликационе логике**

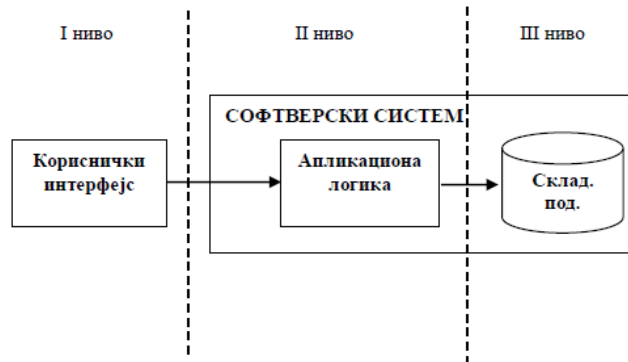
- контролер апликационе логике, пословна логика и брокер базе података.

3. **Складишта података**

- релациони модел који води ка имплементацији базе.

На основу концептуалног модела креира се релациони модел, а на основу системских операција дефинишу се уговори о њиховој имплементацији (предуслови, постуслови, ефекти).

Ова фаза обезбеђује да имплементација буде јасно структурирана и да све компоненте буду међусобно доследне.



Слика 7: Тронивојска архитектура у Лармановој методи развоја софтвера [1]

2.2.4. Имплементација

Имплементација представља претварање пројектованих модела у изворни код. У Јава окружењу ово подразумева:

- креирање класа које представљају доменске концепте,
- реализацију системских операција у оквиру пословне логике,
- имплементацију приступа бази података (нпр. JDBC),
- повезивање корисничког интерфејса са апликационом логиком.

Упрошћена Ларманова метода подстиче да имплементација прати редослед системских операција, што омогућава директну проверу исправности и лако праћење током тестирања.

2.2.5. Тестирање система

Циљ тестирања је провера исправности и квалитета развијене апликације. Тестирање се заснива на:

- тест случајевима,
- тест процедурама,
- тест компонентама,

који прате архитектурске компоненте које су развијене у претходним фазама. Према упрошћеној Лармановој методи развоја софтвера, тестирање се спроводи континуирано, итеративно, након имплементације сваке значајне функционалности.

3. Јава технологије

Приликом израде софтверског система за управљање радом продавнице чоколаде коришћен је програмски језик Јава, као и пратеће технологије које ова платформа нуди. Јава је изабрана због своје стабилности, преносивости, велике заступљености у развоју пословних апликација и добре подршке за објектно оријентисано програмирање, рад са базама података и мрежну комуникацију.

Апликација је реализована као клијент–сервер систем, при чему је клијентска апликација задужена за интеракцију са корисником, док серверска апликација обрађује захтеве, реализује пословну логику и комуницира са базом података. У наставку поглавља описане су Јава технологије које су коришћене током развоја система.

3.1. Објектно оријентисано програмирање

Објектно оријентисано програмирање представља основу развоја савремених софтверских система и кључни концепт на коме се заснива програмски језик Јава. Овај приступ омогућава моделовање система кроз објекте који представљају појмове из реалног света, чиме се постиже боља читљивост, проширивост и одрживост кода.

У оквиру овог пројекта објектно оријентисани приступ примењен је кроз доменске класе које описују основне ентитете система, као што су чоколада, врста чоколаде, купац, поруџбина и ставка поруџбине. Свака од ових класа садржи атрибуте који описују стање објекта, као и методе које дефинишу његово понашање.

```
public class Cokolada extends AbstractDomainObject {

    private int cokoladaID;
    private String naziv;
    private String opis;
    private double cena;
    private VrstaCokolade vrstaCokolade;

    @Override
    public String toString() {
        return naziv + " (Cena: " + cena + "din)";
    }

    public Cokolada(int cokoladaID, String naziv, String opis, double cena, VrstaCokolade vrstaCokolade) {
        this.cokoladaID = cokoladaID;
        this.naziv = naziv;
        this.opis = opis;
        this.cena = cena;
        this.vrstaCokolade = vrstaCokolade;
    }

    public Cokolada() {
    }

    @Override
    public String nazivTabele() {
        return "cokolada";
    }

    @Override
    public String alias() {
        return "c";
    }
}
```

Слика 8: Структура класе Чоколада

Посебна пажња посвећена је принципу уцаурења, тако што су атрибути класа дефинисани као приватни, а приступ подацима омогућен преко јавних *get* и *set* метода. На овај начин обезбеђена је контрола над изменама података и смањена могућност грешака.

```
public int getCokoladaID() {
    return cokoladaID;
}

public void setCokoladaID(int cokoladaID) {
    this.cokoladaID = cokoladaID;
}

public String getNaziv() {
    return naziv;
}

public void setNaziv(String naziv) {
    this.naziv = naziv;
}

public String getOpis() {
    return opis;
}

public void setOpis(String opis) {
    this.opis = opis;
}
```

Слика 9: *Get* и *set* методе класе Чоколада

Наслеђивање је примењено кроз коришћење апстрактних класа, што је омогућило да се заједнички делови логике дефинишу на једном месту, а конкретне имплементације реализују у појединачним класама. Све доменске класе наслеђују апстрактну класу *AbstractDomainObject*, која садржи искључиво апстрактне методе, док све класе које реализују системске операције наслеђују апстрактну класу *AbstractSO*, која поред апстрактних садржи и обичне методе. Да би нека класа или метода била апстрактна, потребно је користити кључну реч *abstract*, а како би се нека класа наследила, морамо употребити кључну реч *extends*.

```
public abstract class AbstractSO {

    protected abstract void validate(AbstractDomainObject ado) throws Exception;
    protected abstract void execute(AbstractDomainObject ado) throws Exception;

    public void templateExecute(AbstractDomainObject ado) throws Exception {
        try {
            validate(ado);
            execute(ado);
            commit();
        } catch (Exception e) {
            rollback();
            throw e;
        }
    }

    public void commit() throws SQLException {
        DBBroker.getInstance().getConnection().commit();
    }

    public void rollback() throws SQLException {
        DBBroker.getInstance().getConnection().rollback();
    }
}
```

Слика 10: Апстрактна класа *AbstractSO*

```

public class SOAddCokolada extends AbstractSO {

    @Override
    protected void validate(AbstractDomainObject ado) throws Exception {
        if (!(ado instanceof Cokolada)) {
            throw new Exception(message: "Prosledjeni objekat nije instanca klase Cokolada!");
        }

        Cokolada c = (Cokolada) ado;

        if (c.getCena() < 50 || c.getCena() > 50000) {
            throw new Exception(message: "Cena cokolade mora biti izmedju 50din i 50000din!");
        }

        ArrayList<Cokolada> cokolade = (ArrayList<Cokolada>) (ArrayList<?>) DBBroker.getInstance().select(ado);

        for (Cokolada cokolada : cokolade) {
            if (cokolada.getNaziv().equals(anObject: c.getNaziv())) {
                throw new Exception(message: "Cokolada sa tim nazivom vec postoji!");
            }
        }
    }

    @Override
    protected void execute(AbstractDomainObject ado) throws Exception {
        DBBroker.getInstance().insert(ado);
    }
}

```

Слика 11: Структура класе системске операције за додавање чоколаде

Апстрактна класа *AbstractDomainObject* имплементира интерфејс *Serializable* коришћењем кључне речи *implements*, што омогућава да се објекти ових класа шаљу кроз мрежу између клијентске и серверске апликације. Овај приступ омогућава да се различити типови објеката преносе истим комуникационим механизмом, чиме се наглашава примена полиморфизма у систему.

```

public abstract class AbstractDomainObject implements Serializable {

    public abstract String nazivTabele();
    public abstract String alias();
    public abstract String join();
    public abstract ArrayList<AbstractDomainObject> vratiListu(ResultSet rs) throws SQLException;
    public abstract String koloneZaInsert();
    public abstract String vrednostiZaInsert();
    public abstract String vrednostiZaUpdate();
    public abstract String uslov();
    public abstract String uslovZaSelect();
}

```

Слика 12: Структура класе *AbstractDomainObject*

Полиморфизам је примењен и кроз употребу интерфејса *Operation*, који садржи константе које представљају различите системске операције. Ове константе се користе у класи *ClientController* приликом слања захтева серверу, чиме се омогућава да се различите функционалности система позивају на јединствен начин. Клијентска страна не познаје конкретну имплементацију системских операција на серверу, већ комуницира са њима искључиво преко дефинисаних операција, што доприноси флексибилности и проширивости система.

```

public interface Operation {

    public static final int LOGIN = 0;
    public static final int LOGOUT = 1;

    public static final int ADD_KUPAC = 2;
    public static final int DELETE_KUPAC = 3;
    public static final int UPDATE_KUPAC = 4;
    public static final int GET_ALL_KUPAC = 5;

    public static final int ADD_COKOLADA = 6;
    public static final int DELETE_COKOLADA = 7;
    public static final int UPDATE_COKOLADA = 8;
    public static final int GET_ALL_COKOLADA = 9;

    public static final int ADD_PORUDZBINA = 10;
    public static final int DELETE_PORUDZBINA = 11;
    public static final int UPDATE_PORUDZBINA = 12;
    public static final int GET_ALL_PORUDZBINA = 13;

    public static final int GET_ALL_VRSTA_COKOLADE = 14;

    public static final int GET_ALL_GRAD = 15;

}

```

Слика 13: Интерфејс Operation

```

private ClientController() {
}

public static ClientController getInstance() {
    if (instance == null) {
        instance = new ClientController();
    }
    return instance;
}

public Administrator login(Administrator administrator) throws Exception {
    return (Administrator) sendRequest(operation:Operation.LOGIN, data: administrator);
}

public void logout(Administrator ulogovani) throws Exception {
    sendRequest(operation:Operation.LOGOUT, data: ulogovani);
}

public void addKupac(Kupac kupac) throws Exception {
    sendRequest(operation:Operation.ADD_KUPAC, data: kupac);
}

public void addCokolada(Cokolada cokolada) throws Exception {
    sendRequest(operation:Operation.ADD_COKOLADA, data: cokolada);
}

public void addPorudzbina(Porudzbina porudzbina) throws Exception {
    sendRequest(operation:Operation.ADD_PORUDZBINA, data: porudzbina);
}

public void deleteKupac(Kupac kupac) throws Exception {
    sendRequest(operation:Operation.DELETE_KUPAC, data: kupac);
}

public void deleteCokolada(Cokolada cokolada) throws Exception {
    sendRequest(operation:Operation.DELETE_COKOLADA, data: cokolada);
}

public void deletePorudzbina(Porudzbina porudzbina) throws Exception {
    sendRequest(operation:Operation.DELETE_PORUDZBINA, data: porudzbina);
}

public void updateKupac(Kupac kupac) throws Exception {
    sendRequest(operation:Operation.UPDATE_KUPAC, data: kupac);
}

public void updateCokolada(Cokolada cokolada) throws Exception {
    sendRequest(operation:Operation.UPDATE_COKOLADA, data: cokolada);
}

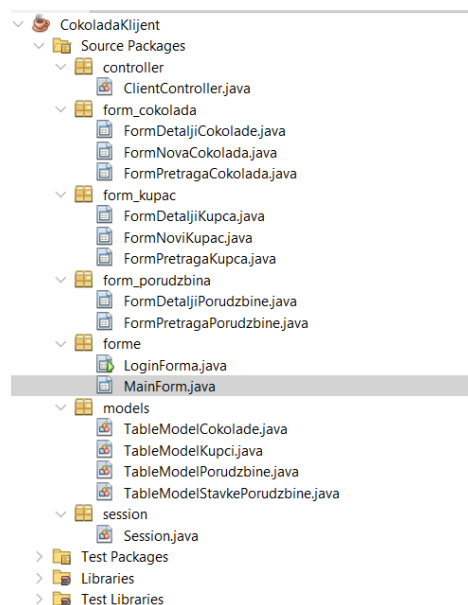
```

Слика 14: Примена интерфејса Operation у класи ClientController

3.2. Графички кориснички интерфејс (GUI)

Графички кориснички интерфејс представља важан део сваке апликације, јер омогућава кориснику интеракцију са системом. У овом пројекту коришћена је Java **Swing** библиотека (), која је део стандардне Java платформе и омогућава развој десктоп апликација са богатим графичким интерфејсом.

Swing пружа велики број компоненти као што су прозори, дугмад, текстуална поља, падајуће листе и табеле. Ове компоненте су коришћене за реализацију корисничких форми у клијентској апликацији. Реализоване су форме за пријаву корисника, као и форме за рад са чоколадама, купцима и поруџбинама.



Слика 15: Форме клијентске апликације

Све форме у клијентској апликацији, изузев главне форме и форме за пријаву, реализоване су као **JDialog**. Оваква организација омогућава да се поједине функционалности система, као што су унос, измена или приказ детаља података, извршавају у засебним дијалозима који су логички повезани са главном формом. Главна форма апликације реализована је као **JFrame** и представља централно место за навигацију кроз систем. Из ње се отварају појединачни дијалози за рад са купцима, чоколадама и поруџбинама. Форма за пријаву корисника такође је реализована као **JFrame**, јер представља почетну тачку рада апликације и није логички зависна од других форми.

Kupac Cokolada Porudzbina Odjava

Ulogovani

Porudzbina

Kupac: Item 1

Datum isporuke: 20.04.2026

Ukupan iznos:

Sacuvaj porudzbinu

Stavke porudzbine

Cokolada: Item 1

Kolicina:

Iznos:

Dodaj stavku Obrisi stavku

Title 1	Title 2	Title 3	Title 4

Слика 16: Главна форма

Поред стандардних форми и дијалога, у оквиру графичког корисничког интерфејса коришћен је и мени (**JMenuBar**), који је постављен на главној форми апликације.

Kupac Cokolada Porudzbina Odjava

☐ Novi kupac shortcut

☐ Pretraga kupca shortcut

Porudzbina

Kupac: Item 1

Datum isporuke: 20.04.2026

Ukupan iznos:

Слика 17: Мени

Посебно место у интерфејсу заузима табеларни приказ података, који је реализован коришћењем **JTable** компоненте. Табеле омогућавају кориснику да прегледа више података одједном, као и да лако одабере ставке за измену или брисање.

Pretraga (ime, prezime, email):

Title 1	Title 2	Title 3	Title 4

Detalji kupca

Слика 18: Форма за претрагу купца

У оквиру система, догађаји се најчешће обрађују кроз слушаоце (*ActionListener*, *KeyListener*), који су повезани са компонентама корисничког интерфејса. На пример, кликом на дугме за додавање ставке у поруџбину покреће се метода која проверава исправност унетих података, креира нови објекат ставке поруџбине и ажурира приказ табеле и укупног износа. На овај начин се обезбеђује да се логика обраде догађаја извршава тачно у тренутку када корисник предузме одређену акцију.

```
private void btnDodajStavkuActionPerformed(java.awt.event.ActionEvent evt) {  
  
    if (txtKolicina.getText().isEmpty()) {  
        JOptionPane.showMessageDialog(parentComponent, this, message: "Morate uneti kolicinu!");  
        return;  
    }  
  
    Cokolada c = (Cokolada) cmbCokolada.getSelectedItem();  
    int kolicina = Integer.parseInt(s: txtKolicina.getText());  
    double iznosStavke = Double.parseDouble(s: txtIznosStavke.getText());  
  
    StavkaPorudzbine sp = new StavkaPorudzbine(porudzbina: null, rb: -1, kolicina,  
        cena: c.getCena(), iznos: iznosStavke, cokolada: c);  
  
    TableModelStavkePorudzbine tm = (TableModelStavkePorudzbine) tblStavke.getModel();  
    tm.dodajStavku(sp);  
  
    ukupanIznos = tm.vratiUkupanIznos();  
    txtUkupanIznos.setText(String.valueOf(d: ukupanIznos) + "din");  
}
```

Слика 19: Код повезан са дугметом за додавање ставке

Поред обраде догађаја кликом, у систему је примењена и обрада догађаја уноса са тастатуре. Овакав приступ је коришћен, на пример, приликом претраге купаца, где се након сваког пуштања тастера ажурира приказ података у табели у складу са унетим параметром. Овим је омогућена динамичка и интуитивна интеракција са корисником, без потребе за додатним дугмадима за покретање претраге.

```
private void txtPretragaKeyReleased(java.awt.event.KeyEvent evt) {  
  
    String param = txtPretraga.getText();  
    TableModelKupci tm = (TableModelKupci) tblKupci.getModel();  
    tm.setParametar(parametar:param);  
}
```

Слика 20: Код за KeyReleased догађај

3.3. Нити

Java пружа снажну подршку за рад са нитима, што омогућава паралелно извршавање више задатака у оквиру једне апликације. Ова могућност је посебно значајна код клијент–сервер система, где сервер мора истовремено да обрађује захтеве више клијената.

3.3.1. Нити – теоријско објашњење

Нит (енг. *thread*) представља најмању јединицу извршавања у оквиру једног процеса. Један процес може да садржи једну или више нити које се извршавају паралелно или наизменично, у зависности од могућности оперативног система и хардвера. Све нити у оквиру истог процеса деле исту меморију, ресурсе и адресни простор, што омогућава брзу комуникацију, али истовремено захтева пажљиво управљање приступом заједничким подацима.

У Јава програмском језику свака апликација почиње извршавање у једној, такозваној **главној програмској нити** (*main thread*). Поред ње, могу се креирати додатне нити како би се одређени делови програма извршавали независно од главног тока извршавања. На овај начин се постиже боља искоришћеност ресурса и већа реактивност апликације, посебно код система који раде са корисничким интерфејсом, мрежом или базом података.

У Јави постоје два основна начина за креирање нити:

- наслеђивањем класе *Thread*,
- имплементацијом интерфејса *Runnable*.

Код наслеђивања класе *Thread*, креира се нова класа која садржи методу *run()*, у којој се дефинише логика коју нит извршава. Други, чешће коришћен и флексибилнији приступ, јесте имплементација интерфејса *Runnable*, где класа имплементира методу *run()*, а сама нит се покреће креирањем објекта класе *Thread* и позивом методе *start()*. Метода *start()* покреће нит и аутоматски позива методу *run()*, док директан позив *run()* не покреће нову нит, већ се код извршава у оквиру тренутне нити, што је важна разлика у односу на прави паралелизам.

Током свог животног циклуса нит може да се налази у различитим стањима:

- **New** – нит је креирана, али још увек није покренута,
- **Runnable** – нит је спремна за извршавање,
- **Running** – нит се тренутно извршава,
- **Blocked / Waiting** – нит је привремено заустављена и чека неки услов (нпр. ослобађање ресурса),
- **Terminated** – нит је завршила извршавање.

Прелазак из једног стања у друго контролише Јава виртуелна машина и оперативни систем, у зависности од услова извршавања.

Јава омогућава контролу рада нити путем одређених метода. Метода *sleep()* омогућава да се нит привремено успава на одређени временски период, након чега наставља извршавање. Ово је корисно у ситуацијама када је потребно периодично извршавање одређених операција.

Метода `interrupt()` служи за слање сигнала нити да треба да прекине или промени ток извршавања. Нит сама одлучује како ће реаговати на прекид, што омогућава контролисано и безбедно гашење нити.

Пошто више нити у истом процесу деле меморију, може доћи до проблема неконзистентности података ако више нити истовремено приступа истим ресурсима. Да би се то избегло, користи се **синхронизација**, која обезбеђује да у датом тренутку само једна нит има приступ заштићеном делу кода или подацима.

У Јави се синхронизација реализује помоћу кључне речи *synchronized*, која се може применити на методе или блокове кода. На овај начин се спречавају конфликтне ситуације и обезбеђује исправно понашање апликације у вишенитном окружењу.

Коришћење нити је посебно важно у апликацијама које имају графички кориснички интерфејс, јер омогућава да се дуготрајне операције (попут комуникације са сервером или базом података) извршавају у позадини, без блокирања рада корисника. Такође, у клијент-сервер архитектурама нити се користе за истовремено опслуживање више клијената, што значајно повећава скалабилност и перформансе система.

3.3.2. Примена нити у развоју софтверског система студијског примера

Java пружа снажну подршку за рад са нитима, што омогућава паралелно извршавање више задатака у оквиру једне апликације. Ова могућност је посебно значајна код клијент-сервер система, где сервер мора истовремено да обрађује захтеве више клијената.

У оквиру серверске апликације нити имају кључну улогу у обезбеђивању истовременог рада више клијената. Сервер мора да буде доступан за нове конекције, али и да паралелно обрађује захтеве клијената који су већ повезани. Због тога су реализоване две основне нитне компоненте: *ThreadServer* и *ThreadClient*.

Класа *ThreadServer* представља нит која иницијализује серверски сокет и непрекидно ослушкује долазне конекције. Унутар своје *run()* методе, која дефинише логику која се извршава унутар нити, она позива *accept()* методу и за сваку нову конекцију креира засебну нит типа *ThreadClient*. На овај начин, главна серверска нит остаје слободна да прихвата нове клијенте, док се комуникација са већ повезаним клијентима одвија независно.

Класа *ThreadClient* представља нит која је задужена за комуникацију са једним конкретним клијентом. У оквиру њене *run()* методе, преко објектних токова (*ObjectInputStream* и *ObjectOutputStream*) прима се *Request*, затим се на основу поља *operation* позива одговарајућа системска операција, и на крају се клијенту враћа *Response*. Овакав приступ омогућава да више клијената истовремено извршава различите системске операције без блокирања рада сервера, што повећава ефикасност и скалабилност система.

```

public class ThreadServer extends Thread {

    private ServerSocket serverSocket;

    public ThreadServer() {
        try {
            serverSocket = new ServerSocket(port: 10000);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    @Override
    public void run() {
        try {
            while (!serverSocket.isClosed()) {
                Socket socket = serverSocket.accept();
                System.out.println(x: "Klijent se povezao!");
                ThreadClient th = new ThreadClient(socket);
                th.start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public ServerSocket getServerSocket() {
        return serverSocket;
    }

    public void setServerSocket(ServerSocket serverSocket) {
        this.serverSocket = serverSocket;
    }
}

```

Слика 21: Hum ThreadServer

```

public class ThreadClient extends Thread {

    private Socket socket;

    ThreadClient(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try {
            while (!socket.isClosed()) {
                ObjectInputStream in = new ObjectInputStream(in: socket.getInputStream());
                Request request = (Request) in.readObject();
                Response response = handleRequest(request);
                ObjectOutputStream out = new ObjectOutputStream(out: socket.getOutputStream());
                out.writeObject(obj: response);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private Response handleRequest(Request request) {
        Response response = new Response(data: null, exc: null, responseStatus: ResponseStatus.Success);
        try {
            switch (request.getOperation()) {
                case Operation.ADD_KUPAC:
                    ServerController.getInstance().addKupac((Kupac) request.getData());
                    break;
                case Operation.ADD_COKOLADA:
                    ServerController.getInstance().addCokolada((Cokolada) request.getData());
                    break;
                case Operation.ADD_PORUDZBINA:
                    ServerController.getInstance().addPorudzbina((Porudzbina) request.getData());
                    break;
                case Operation.DELETE_KUPAC:
                    ServerController.getInstance().deleteKupac((Kupac) request.getData());
                    break;
                case Operation.DELETE_COKOLADA:
                    ServerController.getInstance().deleteCokolada((Cokolada) request.getData());
                    break;
            }
        }
    }
}

```

Слика 22: Hum ThreadClient

На клијентској страни нити су искоришћене пре свега ради побољшања корисничког искуства и спречавања „замрзавања“ графичког интерфејса. Како се подаци преузимају са сервера преко мреже, постоји могућност да позив траје дуже. Уколико би се такве операције извршавале директно у EDT (*Event Dispatch Thread*) нити која је задужена за GUI, интерфејс би постао неактиван током преузимања података.

```
public class TableModelCokolade extends AbstractTableModel implements Runnable {

    private ArrayList<Cokolada> lista;
    private String[] kolone = {"ID", "Naziv", "Opis", "Cena", "Vrsta"};
    private String parametar = "";

    public TableModelCokolade() {
        try {
            lista = ClientController.getInstance().getAllCokolada();
        } catch (Exception ex) {
            Logger.getLogger(name: TableModelCokolade.class.getName()).log(level: Level.SEVERE, msg: null, thrown: ex);
        }
    }
}
```

Слика 23: Нит за модел табеле чоколада

```
@Override
public void run() {
    try {
        while (!Thread.currentThread().isInterrupted()) {
            Thread.sleep(millis:10000);
            refreshTable();
        }
    } catch (InterruptedException ex) {
        Logger.getLogger(name: TableModelCokolade.class.getName()).log(level: Level.SEVERE, msg: null, thrown: ex);
    }
}

public void setParametar(String parametar) {
    this.parametar = parametar;
    refreshTable();
}

public void refreshTable() {
    try {
        lista = ClientController.getInstance().getAllCokolada();
        if (!parametar.equals(anObject: "")) {
            ArrayList<Cokolada> novaLista = new ArrayList<>();
            for (Cokolada c : lista) {
                if (c.getNaziv().toLowerCase().contains(s: parametar.toLowerCase())
                    || c.getOpis().toLowerCase().contains(s: parametar.toLowerCase())) {
                    novaLista.add(e: c);
                }
            }
            lista = novaLista;
        }

        fireTableDataChanged();
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}
```

Слика 24: Нит за модел табеле чоколада

Због тога су класе модела табела *TableModelCokolade*, *TableModelKupci* и *TableModelPorudzbine* реализоване тако да, поред наслеђивања *AbstractTableModel*, имплементирају и интерфејс *Runnable*. То значи да се за ове моделе покреће посебна нит (нпр. у конструктору форме се креира *new Thread(model).start()*), која периодично освежава податке позивом методе *refreshTable()*. Освежавање се извршава у одређеном временском интервалу (нпр. коришћењем *Thread.sleep()*), након чега се табела ажурира позивом *fireTableDataChanged()*.

```

1  //
2  public class FormPretragaCokolada extends javax.swing.JDialog {
3
4      public FormPretragaCokolada(java.awt.Frame parent, boolean modal) {
5          super(owner: parent, modal);
6          initComponents();
7          setLocationRelativeTo(c: null);
8          setTitle(title: "Pretraga cokolade");
9          TableModelCokolade model = new TableModelCokolade();
10         Thread th = new Thread(task: model);
11         th.start();
12         tblCokolade.setModel(dataModel: model);
13     }
14 }

```

Слика 25: Позив нити у конструктору форме

3.4. Мрежна комуникација – сокети

Мрежна комуникација представља један од основних механизма за размену података између различитих софтверских компоненти које се извршавају на различитим рачунарима или у различитим процесима. У оквиру клијент-сервер архитектуре, мрежна комуникација омогућава да клијентска апликација шаље захтеве серверу, док сервер обрађује те захтеве и враћа одговарајуће резултате. У програмском језику Java, мрежна комуникација се најчешће реализује коришћењем сокета.

3.4.1. Сокети – теоријско објашњење

Сокет представља крајњу тачку (*endpoint*) у мрежној комуникацији и омогућава двосмерну размену података између два процеса који комуницирају преко мреже. Свака комуникација путем сокета одвија се између две адресе које су дефинисане комбинацијом **IP адресе** и **броја порта**.

IP адреса служи за идентификацију рачунара у мрежи, док порт представља логички идентификатор процеса или сервиса који на том рачунару прихвата комуникацију. На тај начин је могуће да више различитих апликација истовремено користи мрежу, а да се комуникација јасно усмерава ка одговарајућем сервису.

Код комуникације засноване на сокетима најчешће се користи клијент-сервер модел. Серверска страна има задатак да отвори одређени порт и послушкује долазне захтеве клијената, док клијент иницира комуникацију повезивањем на познату IP адресу и порт сервера.

Серверска апликација најпре креира објекат типа **ServerSocket**, који је везан за одређени порт. Након тога, сервер чека долазне захтеве клијената. Када клијент успостави везу, сервер прихвата ту везу и за комуникацију са конкретним клијентом користи објекат типа **Socket**.

Са друге стране, клијентска апликација користи класу **Socket** како би се повезала на сервер, наводећи IP адресу сервера и број порта. Након успостављања везе, клијент и сервер могу да размењују податке све док је сокет отворен.

Комуникација између клијента и сервера преко сокета реализује се помоћу улазних и излазних токова података. За читање података који пристижу преко сокета користи се **InputStream**, док се за слање података користи **OutputStream**.

У програмском језику Java, ови токови се често додатно обавијају вишим нивоима апстракције, као што су **ObjectInputStream** и **ObjectOutputStream**, како би се омогућила размена објеката. На тај начин је могуће слати комплексне структуре података, а не само примитивне типове или низове бајтова.

Процес комуникације се обично одвија тако што клијент преко излазног тока шаље захтев серверу, након чега сервер преко улазног тока чита захтев, обрађује га и затим преко излазног тока шаље одговор назад клијенту.

Комуникација заснована на сокетима је по својој природи најчешће синхрона, што значи да једна страна чека одговор друге стране након што пошаље захтев. Животни циклус једне комуникације обухвата успостављање везе, размену података и затварање сокета.

Важно је водити рачуна о правилном затварању сокета и токова података како би се ослободили системски ресурси и спречило настајање грешака у раду апликације. Такође, у реалним системима је уобичајено да серверска страна користи више нити како би могла истовремено да опслужује више клијената.

Сокети представљају основу за изградњу дистрибуираних и мрежних апликација. Они омогућавају јасну поделу одговорности између клијентске и серверске стране, као и централизовану обраду података на серверу. Захваљујући томе, логика система може бити смештена на једном месту, док клијенти представљају само кориснички интерфејс и механизам за комуникацију са сервером.

У савременим софтверским системима, комуникација путем сокета често представља основу за имплементацију сложенијих протокола и сервиса, као што су веб сервиси и апликациони сервери.

3.4.2. Примена сокета у развоју софтверског система студијског примера

Клијент–сервер архитектура овог система заснива се на мрежној комуникацији између клијентске и серверске апликације. За реализацију комуникације коришћени су Java сокети, односно класе **ServerSocket** и **Socket**, које омогућавају размену података преко *TCP* протокола.

На серверској страни, покретањем класе *ThreadServer* иницијализује се објекат *ServerSocket* који „слуша“ на одређеном порту. У овом систему сервер је подешен да користи порт **10000**. Порт представља логички канал на рачунару који омогућава да више различитих апликација истовремено користи мрежу, а да се не мешају међусобно.

На клијентској страни, у класи *Session* се приликом креирања инстанце успоставља конекција ка серверу кроз *new Socket(host, port)*. Као адреса сервера коришћен је *localhost*, што означава да су клијент и сервер покренути на истој машини (локално окружење). У реалним условима, уместо *localhost* би се користила конкретна IP адреса сервера (нпр. 192.168.x.x у локалној мрежи или јавна IP адреса), док порт остаје исти све док је тако конфигурисан на серверу.

```
public class Session {  
  
    private static Session instance;  
    private Socket socket;  
    private Administrator ulogovani;  
    private MainForm mf;  
  
    private Session() {  
        try {  
            socket = new Socket(host: "localhost", port: 10000);  
        } catch (IOException ex) {  
            // ex.printStackTrace();  
        }  
    }  
}
```

Слика 26: Структура класе *Session*

Након што се конекција успостави, комуникација се одвија тако што клијент шаље захтев серверу, сервер захтев обрађује и враћа одговор. У систему је уведен једноставан модел комуникације заснован на објектима, где се користе класе *Request* и *Response*.

Клијентска страна у оквиру методе *sendRequest* (у класи *ClientController*) креира *Request* објекат који садржи:

- идентификатор операције (нпр. *Operation.ADD_COKOLADA*, *Operation.GET_ALL_COKOLADA*, итд.)
- податке који се шаљу серверу (нпр. објекат *Cokolada*, *Porudzbina*, *Kupac...*)

Затим се захтев шаље преко излазног тока:

- *getOutputStream()* враћа бајтни излазни ток сокета
- преко *ObjectOutputStream* омогућено је серијализовано слање целих Java објеката

Након слања захтева, клијент чита одговор од сервера преко улазног тока:

- *getInputStream()* враћа бајтни улазни ток сокета
- *ObjectInputStream* омогућава десеријализацију примљеног објекта у *Response*

```

private Object sendRequest(int operation, Object data) throws Exception {
    Request request = new Request(operation, data);

    ObjectOutputStream out = new ObjectOutputStream(out: Session.getInstance().getSocket().getOutputStream());
    out.writeObject(obj: request);

    ObjectInputStream in = new ObjectInputStream(in: Session.getInstance().getSocket().getInputStream());
    Response response = (Response) in.readObject();

    if (response.getResponseStatus().equals(other: ResponseStatus.Error)) {
        throw response.getException();
    } else {
        return response.getData();
    }
}
}

```

Слика 27: Метода која шаље клијентски захтев

Уколико сервер врати статус грешке (нпр. *ResponseStatus.Error*), клијентска страна подиже изузетак који је стигао у оквиру одговора, чиме се омогућава да се грешке из пословне логике прикажу кориснику на јасан начин (нпр. кроз поруке у GUI-ју). Ако је статус успешан, клијент преузима податке из одговора и наставља даље извршавање.

Серверска апликација је реализована тако да омогућава рад са више клијената. Класа *ThreadServer* отвара *ServerSocket* и непрекидно позива *accept()*. Сваки пут када се клијент повеже, *accept()* враћа нови *Socket* који представља конекцију са конкретним клијентом. За ту конекцију се покреће нова нит *ThreadClient*.

Класа *ThreadClient* је задужена за комуникацију са једним клијентом. Унутар њене *run()* методе, све док сокет није затворен, сервер:

1. чита *Request* објекат преко *ObjectInputStream*
2. на основу операције у методи *handleRequest()* одређује коју системску операцију треба извршити (нпр. додавање чоколаде, измена поруџбине...)
3. резултат извршавања пакује у *Response* (податак + статус + евентуални изузетак)
4. шаље *Response* назад клијенту преко *ObjectOutputStream*

```

public class ThreadClient extends Thread {

    private Socket socket;

    ThreadClient(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try {
            while (!socket.isClosed()) {
                ObjectInputStream in = new ObjectInputStream(in: socket.getInputStream());
                Request request = (Request) in.readObject();
                Response response = handleRequest(request);
                ObjectOutputStream out = new ObjectOutputStream(out: socket.getOutputStream());
                out.writeObject(obj: response);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private Response handleRequest(Request request) {
        Response response = new Response(data: null, exc: null, responseStatus: ResponseStatus.Success);
        try {
            switch (request.getOperation()) {
                case Operation.ADD_KUPAC:
                    ServerController.getInstance().addKupac((Kupac) request.getData());
                    break;
                case Operation.ADD_COKOLADA:
                    ServerController.getInstance().addCokolada((Cokolada) request.getData());
                    break;
                case Operation.ADD_PORUDZBINA:
                    ServerController.getInstance().addPorudzbina((Porudzbina) request.getData());
                    break;
                case Operation.DELETE_KUPAC:
                    ServerController.getInstance().deleteKupac((Kupac) request.getData());
                    break;
                case Operation.DELETE_COKOLADA:
                    ServerController.getInstance().deleteCokolada((Cokolada) request.getData());
                    break;
            }
        }
    }
}

```

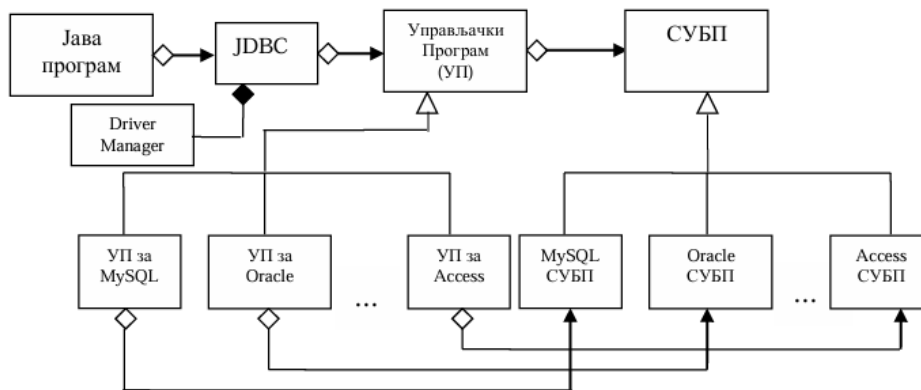
Слика 28: Класа ThreadClient

Како би се Java објекти (нпр. Cokolada, Kupac, Porudzbina) могли слати преко мреже, потребно је да буду серијализовани. У систему је то омогућено имплементацијом интерфејса *Serializable* у доменским класама (посредно преко *AbstractDomainObject*). На тај начин се објекти могу претворити у бајтовни ток и послати преко сокета, након чега се на другој страни поново реконструишу у оригинални објекат.

3.5. Базе података – JDBC

У оквиру ове апликације за складиштење и обраду података коришћен је систем за управљање базом података **MySQL**, док је за рад са базом у току развоја и администрације коришћен алат **SQLyog**. Комуникација између Java апликације и базе података реализована је коришћењем **JDBC (Java Database Connectivity)** технологије, која представља стандардни Java API за приступ релационим базама података.

JDBC омогућава апликацији да успостави везу са базом података, извршава SQL упите, као и да обрађује добијене резултате. У оквиру рада коришћене су основне JDBC класе и интерфејси, као што су **DriverManager**, **Connection**, **Statement**, **PreparedStatement** и **ResultSet**.



Слика 29: Веза Јава програма са СУБП [1]

Конекција са базом података успоставља се у класи **DBBroker**, која је реализована по **Singleton** шаблону, чиме је обезбеђено да у апликацији постоји само једна активна конекција ка бази.

```

public class DBBroker {

    private static DBBroker instance;
    private Connection connection;
    private ArrayList<AbstractDomainObject> testSelectResult;

    private DBBroker() {
        try {
            Properties properties = new Properties();
            properties.load(new FileInputStream(name: "dbconfig.properties"));
            String url = properties.getProperty(key: "url");
            String username = properties.getProperty(key: "username");
            String password = properties.getProperty(key: "password");
            connection = DriverManager.getConnection(url, user: username, password);
            connection.setAutoCommit(autoCommit: false);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    public Connection getConnection() {
        return connection;
    }

    public void setConnection(Connection connection) {
        this.connection = connection;
    }

    public static DBBroker getInstance() {
        if (instance == null) {
            instance = new DBBroker();
        }
        return instance;
    }
}

```

Слика 30: Класа DBBroker

Подаци неопходни за повезивање са базом (URL, корисничко име и лозинка) смештени су у посебном конфигурационом фајлу **dbconfig.properties**, што омогућава лаку измену параметара без потребе за променом изворног кода.

```

password=
url=jdbc:mysql://localhost:3306/cokolada
username=root

```

Слика 31: Датотека за конфигурацију базе

У овом фајлу се налази JDBC URL, где је *localhost* IP адреса сервера, *3306* подразумевани порт за MySQL сервер, а *sokolada* назив базе података.

У класи **DBBroker**, приликом креирања инстанце, помоћу класе **DriverManager** успоставља се конекција са базом података:

- учитавају се параметри из *properties* фајла,
- позива се метода *DriverManager.getConnection()*,
- подешава се режим рада са трансакцијама искључивањем аутоматског *commit*-а (*setAutoCommit(false)*).

На овај начин омогућена је експлицитна контрола над трансакцијама, што је посебно важно код сложенијих системских операција које обухватају више SQL наредби.

Контрола трансакција реализована је у апстрактној класи **AbstractSO**, која представља основу за све системске операције. Метод *templateExecute()* у овој класи обезбеђује следећи ток извршавања:

1. позив методе *validate()* ради провере исправности улазних података,
2. извршавање конкретне пословне логике у методу *execute()*,
3. потврду трансакције позивом *commit()* у случају успешног извршења,
4. поништавање трансакције (*rollback()*) у случају настанка изузетка.

Оваква реализација омогућава очување интегритета података у бази и спречава делимичан упис података у случају грешке.

За извршавање SQL упита коришћене су различите JDBC класе у зависности од типа операције:

- **Statement** – за једноставне SQL наредбе,
- **PreparedStatement** – за INSERT операције, уз могућност повратка генерисаних кључева,
- **ResultSet** – за обраду резултата SELECT упита.

Свака доменска класа у систему наслеђује апстрактну класу **AbstractDomainObject**, која дефинише методе за динамичко формирање SQL упита, као што су назив табеле, услови за селекцију, колоне за унос и вредности за упис. На овај начин постигнута је флексибилност и поновна употребљивост кода приликом рада са базом података.

```

    public void setSelectResult(ArrayList<AbstractDomainObject> result) {
        this.testSelectResult = result;
    }

    public ArrayList<AbstractDomainObject> select(AbstractDomainObject ado) throws SQLException {
        if (testSelectResult != null) {
            return testSelectResult;
        }

        String upit = "SELECT * FROM " + ado.nazivTabele() + " " + ado.aliijas()
            + " " + ado.join() + " " + ado.uslovZaSelect();
        System.out.println(upit + "\n");
        Statement s = connection.createStatement();
        ResultSet rs = s.executeQuery(sql:upit);
        return ado.vratiListu(rs);
    }

    public PreparedStatement insert(AbstractDomainObject ado) throws SQLException {
        String naredba = "INSERT INTO " + ado.nazivTabele() + " "
            + ado.koloneZaInsert() + " VALUES(" + ado.vrednostiZaInsert() + ")";
        System.out.println(naredba + "\n");
        PreparedStatement ps = connection.prepareStatement(sql:naredba, autoGeneratedKeys: Statement.RETURN_GENERATED_KEYS);
        ps.executeUpdate();
        return ps;
    }

    public void update(AbstractDomainObject ado) throws SQLException {
        String naredba = "UPDATE " + ado.nazivTabele() + " SET "
            + ado.vrednostiZaUpdate() + " WHERE " + ado.uslov();
        System.out.println(naredba + "\n");
        Statement s = connection.createStatement();
        s.executeUpdate(sql:naredba);
    }

    public void delete(AbstractDomainObject ado) throws SQLException {
        String naredba = "DELETE FROM " + ado.nazivTabele() + " WHERE " + ado.uslov();
        System.out.println(naredba + "\n");
        Statement s = connection.createStatement();
        s.executeUpdate(sql:naredba);
    }
}

```

Слика 32: Методе за SQL ynume

4. Студијски пример

4.1. Фаза прикупљања корисничких захтева

4.1.1 Вербални опис

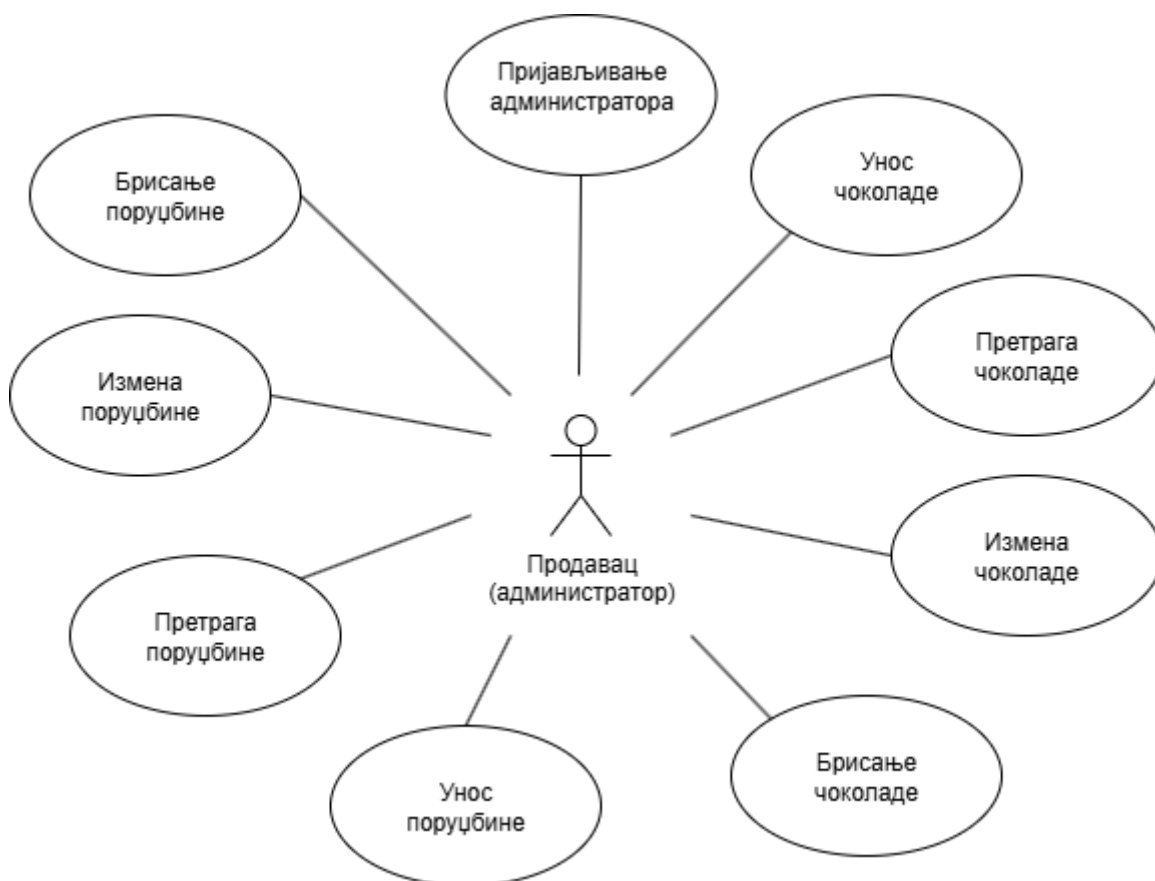
Потребно је направити софтверски систем који ће се користити за потребе продавнице чоколаде. Систем треба да омогући пријаву администратора на систем (у овом случају, улога администратора припада запосленом у продавници), затим евиденцију података о производима (нпр. врста чоколаде, опис, количина на лагеру, цена). Поред тога, систем треба да омогући евиденцију купаца и њихових поруџбина, као и заказивање нових наруџбина приликом којих је потребно назначити који купац наручује, који производ жели да купи, у којој количини, као и датум преузимања или испоруке. Такође, за сваког купца потребно је водити евиденцију претходно купљених производа и услуга.

Софтверски систем треба додатно да омогући администратору (запослени са улогом актора), да поред креирања нових производа и купаца врши претрагу и измену података о постојећим производима и купцима, претрагу по поруџбинама, као и брисање наруџбина уколико купац одустане од куповине.

4.1.2 Случајеви коришћења

Идентификовани случајеви коришћења:

1. Пријава администратора
2. Унос чоколаде
3. Претрага чоколаде
4. Измена чоколаде
5. Брисање чоколаде
6. Унос поруџбине (сложен СК)
7. Претрага поруџбине
8. Измена поруџбине (сложен СК)
9. Брисање поруџбине



Слика 33. Случајеви коришћења

СК1: Случај коришћења - Пријава администратора

Назив СК

Пријављивање **администратора**

Актори СК

Администратор

Учесници СК

Администратор и **систем** (програм)

Предуслов: **Систем** је укључен и приказује форму за пријављивање **администратора**.

Основни сценарио СК

1. **Администратор** уноси податке за пријаву. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке за пријаву. (АНСО)
3. **Администратор** позива **систем** да пронађе **администратора** са задатим подацима. (АПСО)
4. **Систем** претражује **администратора**. (СО)
5. **Систем** приказује поруку: „Успешно сте се пријавили на **систем**“. (ИА)

Алтернативна сценарија

- 5.1. Уколико **систем** није пронашао **администратора**, приказује **администратору** поруку: „**Систем** не може да пронађе **администратора** на основу унетих података“. (ИА)

СК2: Случај коришћења - Унос чоколаде

Назив СК

Унос нове чоколаде

Актери СК

Администратор

Учесници СК

Администратор и **систем** (програм)

Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за рад са **чоколадама**. Учитана је листа врсти чоколада.

Основни сценарио СК

1. **Администратор** уноси податке о **чоколади**. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке о **чоколади**. (АНСО)
3. **Администратор** позива **систем** да креира нову **чоколаду**. (АПСО)
4. **Систем** креира нову **чоколаду**. (СО)
5. **Систем** приказује поруку: „**Систем** је успешно креирао нову **чоколаду**“. (ИА)

Алтернативна сценарија

- 5.1. Уколико **систем** не може да креира нову **чоколаду**, он приказује **администратору** поруку: „**Систем** не може да креира нову **чоколаду**“. (ИА)

СК3: Случај коришћења - Претрага чоколаде

Назив СК

Претрага чоколаде

Актери СК

Администратор

Учесници СК

Администратор и **систем** (програм)

Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за претраживање чоколаде. Учитана је листа чоколада.

Основни сценарио СК

1. **Администратор** уноси вредност по којој претражује чоколаде.(АПУСО)
2. **Администратор** позива **систем** да нађе чоколаду по задатој вредности.(АПСО)
3. **Систем** тражи чоколаде по задатој вредности.(СО)
4. **Систем** приказује **администратору** податке о чоколадама и поруку: „**Систем** је нашао чоколаде по задатој вредности.”.(ИА)
5. **Администратор** бира чоколаду коју жели да му **систем** прикаже. (АПУСО)
6. **Администратор** позива **систем** да прочита чоколаду. (АПСО)
7. **Систем** учитава чоколаду. (СО)
8. **Систем** приказује **администратору** податке о чоколади и поруку: „**Систем** је прочитао чоколаду“.(ИА)

Алтернативна сценарија

- 4.1 Уколико **систем** не може да нађе чоколаде по задатој вредности, он приказује **администратору** поруку: „**Систем** не може да нађе чоколаде по задатој вредности.”. Прекида се извршење сценарија. (ИА)
- 8.1. Уколико **систем** не може да прочита чоколаду он приказује **администратору** поруку: „**Систем** не може да прочита чоколаду.”.(ИА)

СК4: Случај коришћења - Измена чоколаде

Назив СК

Измена података чоколаде

Актери СК

Администратор

Учесници СК

Администратор и **систем** (програм)

Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за рад са чоколадама. Учитана је листа чоколада.

Основни сценарио СК

1. **Администратор** уноси вредност по којој претражује чоколаде.(АПУСО)
2. **Администратор** контролише да ли је коректно унео податке за претрагу чоколада. (АНСО)
3. **Администратор** позива **систем** да пронађе чоколаде по задатој вредности.(АПСО)
4. **Систем** тражи чоколаде по задатој вредности.(СО)
5. **Систем** приказује податке о траженим чоколадама уз поруку: „Подаци о траженим чоколадама су успешно читани”.(ИА)
6. **Администратор** бира чоколаду коју жели да измени. (АПУСО)
7. **Администратор** позива **систем** да пронађе изабрану чоколаду. (АПСО)
8. **Систем** проналази изабрану чоколаду. (СО)
9. **Систем** приказује податке о траженој чоколади уз поруку: „**Систем** је прочитао чоколаду”.(ИА)
10. **Администратор** мења податке о чоколади.(АПУСО)
11. **Администратор** контролише да ли је коректно унео податке о чоколади.(АНСО)
12. **Администратор** позива **систем** да запамти податке о чоколади.(АПСО)
13. **Систем** памти податке о чоколади.(СО)
14. **Систем** приказује **администратору** измењену чоколаду уз поруку: „**Систем** је изменио податке чоколаде”. (ИА)

Алтернативна сценарија

5.1. Уколико **систем** не може да пронађе чоколаде, приказује следећу поруку **администратору**: „**Систем** не може да пронађе чоколаде на основу унетих података”. Прекида се извршење сценарија. (ИА)

9.1 Уколико **систем** не може да прикаже податке о чоколади, приказује следећу поруку **администратору**: „**Систем** не може да пронађе чоколаду“. Прекида се извршење сценарија. (ИА)

14.1. Уколико **систем** не може да измени чоколаду, он приказује следећу поруку **администратору**: „**Систем** не може да измени чоколаду”. (ИА)

СК5: Случај коришћења - Брисање чоколаде

Назив СК

Брисање чоколаде

Актери СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за рад са чоколадама. Учитана је листа чоколаде.

Основни сценарио СК

1. Администратор уноси вредност по којој претражује чоколаде. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за претрагу чоколаде. (АНСО)
3. Администратор позива систем да пронађе чоколаде по задатој вредности. (АПСО)
4. Систем тражи чоколаде по задатој вредности. (СО)
5. Систем приказује чоколаде администратору и поруку: „Систем је пронашао чоколаде по задатој вредности“. (ИА)
6. Администратор бира чоколаду коју жели да избрише. (АПУСО)
7. Администратор позива систем да нађе избрану чоколаду. (АПСО)
8. Систем проналази избрану чоколаду. (СО)
9. Систем приказује чоколаду и поруку: „Систем је пронашао чоколаду “. (ИА)
10. Администратор позива систем да обрише чоколаду. (АПСО)
11. Систем брише чоколаду. (СО)
12. Систем приказује администратору поруку: „Систем је обрисао чоколаду “. (ИА)

Алтернативна сценарија:

- 5.1. Уколико систем не може да нађе чоколаде, он приказује администратору поруку: „Систем не може да пронађе чоколаде по задатој вредности“. Прекида се извршење сценарија. (ИА)
- 9.1. Уколико систем не може да нађе чоколаду, он приказује администратору поруку: „Систем не може да прочита чоколаду.“. Прекида се извршење сценарија. (ИА)
- 12.1. Уколико систем не може да обрише чоколаду он приказује администратору поруку: „Систем не може да обрише чоколаду “. (ИА)

СК6: Случај коришћења - Унос поруџбине (сложен СК)

Назив СК

Унос **поруџбине**

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за рад са **поруџбинама**. Учитане су листе купаца и чоколада.

Основни сценарио СК

1. **Администратор** уноси податке о **поруџбини**. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке о **поруџбини**. (АНСО)
3. **Администратор** позива систем да креира **поруџбину**. (АПСО)
4. **Систем** креира **поруџбину**. (СО)
5. **Систем** приказује **администратору** креирану **поруџбину** и поруку: „**Систем** је креирао **поруџбину**.”. (ИА)

Алтернативна сценарија:

- 5.1. Уколико **систем** не може да креира **поруџбину** он приказује **администратору** поруку: „**Систем** не може да креира **поруџбину**“.(ИА)

СК7: Случај коришћења - Претрага поруџбине

Назив СК

Претрага поруџбине

Актери СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за претраживање поруџбина. Учитана је листа поруџбина.

Основни сценарио СК

1. Администратор уноси вредност по којој претражује поруџбине. (АПУСО)
2. Администратор позива систем да нађе поруџбине по задатој вредности. (АПСО)
3. Систем тражи поруџбине по задатој вредности. (СО)
4. Систем приказује администратору податке о поруџбинама и поруку: „Систем је нашао поруџбине по задатој вредности.”. (ИА)
5. Администратор бира поруџбину коју жели да му систем прикаже. (АПУСО)
6. Администратор позива систем да прочита поруџбину. (АПСО)
7. Систем учитава поруџбину. (СО)
8. Систем приказује администратору податке о поруџбини и поруку: „Систем је прочитао поруџбину.”. (ИА)

Алтернативна сценарија:

- 4.1. Уколико систем не може да нађе поруџбине по задатој вредности, он приказује администратору поруку: „Систем не може да нађе поруџбине по задатој вредности.”. Прекида се извршење сценарија. (ИА)
- 8.1 Уколико систем не може да прочита поруџбину приказује администратору поруку: „Систем не може да прочита поруџбину”.(ИА)

СК8: Случај коришћења - Измена поруџбине (сложен СК)

Назив СК

Измена **поруџбине**

Актори СК

Администратор

Учесници СК

Администратор и **систем** (програм)

Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за рад са **поруџбинама**. Учитане су листе поруџбина и чоколада.

Основни сценарио СК

1. **Администратор** уноси вредност по којој претражује **поруџбине**. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке за претрагу **поруџбина**. (АНСО)
3. **Администратор** позива **систем** да нађе **поруџбине** по задатој вредности. (АПСО)
4. **Систем** тражи **поруџбине** по задатој вредности. (СО)
5. **Систем** приказује **администратору** **поруџбине** и поруку: „**Систем** је нашао **поруџбине** по задатој вредности.“. (ИА)
6. **Администратор** бира **поруџбину** коју жели да измени. (АПУСО)
7. **Администратор** позива **систем** да нађе изабрану **поруџбину**. (АПСО)
8. **Систем** проналази изабрану **поруџбину**. (СО)
9. **Систем** приказује **администратору** **поруџбину** и поруку: „**Систем** је прочитао **поруџбину**“. (ИА)
10. **Администратор** мења податке о **поруџбини**. (АПУСО)
11. **Администратор** контролише да ли је коректно унео податке о **поруџбини**. (АНСО)
12. **Администратор** позива **систем** да запамти податке о **поруџбини**. (АПСО)
13. **Систем** памти податке о **поруџбини**. (СО)
14. **Систем** приказује **администратору** поруку: „**Систем** је изменио **поруџбину**“. (ИА)

Алтернативна сценарија:

- 5.1 Уколико **систем** не може да пронађе **поруџбине** он приказује **администратору** поруку: „**Систем** не може да пронађе **поруџбине** по задатој вредности“. Прекида се извршење сценарија. (ИА)
- 9.1. Уколико **систем** не може да прикаже податке о **поруџбини**, приказује следећу поруку **администратору**: „**Систем** не може учита **поруџбину**“. Прекида се извршење сценарија. (ИА)
- 14.1 Уколико **систем** не може да измени податке о **поруџбини** он приказује **администратору** поруку: „**Систем** не може да измени **поруџбину**“. (ИА)

СК9: Случај коришћења - Брисање поруџбине

Назив СК

Брисање поруџбине

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за рад са поруџбинама. Учитана је листа поруџбина.

Основни сценарио СК

1. Администратор уноси вредност по којој претражује поруџбине. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за претрагу поруџбина. (АНСО)
3. Администратор позива систем да нађе поруџбине по задатој вредности. (АПСО)
4. Систем тражи поруџбине по задатој вредности. (СО)
5. Систем приказује поруџбине администратору и поруку: „Систем је нашао поруџбине по задатој вредности“. (ИА)
6. Администратор бира поруџбину који жели да избрише. (АПУСО)
7. Администратор позива систем да нађе изабрану поруџбину. (АПСО)
8. Систем проналази изабрану поруџбину. (СО)
9. Систем приказује администратору поруџбину и поруку: „Систем је прочитао поруџбину“. (ИА)
10. Администратор позива систем да обрише поруџбину. (АПСО)
11. Систем брише поруџбину. (СО)
12. Систем приказује администратору поруку: „Систем је обрисао поруџбину“. (ИА)

Алтернативна сценарија:

5.1 Уколико систем не може да нађе поруџбине, он приказује администратору поруку: „Систем не може да нађе поруџбине по задатој вредности“. Прекида се извршење сценарија. (ИА)

9.1 Уколико систем не може да прикаже податке о поруџбини, он приказује администратору поруку: „Систем не може да прочита поруџбину.“. Прекида се извршење сценарија. (ИА)

- 12.1 Уколико систем не може да обрише поруџбину он приказује администратору поруку: „Систем не може да обрише поруџбину.“. (ИА)

4.2. Фаза анализе

У фази анализе описујемо логичку структуру и понашање софтвера. Заправо, резултат фазе анализе јесте *пословна логика* софтверског система. Понашање софтверског система се описује помоћу *дијаграма секвенци* и *системских операција*, док се структура система описује помоћу *концептуалног* и *релационог* модела.

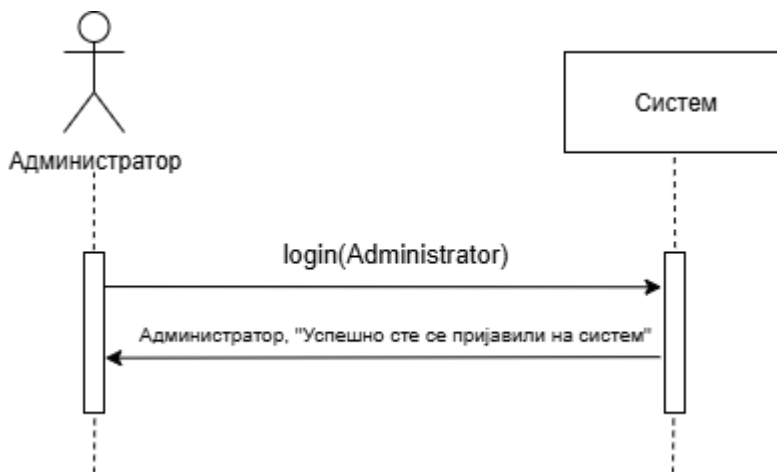
4.2.1. Понашање софтверског система

Понашање софтверског система приказујемо путем системских дијаграма секвенци, где ћемо за сваки *случај коришћења*, уочен у *фази прикупљања захтева*, дати дијаграм секвенци. Системски дијаграм секвенци треба да прикаже интеракцију између актора и система, путем активности у одређеном редоследу. За сваки сценарио се праве дијаграми и то само за **АПСО** и **ИА** акције.

ДС1: Дијаграм секвенци случаја коришћења – Пријављивање администратора

Основни сценарио СК

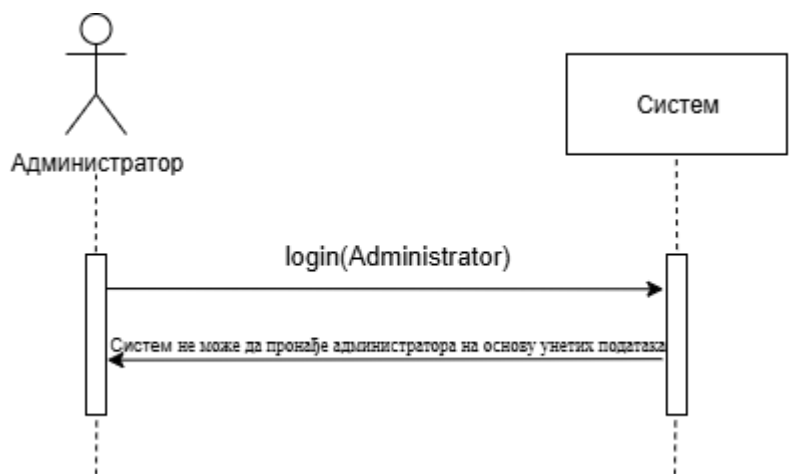
1. **Администратор** позива **систем** да пронађе **администратора** са задатим подацима. (АПСО)
2. **Систем** приказује поруку: „Успешно сте се пријавили на **систем**“. (ИА)



Дијаграм 1: ДС Пријављивање администратора (основни сценарио)

Алтернативна сценарија

- 1.1. Уколико **систем** није пронашао **администратора**, приказује **администратору** поруку: „**Систем** не може да пронађе **администратора** на основу унетих података“. (ИА)



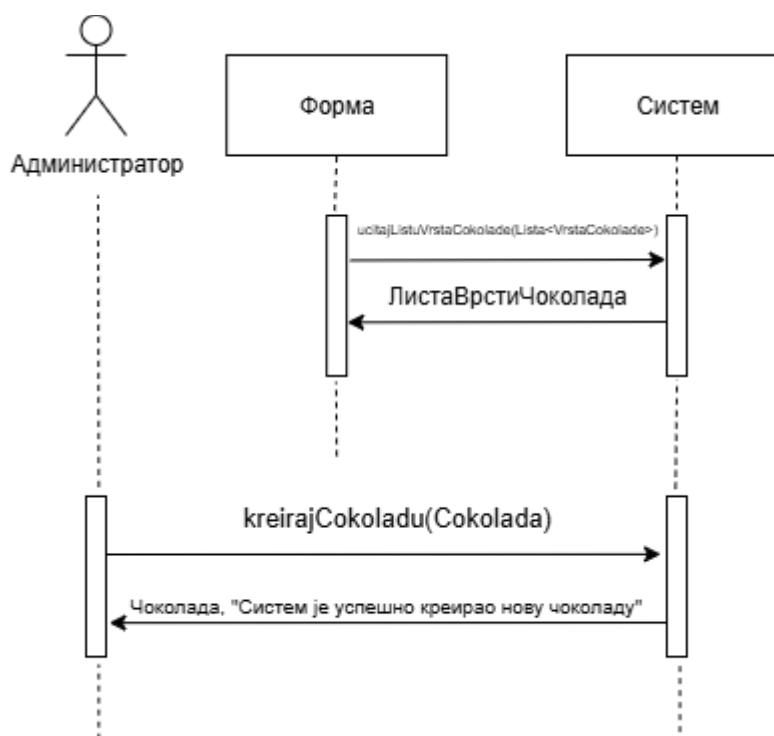
Дијаграм 2: ДС Пријављивање администратора (алтернативни сценарио 1)

Са наведених секвенцих дијаграма уочава се једна системска операција коју треба пројектовати:

1. *signal* **login(Administrator)**

Основни сценарио СК

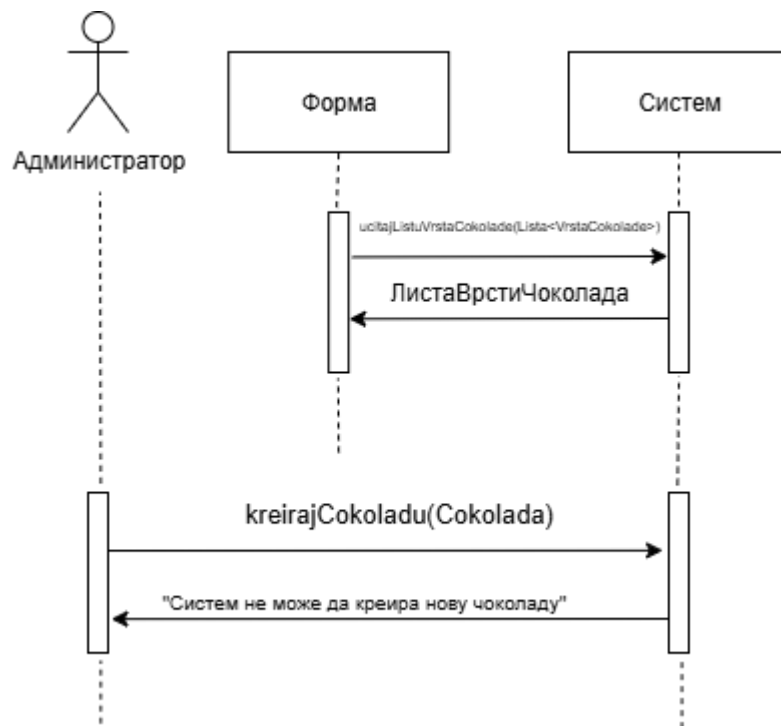
1. **Форма** **позива** систем да врати **листу врста чоколада**. (АПСО)
2. **Систем** **враћа** форми **листу врста чоколада**. (ИА)
3. **Администратор** **позива** **систем** да креира нову **чоколаду**. (АПСО)
4. **Систем** **приказује** **администратору** креирану чоколаду и поруку: „**Систем** је успешно креирао нову **чоколаду**.”. (ИА)



Дијаграм 3: ДС Унос нове чоколаде (основни сценарио)

Алтернативна сценарија СК

- 4.1. Уколико **систем** не може да креира нову **чоколаду** он приказује **администратору** поруку: „**Систем** не може да креира нову **чоколаду**.”. (ИА)



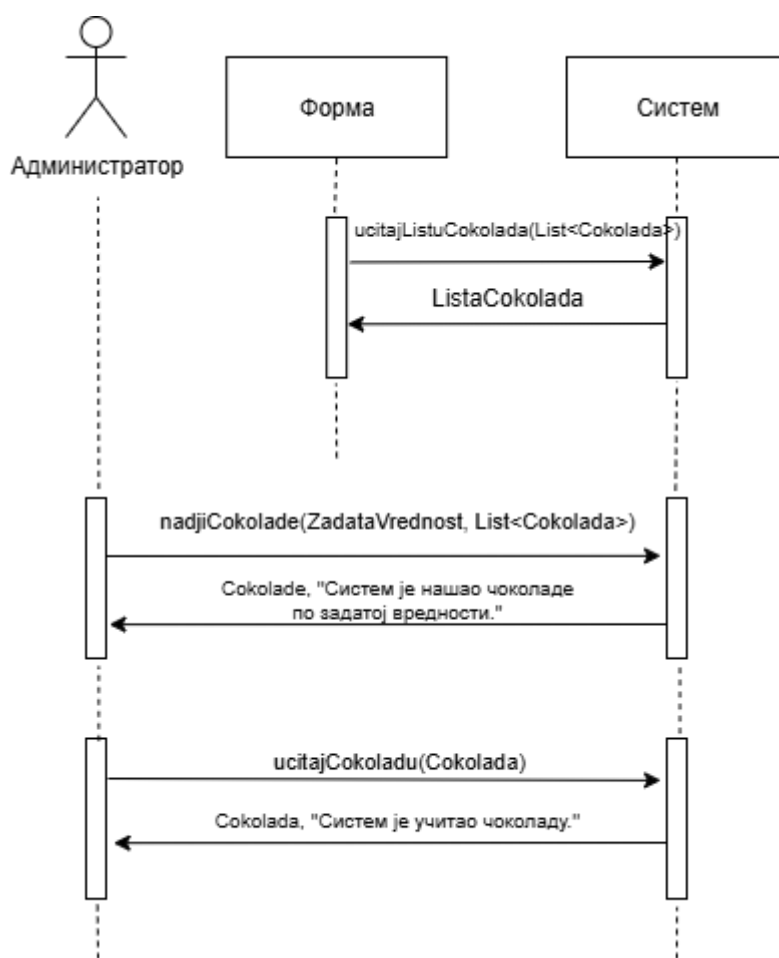
Дијаграм 4: ДС Унос нове чоколаде (алтернативни сценарио)

Са наведених секвенчних дијаграма уочавају се две системске операције које треба пројектовати:

1. *signal* **ucitajListuVrstaCokolade(List<VrstaCokolade>)**
2. *signal* **kreirajCokoladu(Cokolada)**

Основни сценарио СК

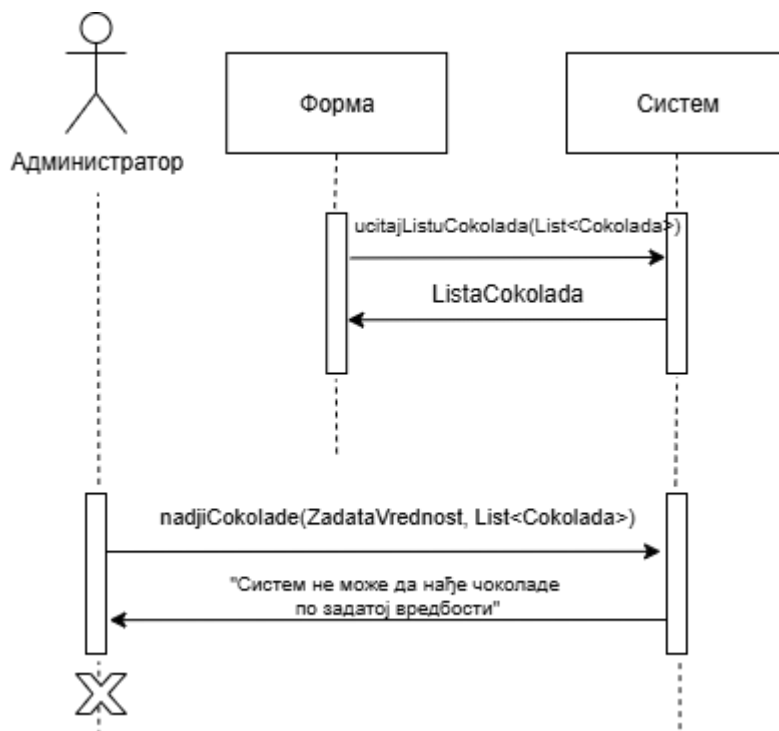
1. **Форма** позива **систем** да врати листу **чоколада**. (АПСО)
2. **Систем** враћа форми листу **чоколада**. (ИА)
3. **Администратор** позива **систем** да нађе **чоколаде** по задатој вредности.(АПСО)
4. **Систем** приказује **администратору** податке о **чоколадама** и поруку: „**Систем** је нашао **чоколаде** по задатој вредности.”.(ИА)
5. **Администратор** позива **систем** да учита **чоколаду**. (АПСО)
6. **Систем** приказује **администратору** податке о **чоколади** и поруку: “**Систем** је прочитао **чоколаду**”. (ИА)



Дијаграм 5: ДС Претрага чоколаде (основни сценарио)

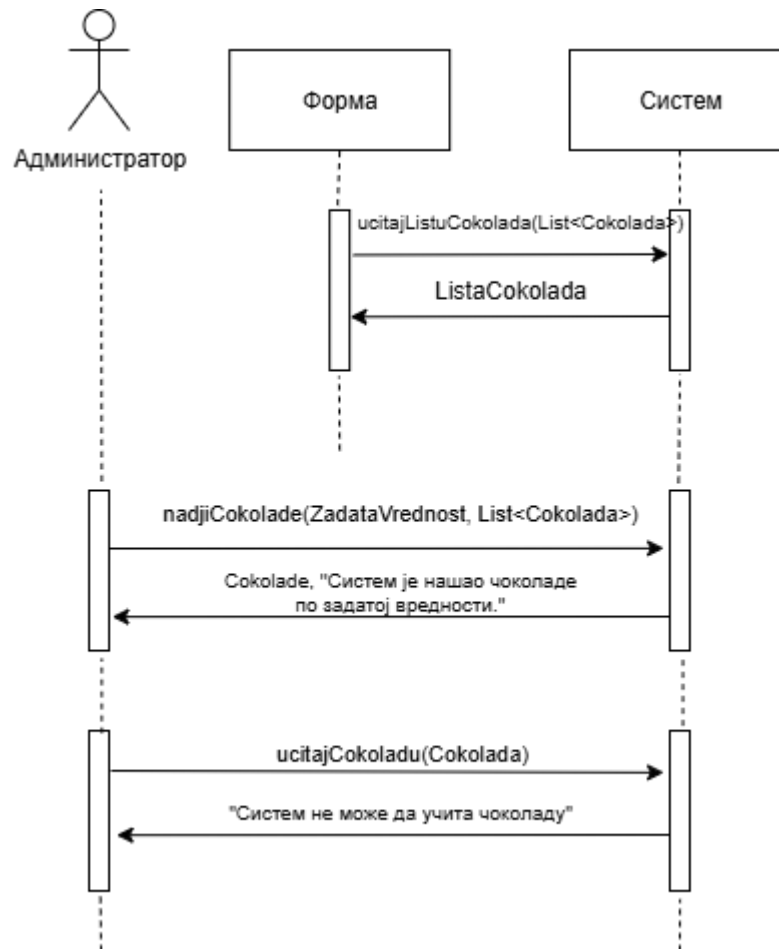
Алтернативна сценарија

- 4.1 Уколико **систем** не може да нађе **чоколаде** по задатој вредности, он приказује **администратору** поруку: „**Систем** не може да нађе **чоколаде** по задатој вредности.” Прекида се извршење сценарија. (ИА)



Дијаграм 6: ДС Претраживање чоколаде (алтернативни сценарио 1)

6.1 Уколико **систем** не може да учита **чоколаду** он приказује **администратору** поруку:
“**Систем** не може да учита **чоколаду**.”.(ИА)



Дијаграм 7: ДС Претраживање чоколада (алтернативни сценарио 2)

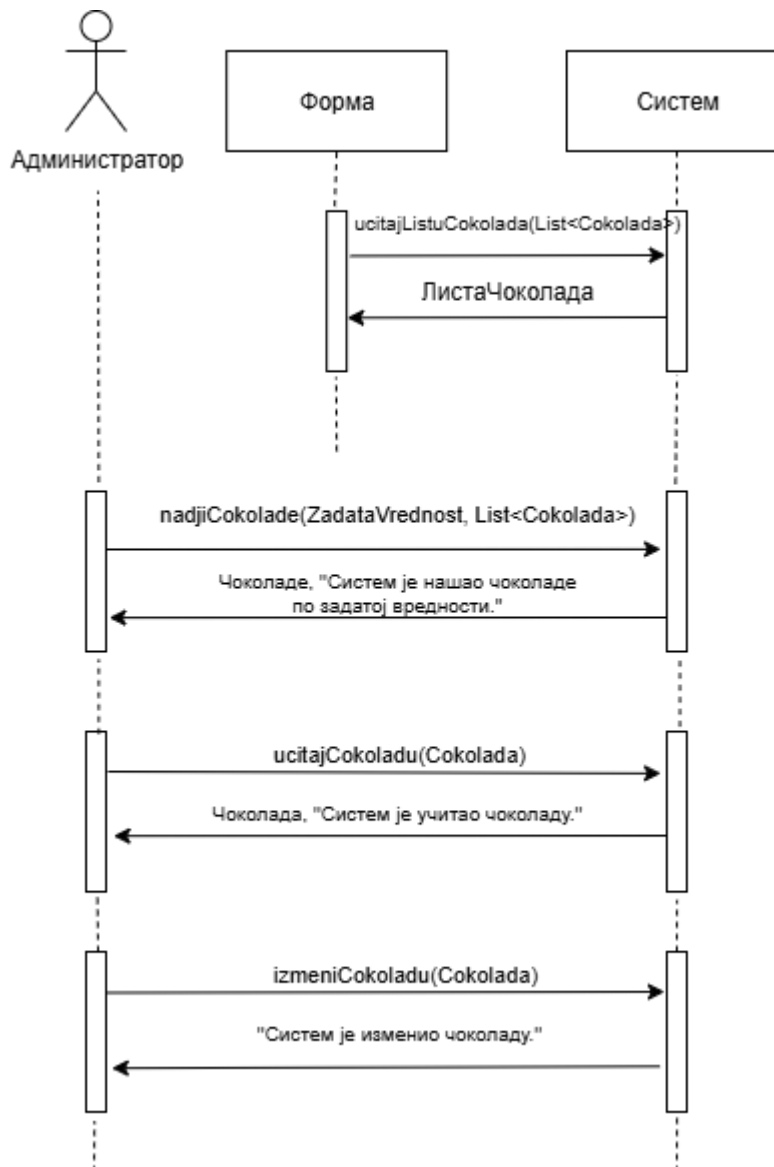
Са наведених секвенцих дијаграма уочавају се три системске операције које треба пројектовати:

1. *signal* **ucitajListuCokolada(List<Cokolada>)**
2. *signal* **nadjiCokolade(String zadataVrednost, List<Cokolada>)**
3. *signal* **ucitajCokoladu(Cokolada)**

ДС4: Дијаграм секвенци случаја коришћења – Измена чоколаде

Основни сценарио СК

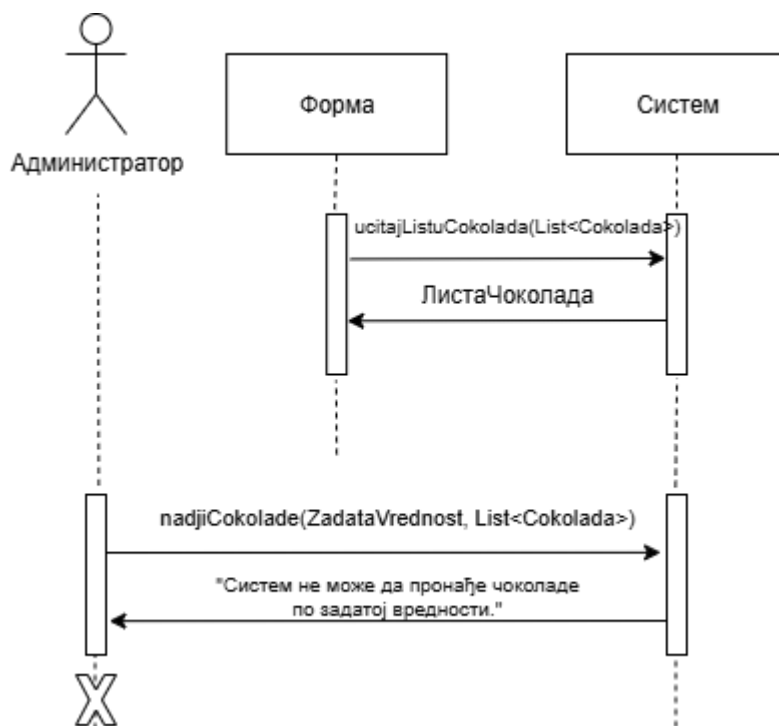
1. **Форма** позива **систем** да врати листу **чоколада**. (АПСО)
2. **Систем** враћа форми листу **чоколада**. (ИА)
3. **Администратор** позива **систем** да пронађе **чоколаде** по задатој вредности. (АПСО)
4. **Систем** приказује **администратору** податке о траженим **чоколадама** и поруку: „**Систем** је нашао **чоколаде** по задатој вредности.“. (ИА)
5. **Администратор** позива **систем** да пронађе изабрану **чоколаду**. (АПСО)
6. **Систем** приказује податке о траженој **чоколади** и поруку: „**Систем** је прочитао **чоколаду**“. (ИА)
7. **Администратор** позива **систем** да запамти податке о **чоколади**. (АПСО)
8. **Систем** приказује **администратору** измењену **чоколаду** уз поруку: „**Систем** је изменио **чоколаду**“. (ИА)



Дијаграм 8: ДС Измена података чоколаде (основни сценарио)

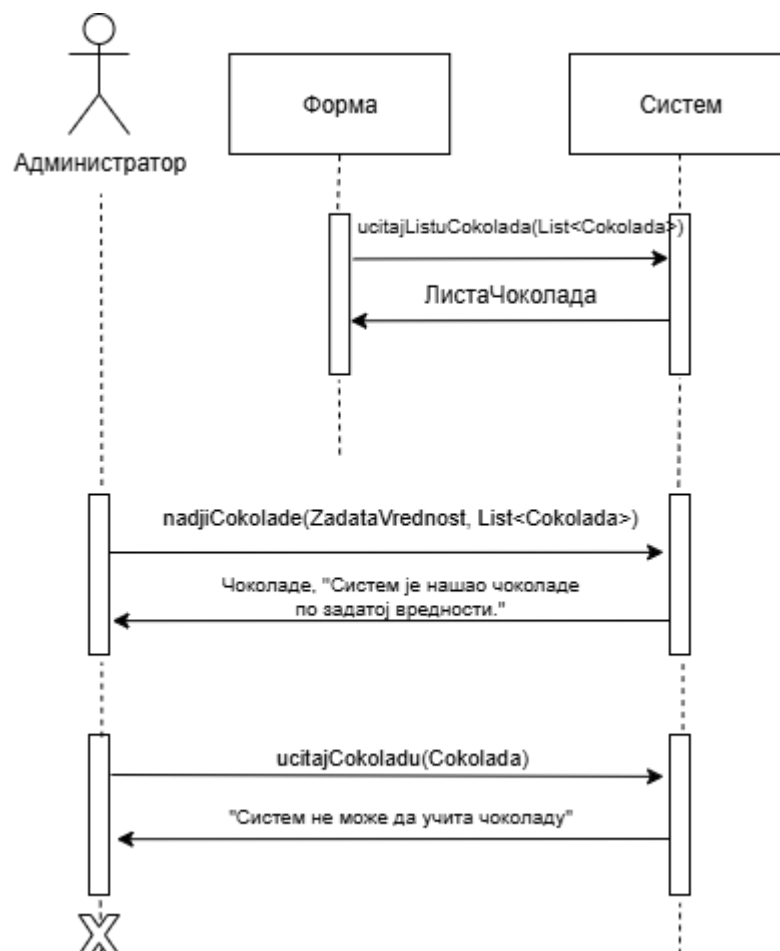
Алтернативна сценарија СК

- 4.1 Уколико **систем** не може да прикаже податке о **чоколадама**, приказује следећу поруку **администратору**: „Систем не може да пронађе **чоколаде** на основу унетих података”. Прекида се извршење сценарија. (ИА)



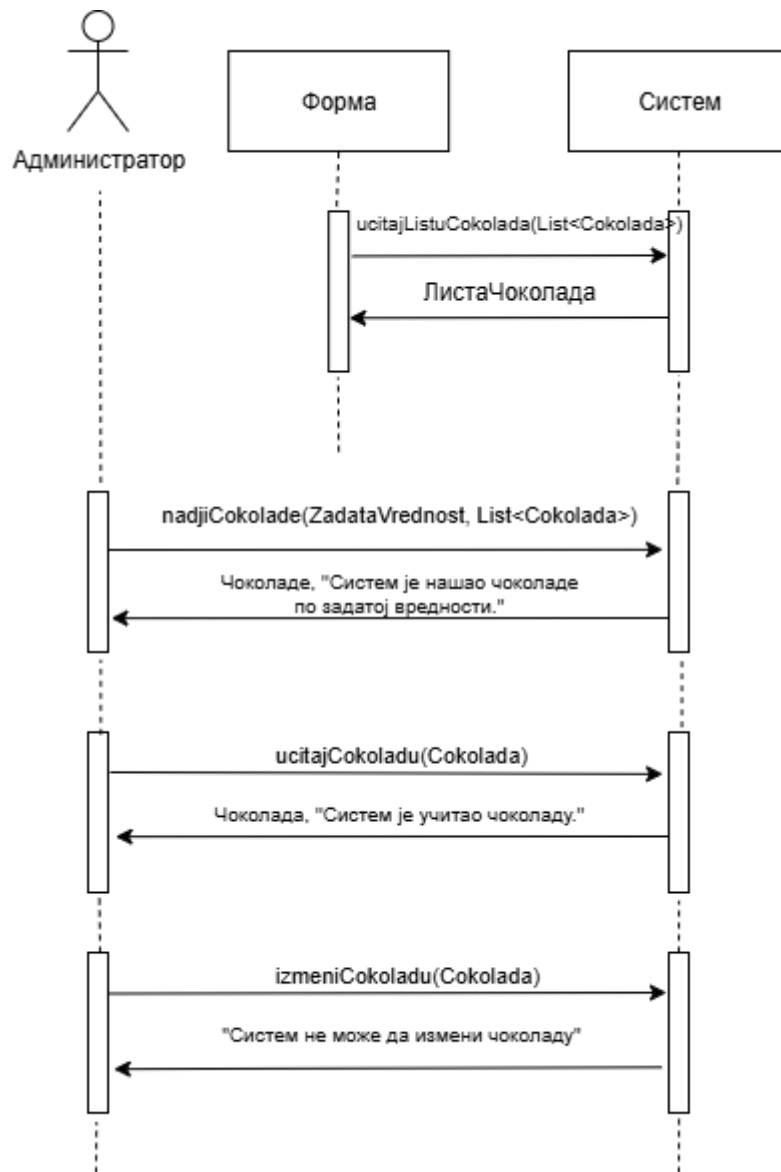
Дијаграм 9: ДС Измена података чоколаде (алтернативни сценарио 1)

6.1. Уколико **систем** не може да прикаже податке о **чоколади**, приказује следећу поруку **администратору**: „**Систем** не може да учита **чоколаду**.“. Прекида се извршење сценарија. (ИА)



Дијаграм 10: ДС Измена података чоколаде (алтернативни сценарио 2)

8.1 Уколико **систем** не може да измени податке **чоколаде**, приказује следећу поруку **администратору**: „**Систем** не може да измени **чоколаду**.”. (ИА)



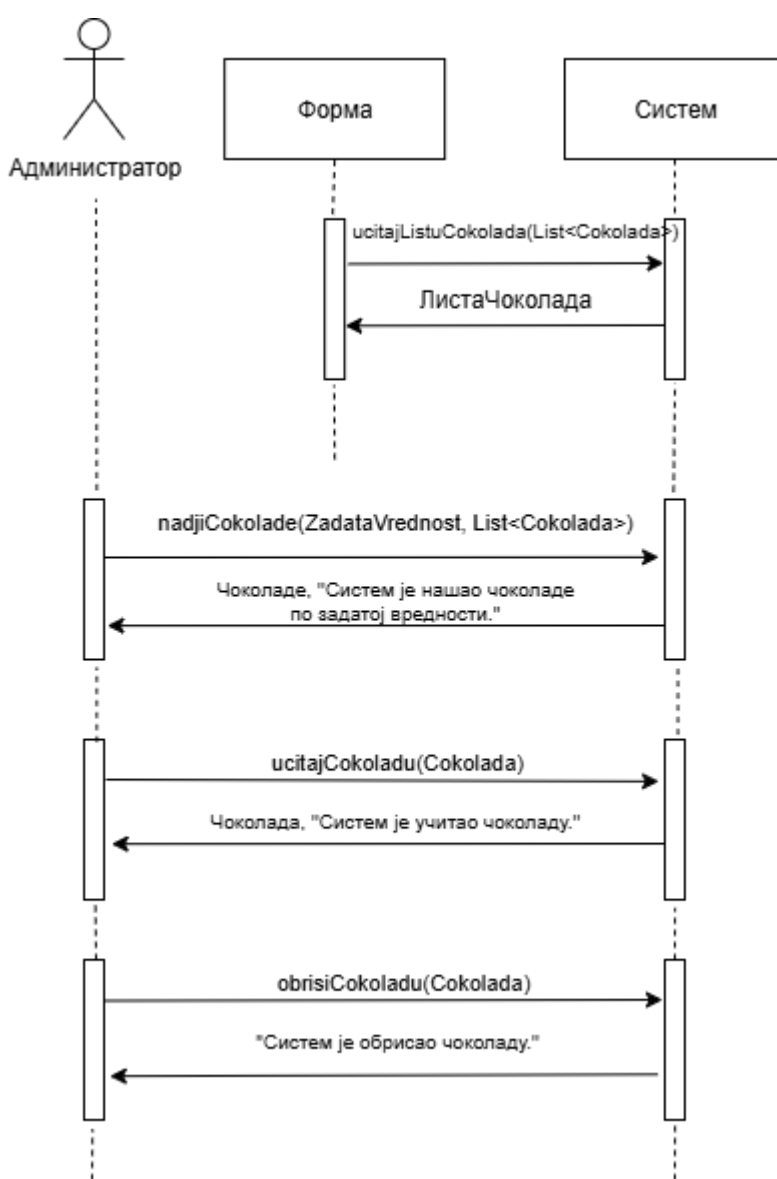
Дијаграм 11: ДС Измена података чоколаде (алтернативни сценарио 3)

Са наведених секвенцих дијаграма уочавају се четири системске операције које треба пројектовати:

1. *signal* **ucitajListuCokolada(List<Cokolada>)**
2. *signal* **nadjiCokolade(String zadataVrednost, List<Cokolada>)**
3. *signal* **ucitajCokoladu(Cokolada)**
4. *signal* **izmeniCokoladu (Cokolada)**

Основни сценарио СК

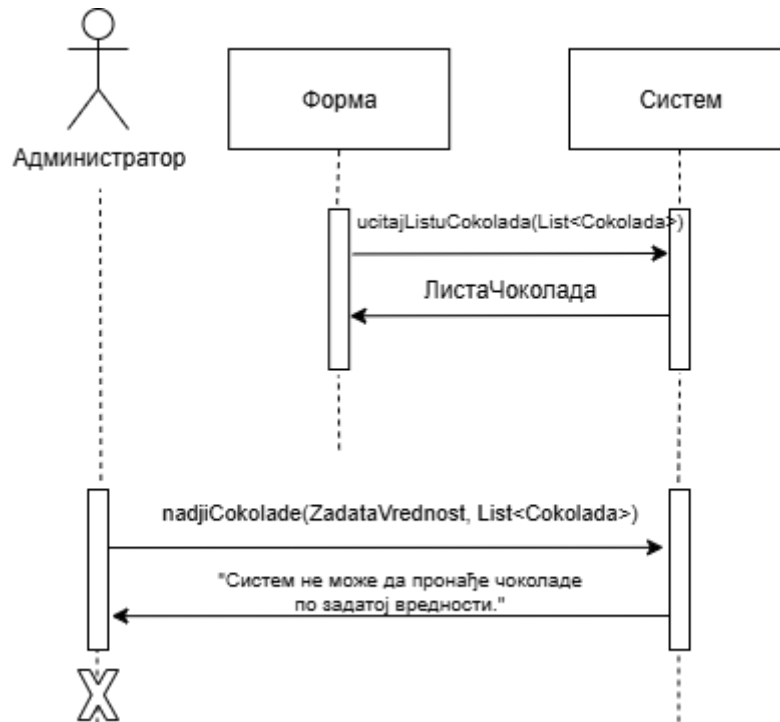
1. **Форма** позива **систем** да врати листу **чоколада**. (АПСО)
2. **Систем** враћа форми листу **чоколада**. (ИА)
3. **Администратор** позива **систем** да пронађе **чоколаде** по задатој вредности. (АПСО)
4. **Систем** приказује **чоколаде администратору** и поруку: „**Систем** је пронашао **чоколаде** по задатој вредности“. (ИА)
5. **Администратор** позива **систем** да нађе изабрану **чоколаду**. (АПСО)
6. **Систем** приказује **чоколаду** и поруку: „**Систем** је прочитао **чоколаду**“. (ИА)
7. **Администратор** позива **систем** да обрише **чоколаду**. (АПСО)
8. **Систем** приказује **администратору** поруку: „**Систем** је обрисао **чоколаду**“. (ИА)



Дијаграм 12: ДС Брисање чоколаде (основни сценарио)

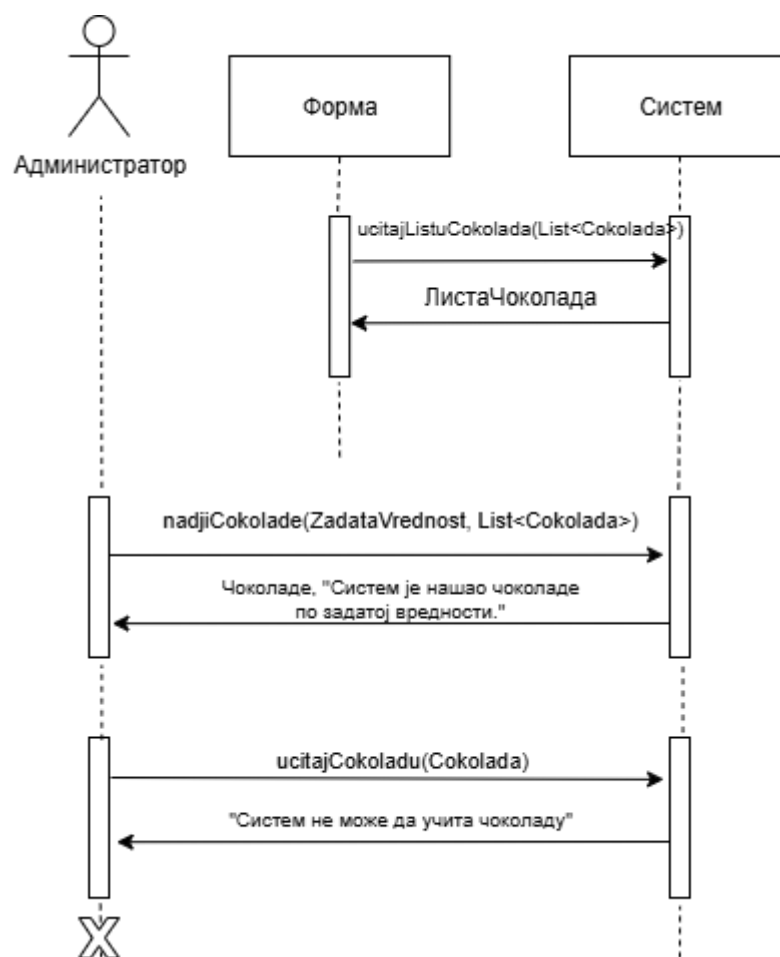
Алтернативна сценарија СК

4.1 Уколико **систем** не може да нађе **чоколаде**, он приказује **администратору** поруку: „**Систем** не може да нађе **чоколаде** по задатој вредности”. Прекида се извршење сценарија. (ИА)



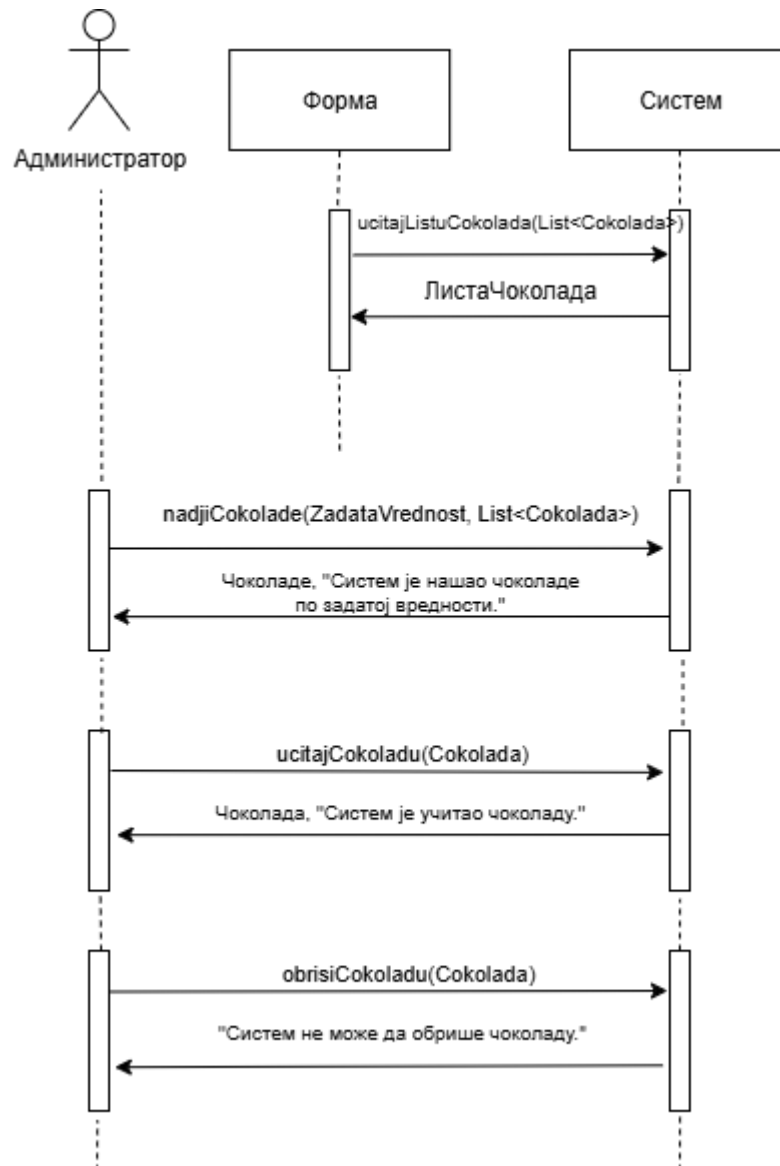
Дијаграм 13: ДС Брисање чоколаде (алтернативни сценарио 1)

6.1. Уколико **систем** не може да прикаже податке о **чоколади**, приказује следећу поруку **администратору**: „**Систем** не може да учита **чоколаду**.“. Прекида се извршење сценарија. (ИА)



Дијаграм 14: ДС Брисање чоколаде (алтернативни сценарио 2)

8.1 Уколико **систем** не може да обрише **чоколаду**, приказује следећу поруку **администратору**: „**Систем** не може да обрише **чоколаду**.”. (ИА)



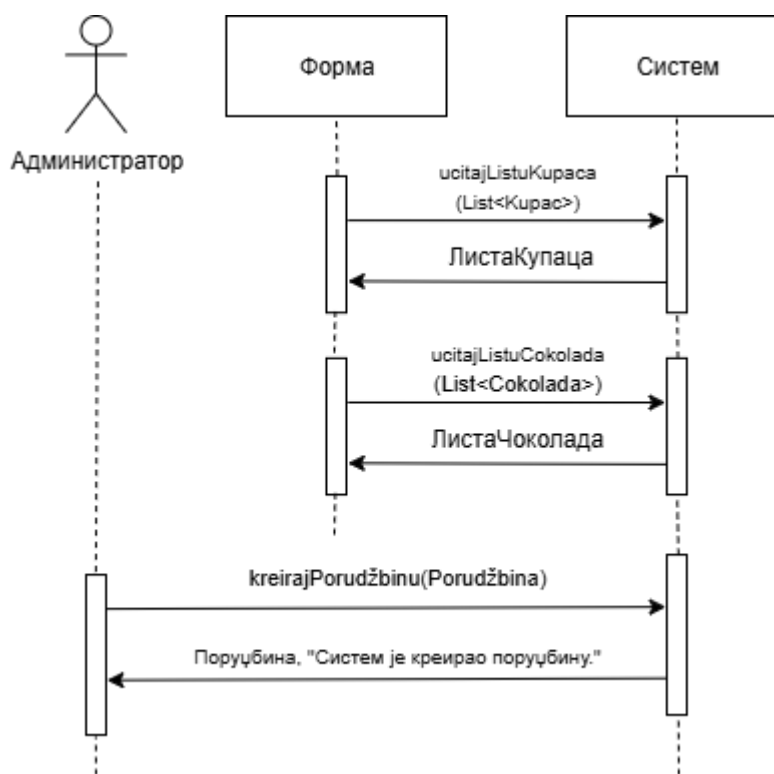
Дијаграм 15: ДС Брисање чоколаде (алтернативни сценарио 2)

Са наведених секвенцих дијаграма уочавају се четири системске операције које треба пројектовати:

1. *signal* **ucitajListuCokolada(List<Cokolada>)**
2. *signal* **nadjiCokolade(ZadataVrednost, List<Cokolada>)**
3. *signal* **ucitajCokoladu(Cokolada)**
4. *signal* **obrisiCokoladu(Cokolada)**

Основни сценарио СК

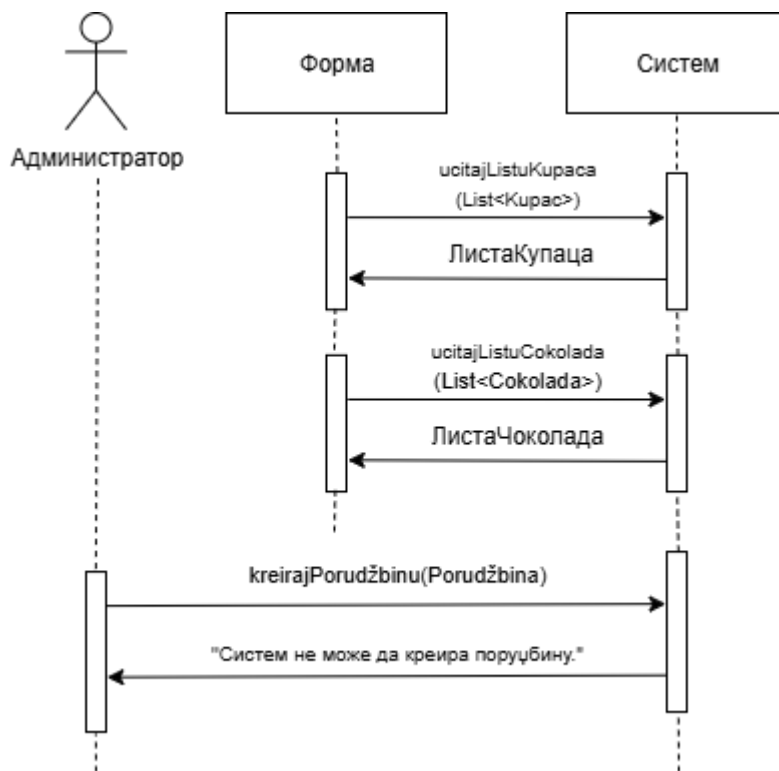
1. **Форма** позива **систем** да врати листу **купаца**. (АПСО)
2. **Систем** враћа форми листу **купаца**. (ИА)
3. **Форма** позива **систем** да врати листу **чоколада**. (АПСО)
4. **Систем** враћа форми листу **чоколада**.
5. **Администратор** позива **систем** да креира **поруџбину**.(АПСО)
6. **Систем** приказује **администратору** креирану **поруџбину** и поруку: „**Систем** је креирао **поруџбину**.”. (ИА)



Дијаграм 16: ДС Унос поруџбине (основни сценарио)

Алтернативна сценарија

6.1. Уколико **систем** не може да креира **поруџбину**, он приказује **администратору** поруку:
„**Систем** не може да креира **поруџбину**.”(ИА)



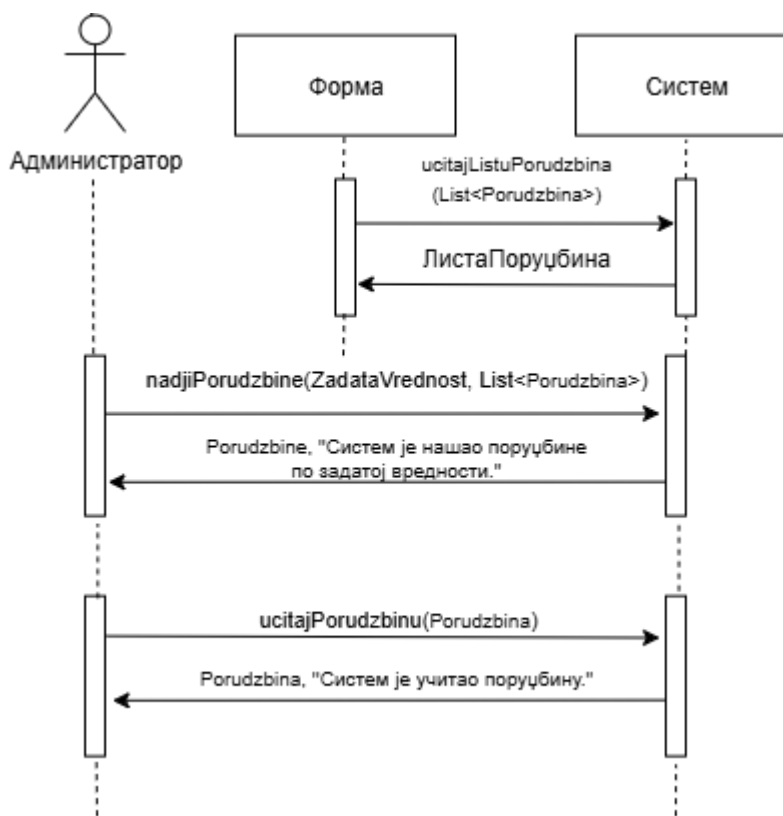
Дијаграм 17: ДС Унос поруџбине (алтернативни сценарио)

Са наведених секвенцих дијаграма уочавају се три системске операције које треба пројектовати:

1. *signal* `ucitajListuKupaca(List<Kupac>)`
2. *signal* `ucitajListuCokolada(List<Cokolada>)`
3. *signal* `kreirajPorudzbenu(Porudzbina)`

Основни сценарио СК

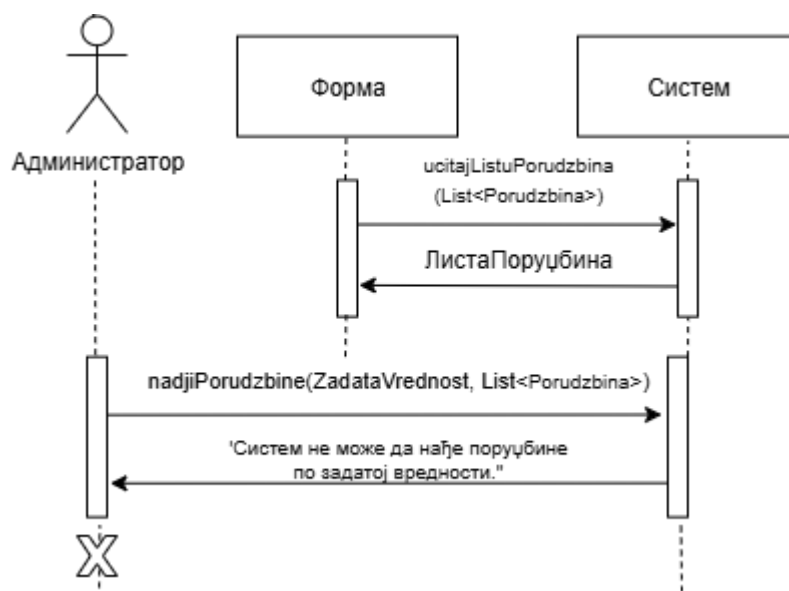
1. **Форма** позива **систем** да врати листу **поруџбина**. (АПСО)
2. **Систем** враћа форми листу **поруџбина**. (ИА)
3. **Администратор** позива **систем** да нађе **поруџбине** по задатој вредности. (АПСО)
4. **Систем** приказује **администратору** податке о **поруџбинама** и поруку: „**Систем** је нашао **поруџбине** по задатој вредности.”. (ИА)
5. **Администратор** позива **систем** да учита **поруџбину**. (АПСО)
6. **Систем** приказује **администратору** податке о **поруџбини** и поруку: “**Систем** је учитао **поруџбину**.”. (ИА)



Дијаграм 18: ДС Претрага поруџбине (основни сценарио)

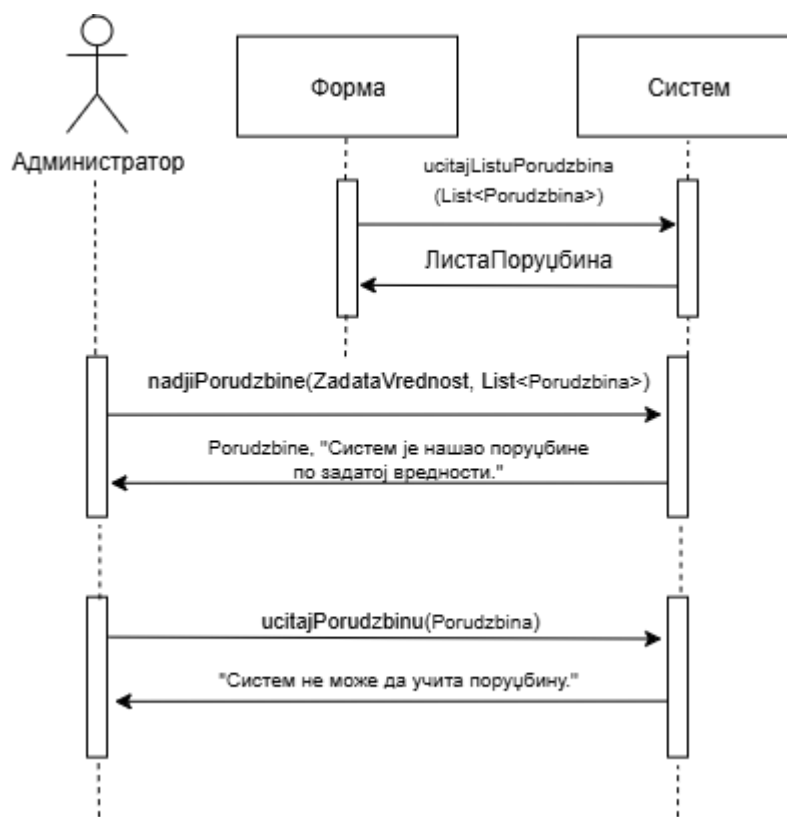
Алтернативна сценарија

4.1 Уколико **систем** не може да нађе **поруџбине** по задатој вредности, приказује следећу поруку **администратору**: „**Систем** не може да нађе **поруџбине** по задатој вредности”. Прекида се извршење сценарија. (ИА)



Дијаграм 19: ДС Претрага поруџбине (алтернативни сценарио 1)

6.1 Уколико **систем** не може да учита **поруџбину** он приказује **администратору** поруку:
 „Систем не може да учита **поруџбину**.”. (ИА)



Дијаграм 20: ДС Претрага поруџбине (алтернативни сценарио 2)

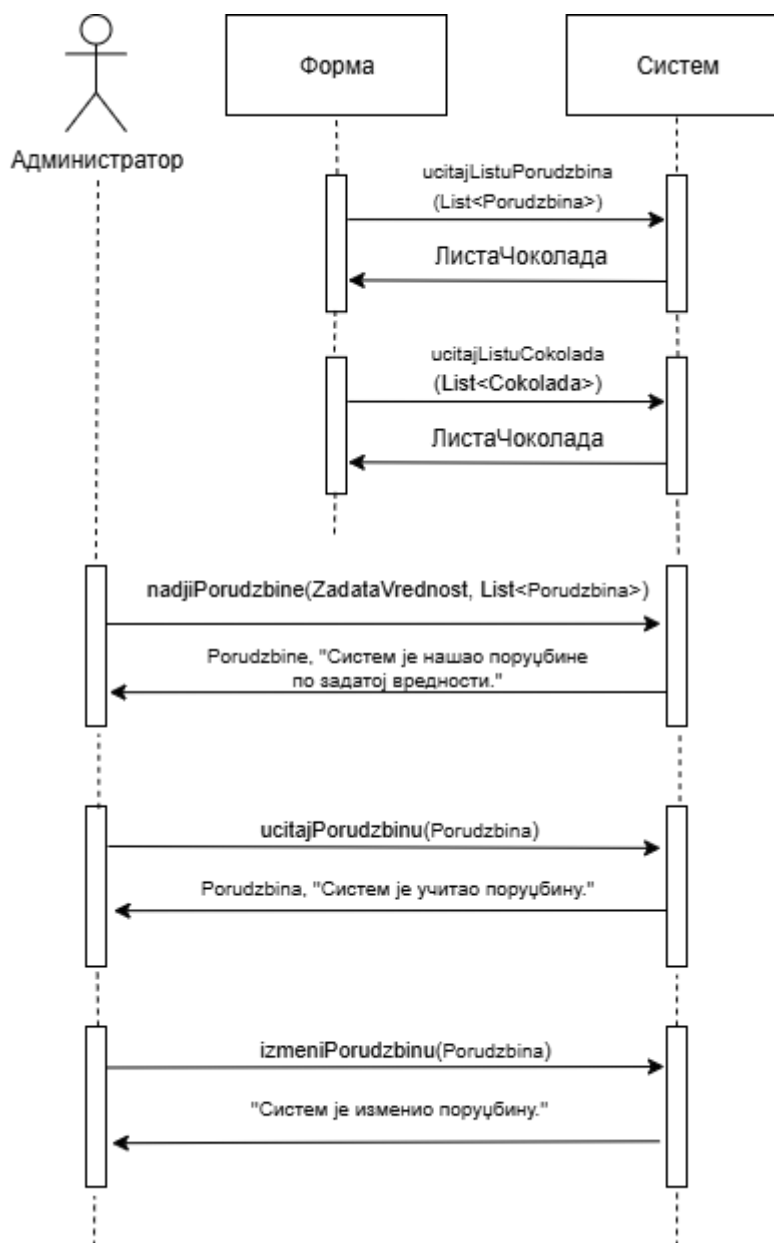
Са наведених секвенцих дијаграма уочавају се три системске операције које треба пројектовати:

1. *signal* **ucitajListuPorudzbina (List<Porudzbina>)**
2. *signal* **nadjiPorudzbine(String zadataVrednost, List<Porudzbina>)**
3. *signal* **ucitajPorudzbinu(Porudzbina)**

ДС8: Дијаграм секвенци случаја коришћења – Измена поруџбине

Основни сценарио СК

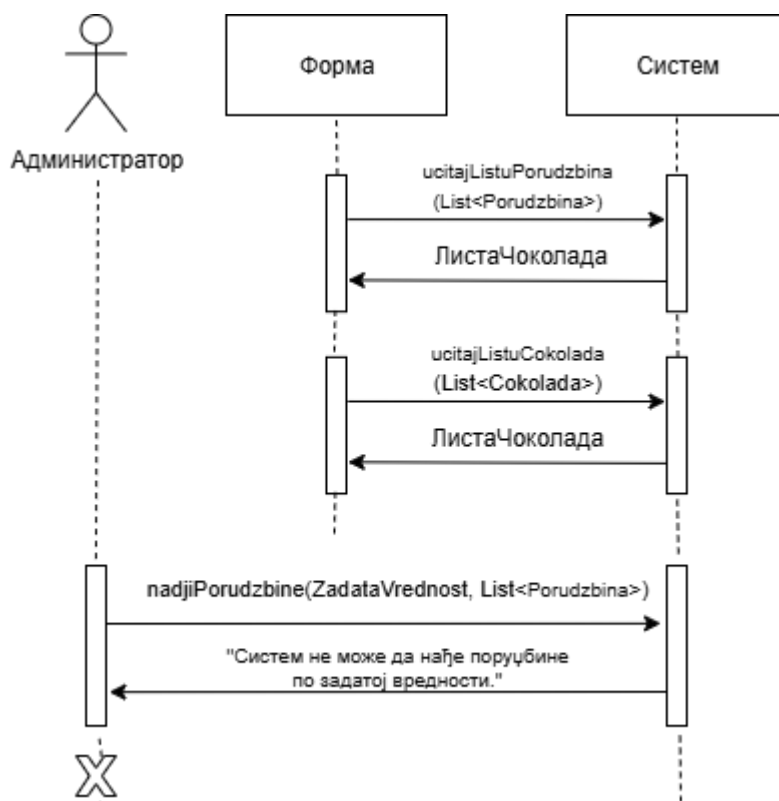
1. Форма позива систем да врати листу поруџбина. (АПСО)
2. Систем враћа форми листу поруџбина. (ИА)
3. Форма позива систем да врати листу чоколада. (АПСО)
4. Систем враћа форми листу чоколада. (ИА)
5. Администратор позива систем да нађе поруџбине по задатој вредности. (АПСО)
6. Систем приказује администратору поруџбине и поруку: „Систем је нашао поруџбине по задатој вредности.“. (ИА)
7. Администратор позива систем да нађе изабрану поруџбину. (АПСО)
8. Систем приказује администратору поруџбину и поруку: „Систем је прочитао поруџбину.“. (ИА)
9. Администратор позива систем да запамти податке о поруџбини. (АПСО)
10. Систем приказује администратору поруку: „Систем је изменио поруџбину.“. (ИА)



Дијаграм 21: ДС Измена поруџбине (основни сценарио)

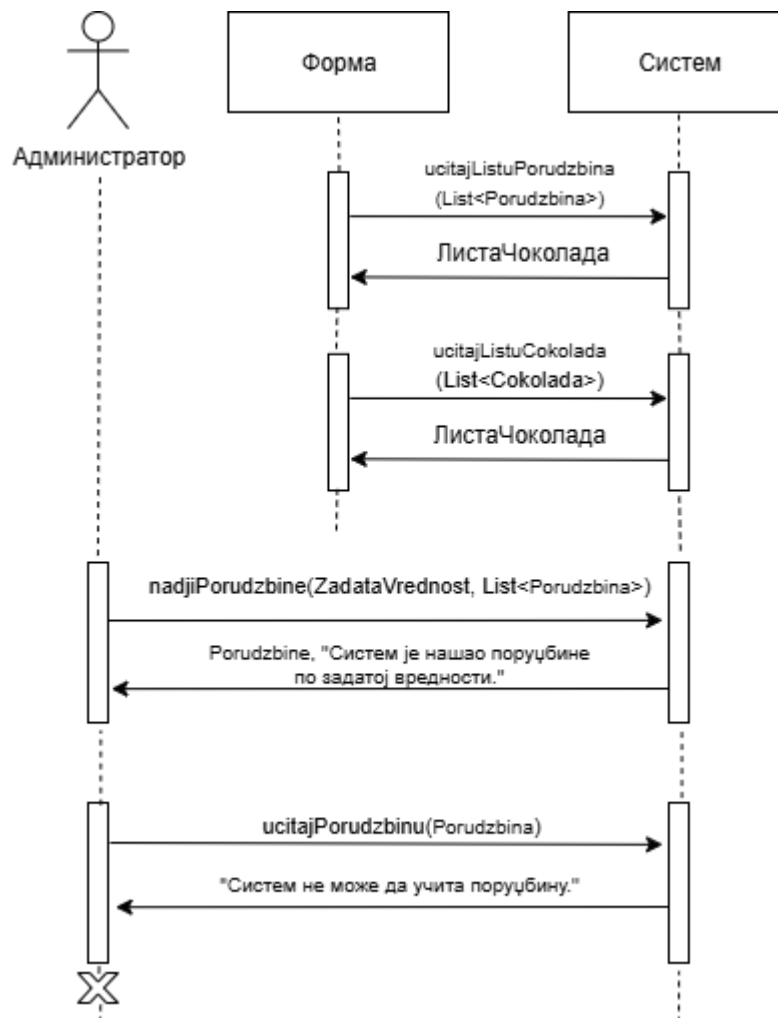
Алтернативна сценарија

6.1 Уколико **систем** не може да пронађе **поруџбине**, он приказује **администратору** поруку: „**Систем** не може да пронађе **поруџбине** по задатој вредности”. Прекида се извршење сценарија. (ИА)



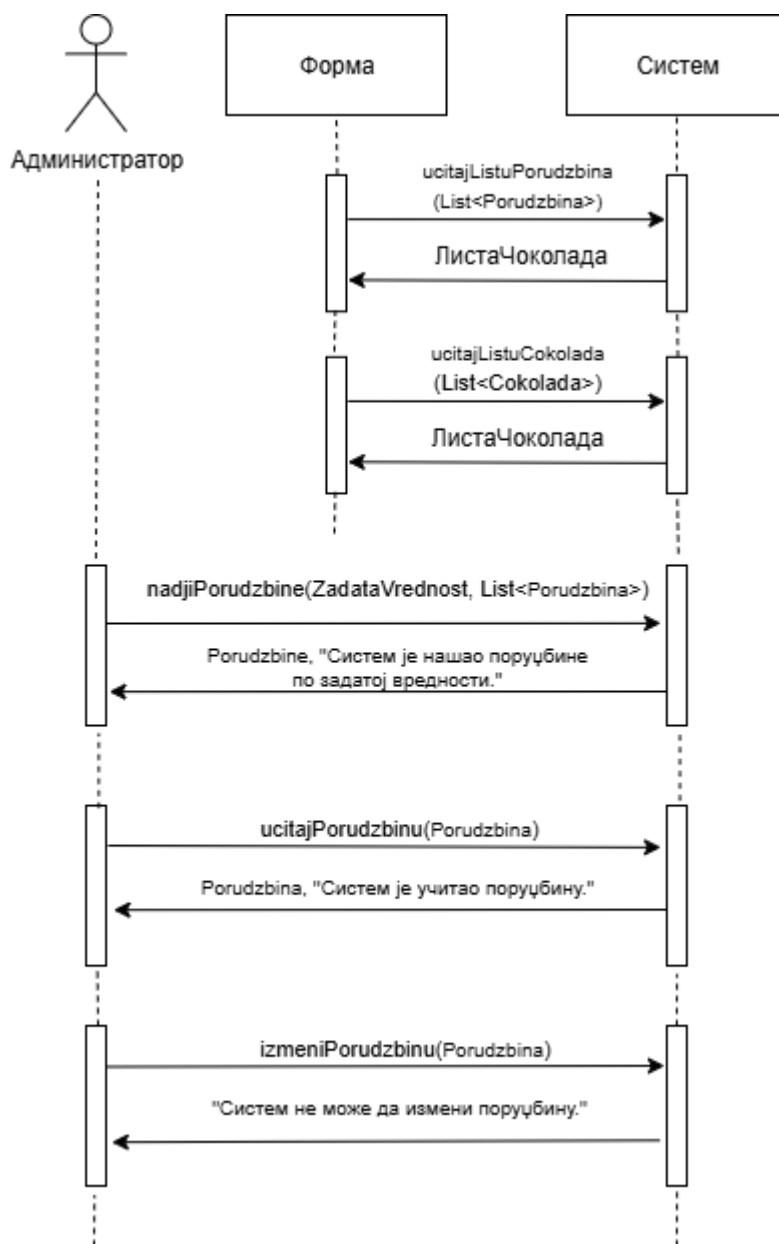
Дијаграм 22: ДС Измена поруџбине (алтернативни сценарио 1)

8.1 Уколико **систем** не може да прикаже податке о **поруџбини**, он приказује **администратору** поруку: „**Систем** не може да учита **поруџбину**”. Прекида се извршење сценарија. (ИА)



Дијаграм 23: ДС Измена поруџбине (алтернативни сценарио 2)

10.1. Уколико **систем** не може да измени податке о **поруџбини**, он приказује **администратору** поруку: „**Систем** не може да измени **поруџбину**“. (ИА)



Дијаграм 24: ДС Измена поруџбине (алтернативни сценарио 3)

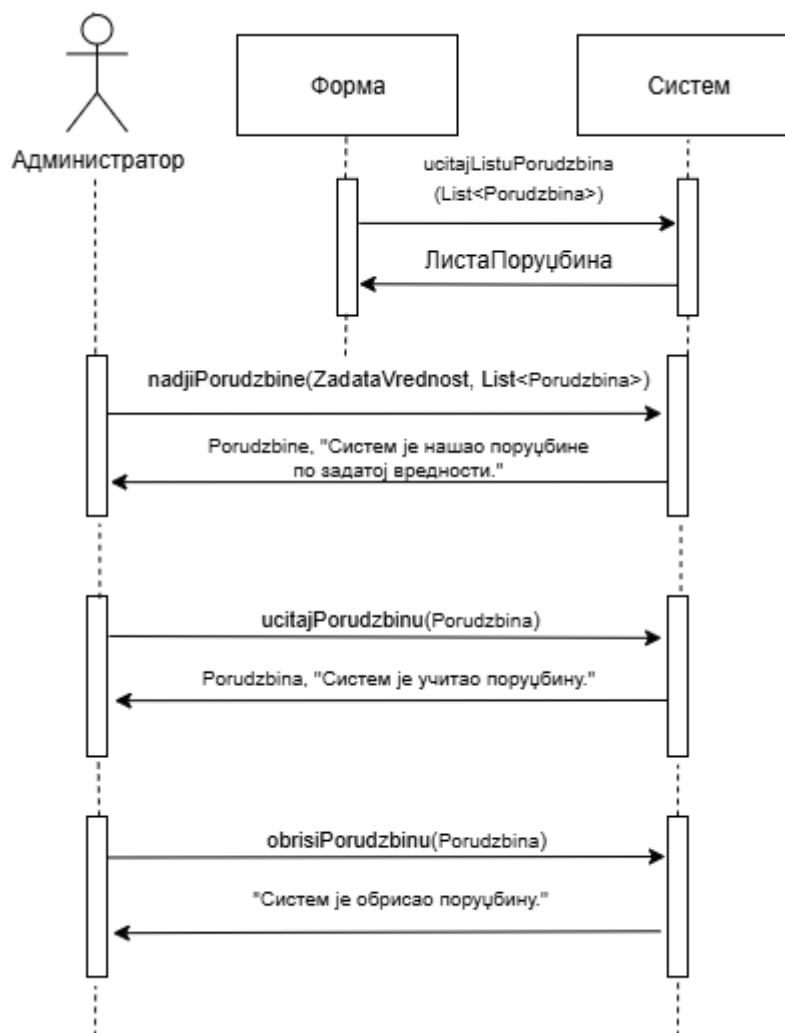
Са наведених секвенцих дијаграма уочавају се пет системских операција које треба пројектовати:

1. *signal* **ucitajListuPorudzbina**(List<Porudzbina>)
2. *signal* **ucitajListuCokolada**(List<Cokolada>)
3. *signal* **nadjiPorudzbine**(String zadataVrednost, List<Porudzbina>)
4. *signal* **ucitajPorudzbину**(Porudzbina)
5. *signal* **izmeniPorudzbину**(Porudzbina)

ДС9: Дијаграм секвенци случаја коришћења – Брисање поруџбине

Основни сценарио СК

1. **Форма** позива **систем** да врати листу **поруџбина**. (АПСО)
2. **Систем** враћа форми листу **поруџбина**. (ИА)
3. **Администратор** позива **систем** да нађе **поруџбине** по задатој вредности. (АПСО)
4. **Систем** приказује **поруџбине администратору** и поруку: „**Систем** је нашао **поруџбине** по задатој вредности“. (ИА)
5. **Администратор** позива **систем** да нађе изабрану **поруџбину**. (АПСО)
6. **Систем** приказује **администратору** **поруџбину** и поруку: „**Систем** је прочитао **поруџбину**“. (ИА)
7. **Администратор** позива **систем** да обрише **поруџбину**. (АПСО)
8. **Систем** приказује **администратору** поруку: „**Систем** је обрисао **поруџбину**“. (ИА)

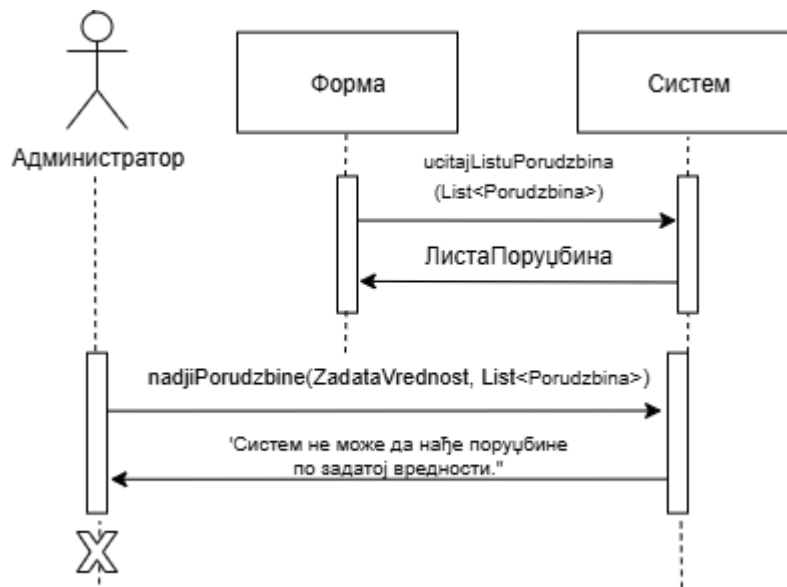


Дијаграм 25: ДС Брисање поруџбине (основни сценарио)

Алтернативна сценарија

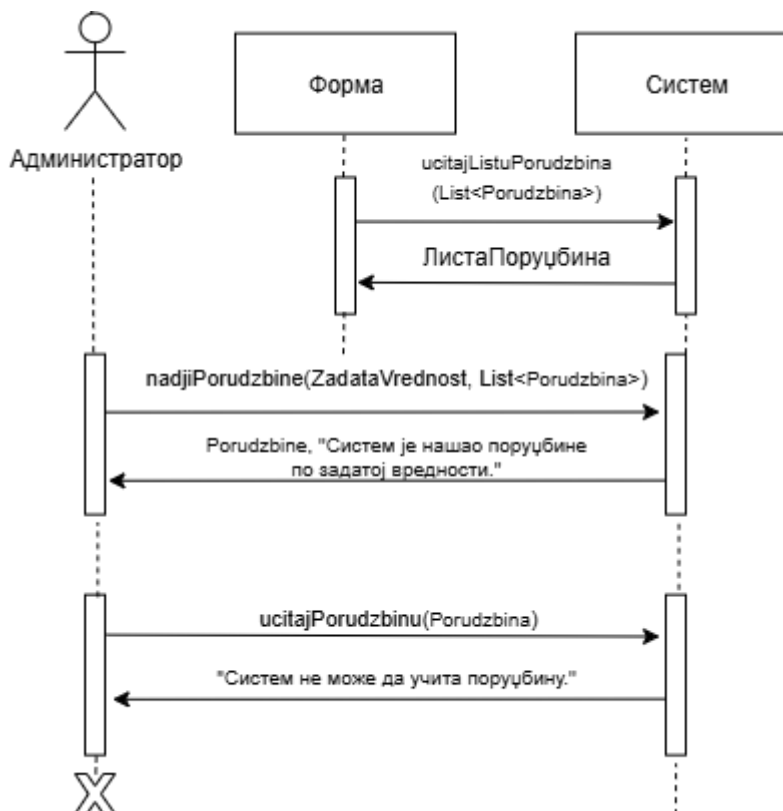
4.1. Уколико **систем** не може да нађе **поруџбине** он приказује **администратору** поруку:

„**Систем** не може да нађе **поруџбине** по задатој вредности”. Прекида се извршење сценарија. (ИА)



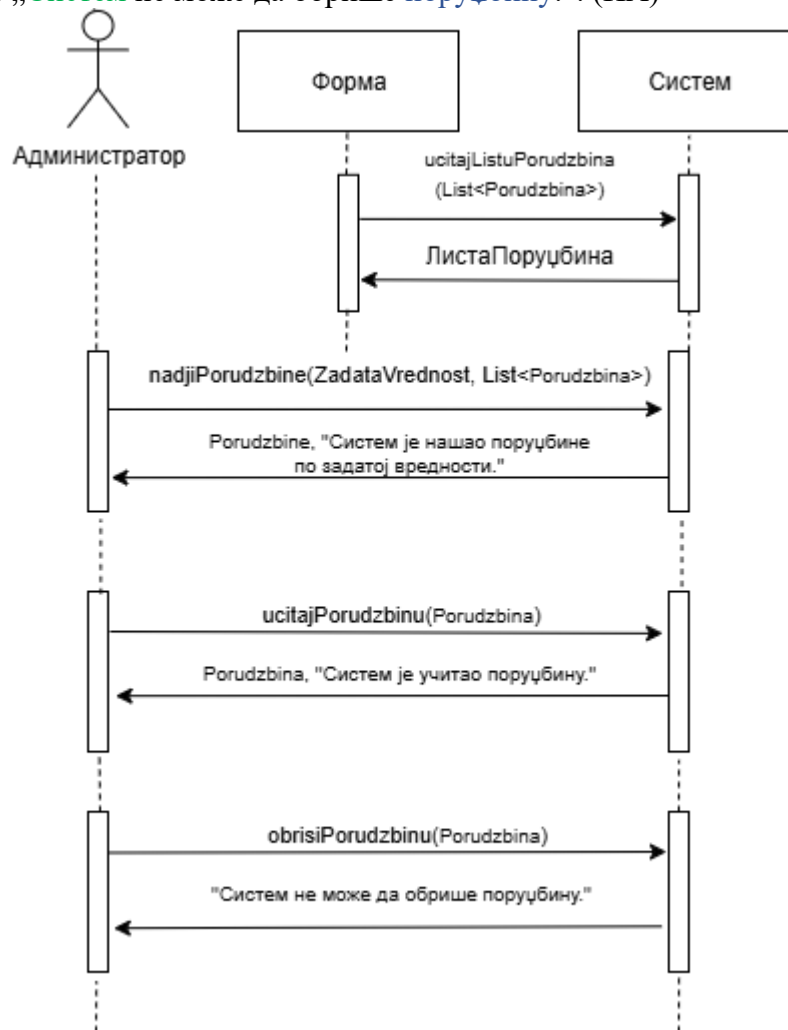
Дијаграм 26: ДС Брисање поруџбине (алтернативни сценарио 1)

6.1. Уколико **систем** не може да прикаже податке о **поруџбини**, он приказује **администратору** поруку: „**Систем** не може да учита **поруџбину**”. Прекида се извршење сценарија. (ИА)



Дијаграм 27: ДС Брисање поруџбине (алтернативни сценарио 2)

8.1. Уколико **систем** не може да обрише **поруџбину** он приказује **администратору** поруку: „**Систем** не може да обрише **поруџбину**.“. (ИА)



Дијаграм 28: ДС Брисање поруџбине (алтернативни сценарио 3)

Са наведених секвенцих дијаграма уочавају се четири системске операције које треба пројектовати:

1. *signal* **ucitajListuPorudzbina(List<Porudzbina>)**
2. *signal* **nadjiPorudzbine(String zadataVrednost, List<Porudzbina>)**
3. *signal* **ucitajPorudzbину(Porudzbina)**
4. *signal* **obrisiPorudzbину(Porudzbina)**

4.2.2. Резултирајуће системске операције

Као резултат анализе сценарија добијамо укупно 15 системских операција које треба пројектовати:

1. *signal* **login**(Administrator)
2. *signal* **ucitajListuVrstaCokolade**(List<VrstaCokolade>)
3. *signal* **kreirajCokoladu**(Cokolada)
4. *signal* **ucitajListuCokolada**(List<Cokolada>)
5. *signal* **nadjiCokolade**(ZadataVrednost, List<Cokolada>)
6. *signal* **ucitajCokoladu**(Cokolada)
7. *signal* **izmeniCokoladu**(Cokolada)
8. *signal* **obrisiCokoladu**(Cokolada)
9. *signal* **ucitajListuKupaca**(List<Kupac>)
10. *signal* **kreirajPorudzbину**(Porudzbina)
11. *signal* **ucitajListuPorudzbina**(List< Porudzbina >)
12. *signal* **nadjiPorudzbine**(ZadataVrednost, List< Porudzbina >)
13. *signal* **ucitajPorudzbину**(Porudzbina)
14. *signal* **izmeniPorudzbину** (Porudzbina)
15. *signal* **obrisiPorudzbину** (Porudzbina)

4.2.3. Дефинисање уговора о системским операцијама

Понашање софтверског система се описује преко системских операција, а за сваку системску операцију се прави уговор. Уговор описује понашање системске операције, то јест описује се оно шта та системска операција треба да одради (али не и како то треба да одради).

Један уговор везује се за једну системску операцију и састоји се од следећих секција:

- *операција*
- *веза са СК*
- *предуслов*
- *постуслов*

Уговор UG1: Login

Операција: login(Administrator): signal;

Веза са СК: СК1

Предуслови: /

Постуслови: Администратор је улогван.

Уговор UG2: ucitajListuVrstaCokolade

Операција: ucitajListuVrstaCokolade(List<VrstaCokolade>): signal;

Веза са СК: СК2

Предуслови: /

Постуслови: Учитана је листа врсти чоколада.

Уговор UG3: kreirajCokoladu

Операција: kreirajCokoladu(Cokolada): signal;

Веза са СК: СК2

Предуслови: Вредносна и структурна ограничења над објектом Чоколада морају бити задовољена.

Постуслови: Креирана је нова чоколада.

Уговор UG4: ucitajListuCokolada

Операција: ucitajListuCokolada(List< Cokolada >): signal;

Веза са СК: СК3, СК4, СК5, СК6, СК8

Предуслови: /

Постуслови: Учитана је листа чоколада.

Уговор UG5: nadjiCokolade

Операција: nadjiCokolade(ZadataVrednost, List<Cokolada>): signal;

Веза са СК: СК3, СК4, СК5

Предуслови: /

Постуслови: Пронађене су тражене чоколаде.

Уговор UG6: ucitajCokoladu

Операција: ucitajCokoladu(Cokolada): signal;

Веза са СК: СК3, СК4, СК5

Предуслови: /

Постуслови: Учитана је тражена чоколада.

Уговор UG7: izmeniCokoladu

Операција: izmeniCokoladu(Cokolada): signal;

Веза са СК: СК4

Предуслови: Вредносна и структурна ограничења над објектом Чоколада морају бити задовољена.

Постуслови: Подаци о чоколади су измењени.

Уговор UG8: obrisiCokoladu

Операција: obrisiCokoladu(Cokolada): signal;

Веза са СК: СК5

Предуслови: Структурна ограничења над објектом Чоколада морају бити задовољена.

Постуслови: Чоколада је обрисана.

Уговор UG9: ucitajListuKupaca

Операција: ucitajListuKupaca(List<Kupac>): signal;

Веза са СК: СК6

Предуслови: /

Постуслови: Листа купаца је учитана.

Уговор UG10: kreirajPorudzbinu

Операција: kreirajPorudzbinu(Porudzbina): signal;

Веза са СК: СК6

Предуслови: Вредносна и структурна ограничења над објектима Поруџбина и СтавкаПоруџбине морају бити задовољена.

Постуслови: Унети подаци о поруџбини су запамћени.

Уговор UG11: ucitajListuPorudzbina

Операција: ucitajListuPorudzbina (List<Porudzbina>): signal;

Веза са СК: СК7, СК8, СК9

Предуслови: /

Постуслови: Учитана је листа поруџбина.

Уговор UG12: nadjiPorudzbine

Операција: nadjiPorudzbine (ZadataVrednost, List<Porudzbina>): signal;

Веза са СК: СК7, СК8, СК9

Предуслови: /

Постуслови: Пронађене су тражене поруџбине.

Уговор UG13: ucitajPorudzbinu

Операција: ucitajPorudzbinu(Porudzbina): signal;

Веза са СК: СК7, СК8, СК9

Предуслови: /

Постуслови: Учитана је тражена поруџбина.

Уговор UG14: izmeniPorudzbinu

Операција: izmeniPorudzbinu(Porudzbina): signal;

Веза са СК: СК8

Предуслови: Вредносна и структурна ограничења над објектима Поруџбина и СтавкаПоруџбине морају бити задовољена.

Постуслови: Поруџбина је измењена.

Уговор UG15: obrisiPorudzbinu

Операција: obrisiPorudzbinu (Porudzbina):signal;

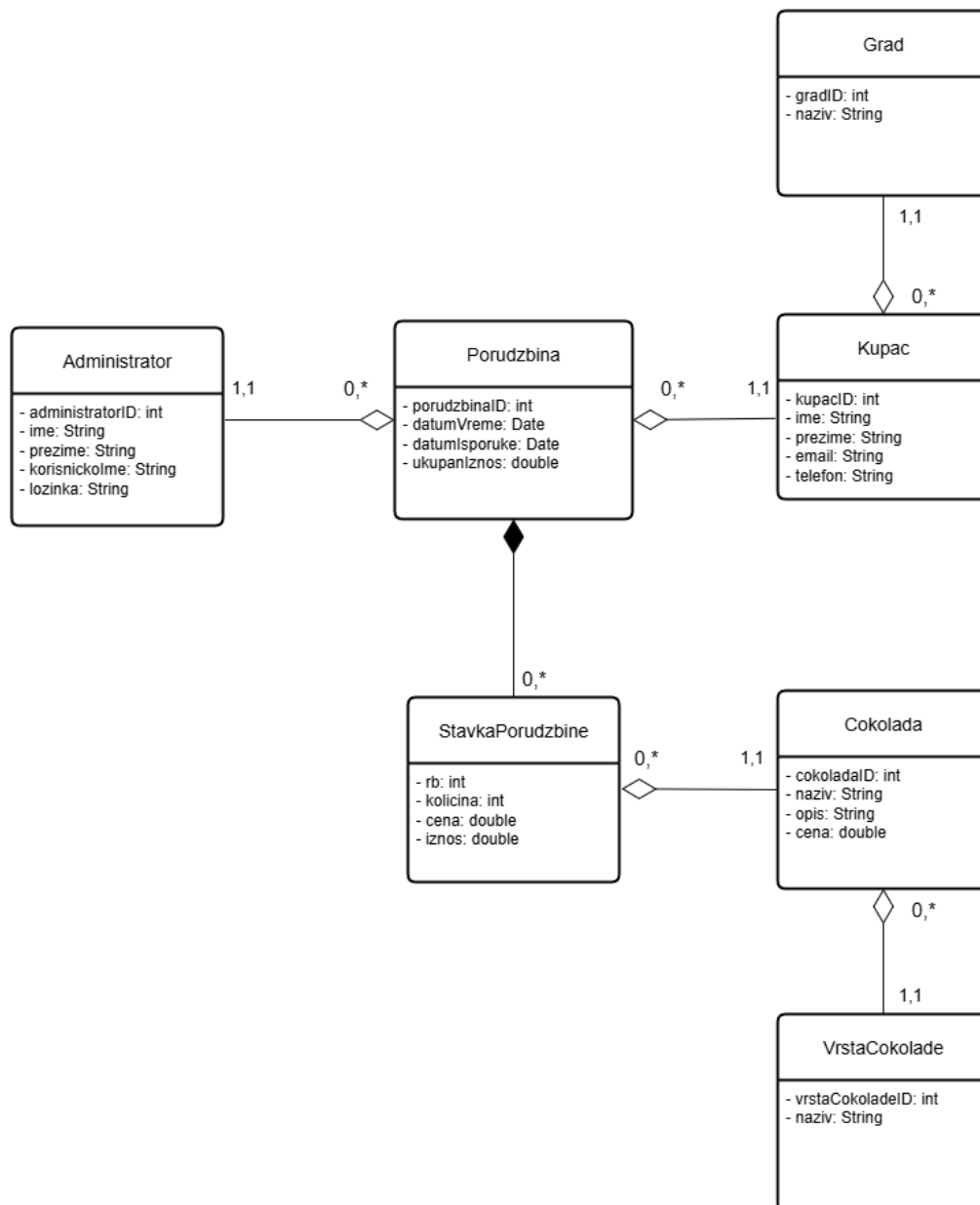
Веза са СК: СК9

Предуслови: Структурна ограничења над објектом Поруџбина морају бити задовољена.

Постуслови: Поруџбина је обрисана.

4.2.4. Структура софтверског система - Концептуални (доменски) модел

Помоћу концептуалног модела описујемо структуру система. Концептуални модел садржи концептуалне класе (доменске објекте) и асоцијације између концептуалних класа.



Слика 34: Концептуални модел

4.2.5. Структура софтверског система- Релациони модел

На основу концептуалног модела, прави се релациони модел, а на основу њега се пројектује релациона база података.

У концептуалном моделу се могу идентификовати следеће класе: Administrator, VrstaCokolade, Cokolada, Grad, Kupac, Porudzbina i StavkaPorudzbine. Свака релација ће бити представљена као једна табела у релационом моделу.

Administrator(AdministratorID, Ime, Prezime, KorisnikIme, Lozinka)

VrstaCokolade(VrstaCokoladeID, Naziv)

Cokolada(CokoladaID, Naziv, Opis, Cena, *VrstaCokoladeID*)

Grad(GradID, Naziv)

Kupac(KupacID, Ime, Prezime, Email, Telefon, *GradID*)

Porudzbina(PorudzbinaID, DatumVreme, DatumIsporuke, UkupanIznos, *KupacID*, *AdministratorID*)

StavkaPorudzbine(PorudzbinaID, Rb, Kolicina, Cena, Iznos, *CokoladaID*)

Табеле ограничења

Tabela Administrator		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT /
	AdministratorID	int	Not null and > 0			UPDATE CASCADES Porudzbina
	Ime	String	Not null			DELETE RESTRICTED Porudzbina
	Prezime	String	Not null			
	KoriscnikoIme	String	Not null			
	Lozinka	String	Not null			

Tabela VrstaCokolade		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT /
	VrstaCokoladeID	int	Not null and > 0			UPDATE CASCADES Cokolada
	Naziv	String	Not null			DELETE RESTRICTED Cokolada

Tabela Cokolada		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT RESTRICTED VrstaCokolade
	CokoladaID	int	Not null and > 0			UPDATE RESTRICTED VrstaCokolade CASCADES StavkaPorudzbine
	Naziv	String	Not null			
	Opis	String	Not null			
	Cena	double	Not null			
	VrstaCokoladeID	String	Not null and > 0			DELETE RESTRICTED StavkaPorudzbine

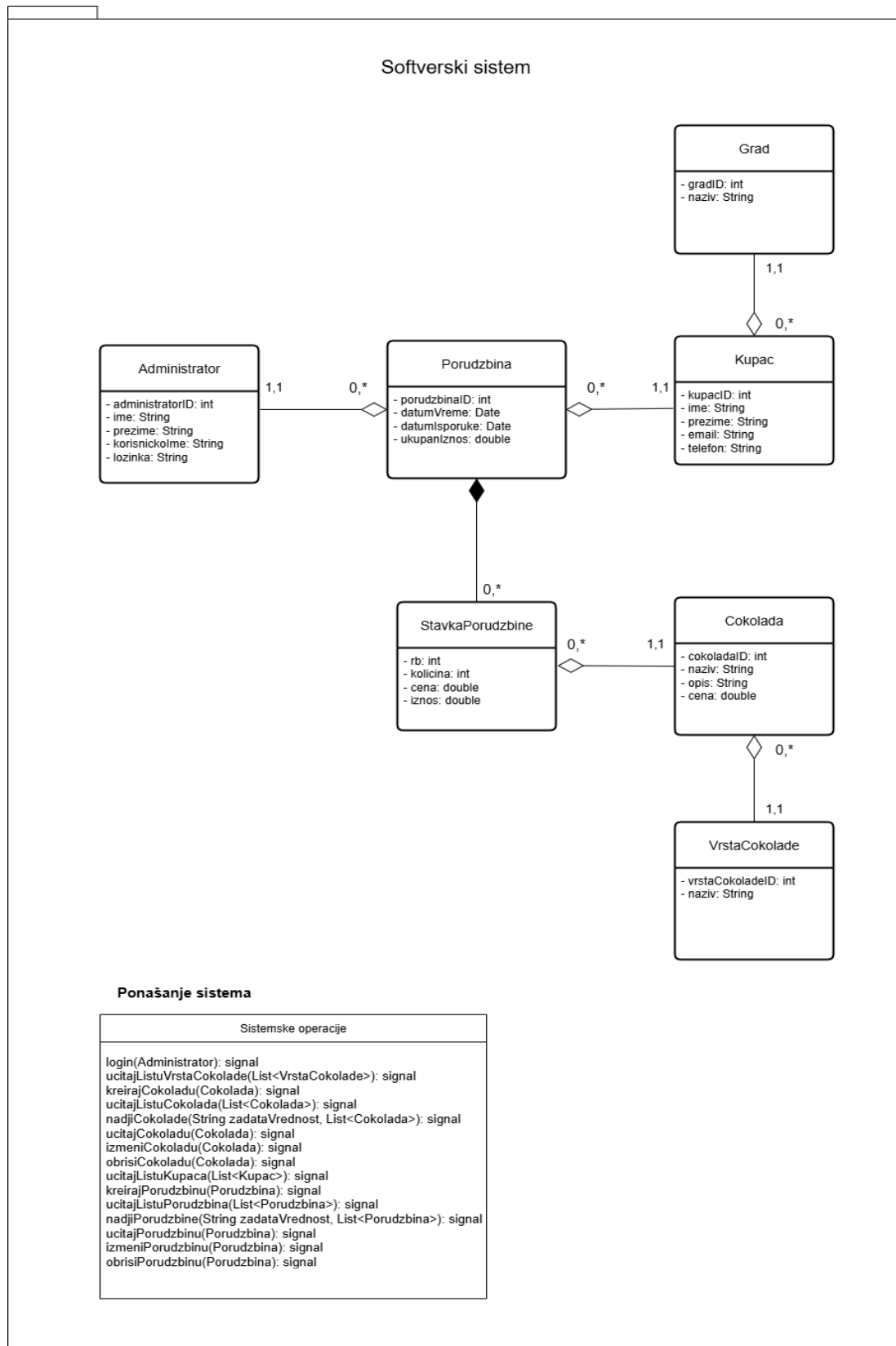
Tabela Grad		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT /
	GradID	int	Not null and > 0			UPDATE CASCADES Kupac
	Naziv	String	Not null			
						DELETE RESTRICTED Kupac

Tabela Kupac		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT RESTRICTED Grad
	KupacID	int	Not null and >0			UPDATE RESTRICTED Grad CASCADES Porudzbina
	Ime	String	Not null			
	Prezime	String	Not null			
	Email	String	Not null			
	Telefon	String	Not null			DELETE RESTRICTED Porudzbina
	GradID	int	Not null and > 0			

Tabela Porudzbina		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT RESTRICTED Kupac, Administrator
	PorudzbinaID	int	Not null and >0			UPDATE RESTRICTED Kupac, Administrator CASCADES StavkaPorudzbine
	DatumVreme	DateTime	Not null			
	DatumIsporuke	Date	Not null			
	UkupanIznos	double	Not null		Ukupan Iznos = SUM(StavkaPorudzbine. Iznos)	DELETE CASCADES StavkaPorudzbine
	KupacID	int	Not null and >0			
	AdministratorID	int	Not null and >0			

Tabela StavkaPorudzbine		Prosto vrednosno ograničenje		Složeno vrednosno ograničenje		Strukturno ograničenje
Atributi		Tip atributa	Vrednost atributa	Međuzavisnost atributa jedne tabele	Međuzavisnost atributa više tabela	INSERT RESTRICTED Porudzbina, Cokolada
	PorudzbinaID	int	Not null and >0			UPDATE RESTRICTED Porudzbina, Cokolada
	Rb	int	Not null			
	Kolicina	int	Not null			
	Cena	double	Not null			DELETE /
	Iznos	double	Not null	Iznos = Kolicina*Cena		
	CokoladaID	int	Not null and >0			

Као резултат анализе сценарија случајева коришћења и прављења концептуалног модела добија се логичка структура и понашање софтверског система.



Слика 35: Софтверски систем

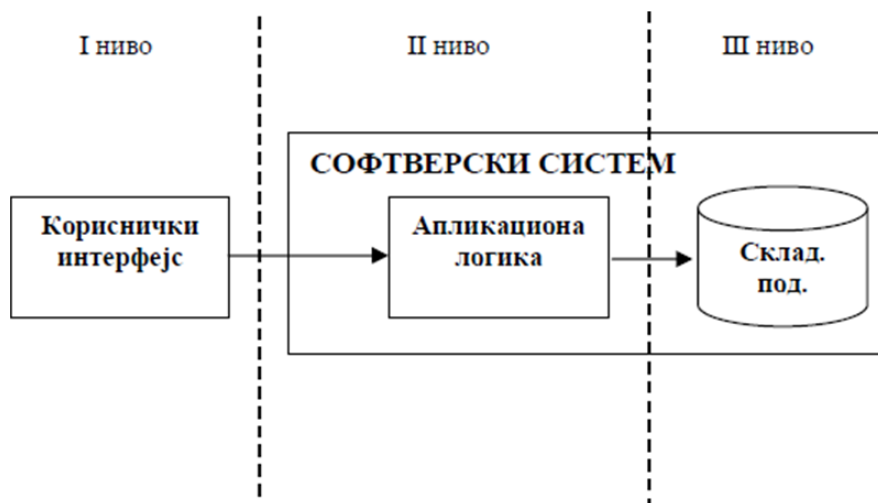
4.3.Фаза пројектовања

Фаза пројектовања описује физичку структуру и понашање софтверског система. Пројектовање архитектуре софтверског система обухвата пројектовање корисничког интерфејса (пројектовање контролера корисничког интерфејса и екранских форми), апликационе логике (пројектовање контролера апликационе логике и пословне логике) и складишта података (брокер базе података).

Архитектура система је тронивојска и састоји се од следећих нивоа:

- кориснички интерфејс
- апликациона логика
- складиште података

Ниво корисничког интерфејса ја на страни клијента, док су апликациона логика и складиште на страни сервера.

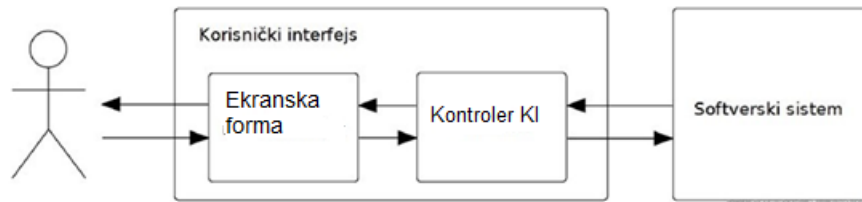


Слика 36: Тронивојска архитектура [1]

4.3.1 Пројектовање корисничког интерфејса

Кориснички интерфејс представља улазно-излазну реализацију софтверског система. Састоји се од:

1. Екранске форме
2. Контролера корисничког интерфејса



Слика 37: Структура корисничког интерфејса [1]

4.3.1.1 Пројектовање екранских форми

Кориснички интерфејс је дефинисан преко скупа екранских форми. Сценарио коришћења екранских форми је директно повезан са сценаријима случајева коришћења.

Постоје два аспекта пројектовања екранске форме:

1. Пројектовање сценарија СК који се изводе преко екранске форме
2. Пројектовање метода екранске форме

СК1: Случај коришћења - Пријава администратора

Назив СК

Пријављивање **администратора**

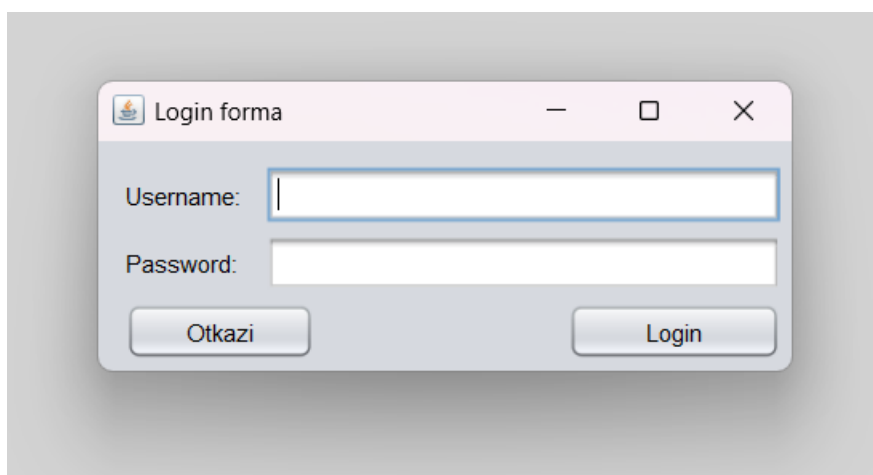
Актори СК

Администратор

Учесници СК

Администратор и **систем** (програм)

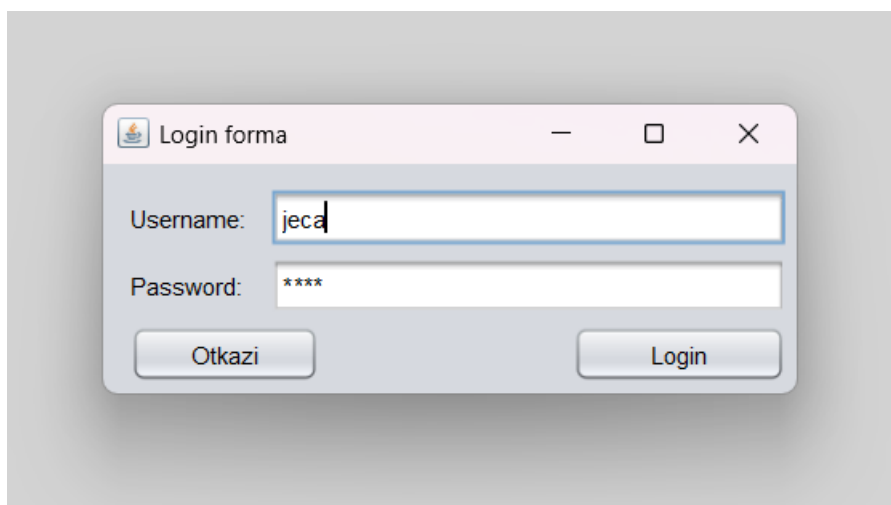
Предуслов: **Систем** је укључен и приказује форму за пријављивање **администратора**.

A screenshot of a software window titled "Login forma". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. Inside the window, there are two input fields: "Username:" and "Password:". The "Username:" field is currently empty and has a blue border. The "Password:" field is also empty. Below the input fields, there are two buttons: "Otkazi" (Cancel) on the left and "Login" on the right.

Слика 38: Форма за пријаву администратора на систем

Основни сценарио СК

1. **Администратор** уноси податке за пријаву. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке за пријаву. (АНСО)

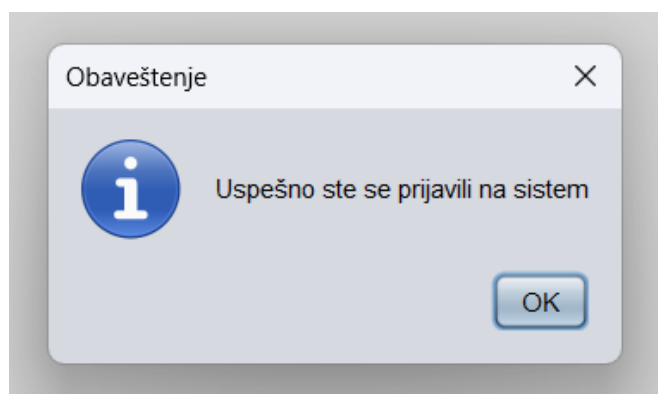
A screenshot of the same "Login forma" window. In this state, the "Username:" field is filled with the text "jeca". The "Password:" field is filled with five asterisks "*****". The "Otkazi" and "Login" buttons remain visible at the bottom.

Слика 39: Пријава администратора на систем

3. **Администратор** позива **систем** да пронађе **администратора** са задатим подацима. (АПСО)

Опис акције: Администратор кликом на дугме „Login“ позива системску операцију *login(Administrator)*.

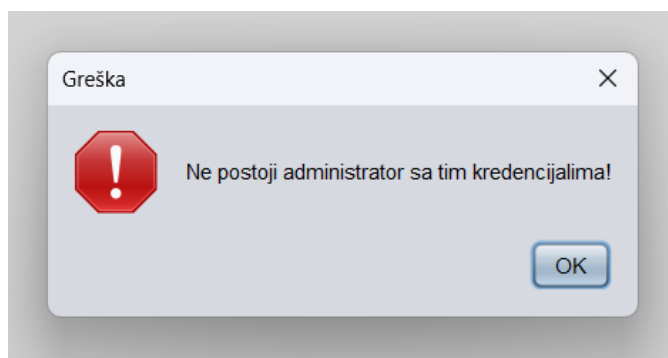
4. **Систем** претражује **администратора**. (СО)
5. **Систем** приказује поруку: „Успешно сте се пријавили на **систем**“. (ИА)



Слика 40: Успешна пијава администратора

Алтернативна сценарија

- 5.1. Уколико **систем** није пронашао **администратора**, приказује **администратору** поруку: „**Систем** не може да пронађе **администратора** на основу унетих података“. (ИА)



Слика 41: Неуспешна пријава администратора

СК2: Случај коришћења - Унос чоколаде

Назив СК

Унос нове чоколаде

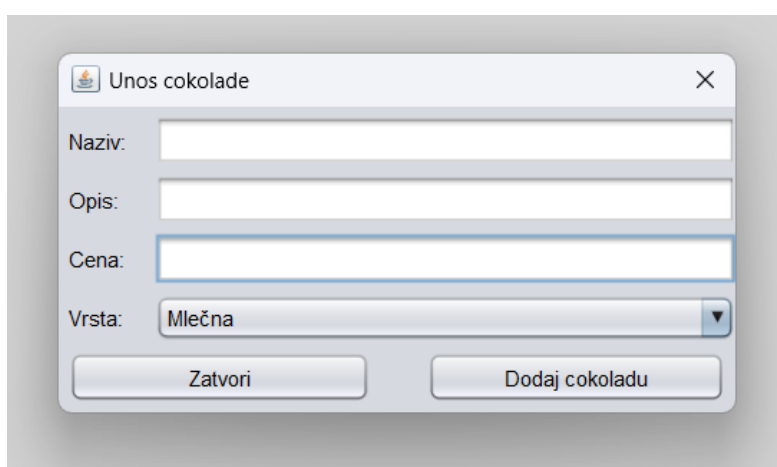
Актори СК

Администратор

Учесници СК

Администратор и **систем** (програм)

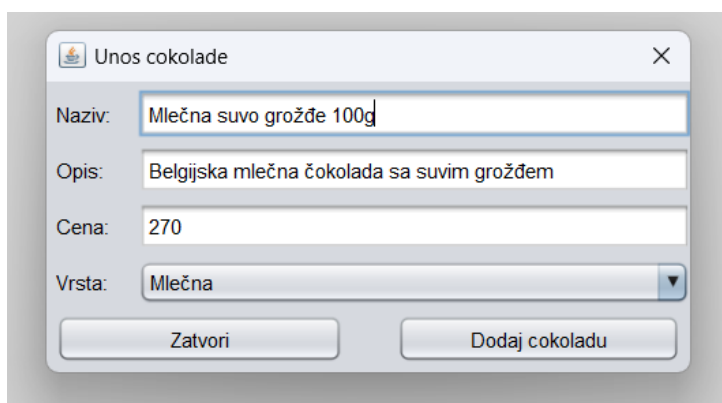
Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за рад са **чоколадама**. Учитана је листа врсти чоколаде.



Слика 42: Форма за унос нове чоколаде

Основни сценарио СК

1. **Администратор** уноси податке о **чоколади**. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке о **чоколади**. (АНСО)

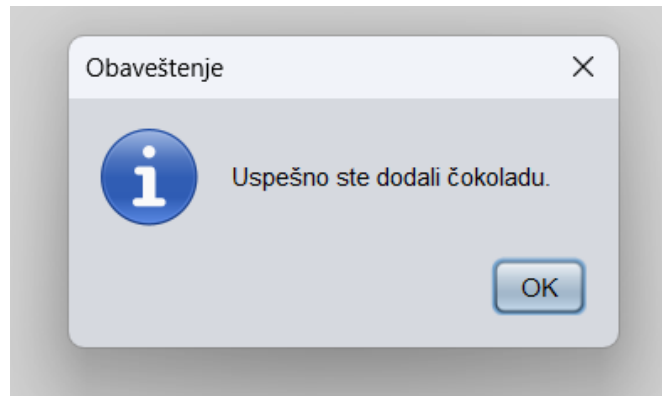


Слика 43: Креирање нове чоколаде од стране администратора

3. **Администратор** позива **систем** да креира нову **чоколаду**. (АПСО)

Опис акције: При самом отварању форме за унос чоколаде, иницијално се позива системска операција `ucitajListuVrstaCokolade(List<VrstaCokolade>)`, а учитане чоколаде се приказују у падајућој листи. Администратор кликом на дугме „Dodaj sokoladu“ позива системску операцију `kreirajCokoladu(Cokolada)`.

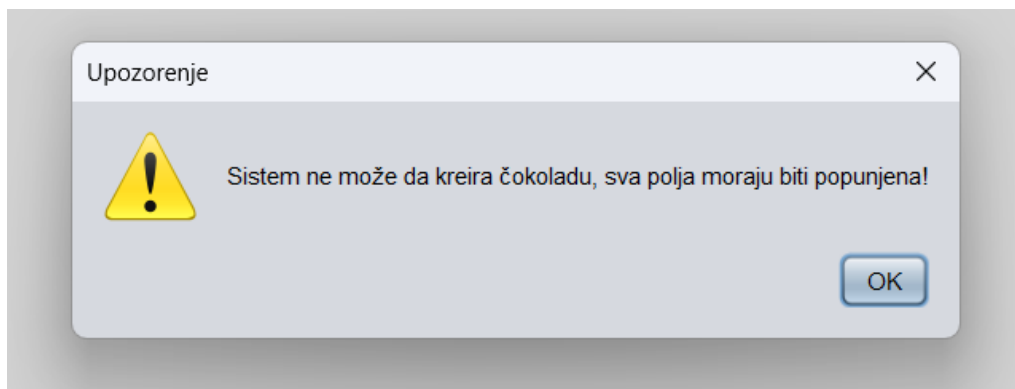
4. Систем креира нову чоколаду. (СО)
5. Систем приказује поруку: „Систем је успешно креирао нову чоколаду“. (ИА)



Слика 44: Порука о успешном креирању чоколаде

Алтернативна сценарија

- 5.1. Уколико систем не може да креира нову чоколаду, он приказује администратору поруку: „Систем не може да креира нову чоколаду“. (ИА)



Слика 45: Порука о неуспешном креирању чоколаде

СКЗ: Случај коришћења - Претрага чоколаде

Назив СК

Претрага чоколаде

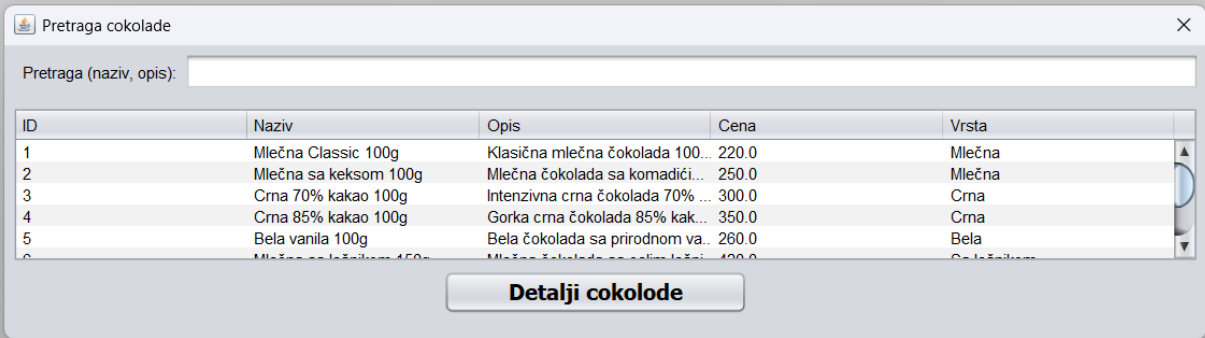
Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за претраживање чоколаде. Учитана је листа чоколада.



ID	Naziv	Opis	Cena	Vrsta
1	Mlečna Classic 100g	Klasična mlečna čokolada 100...	220.0	Mlečna
2	Mlečna sa keksom 100g	Mlečna čokolada sa komadići...	250.0	Mlečna
3	Crna 70% kakao 100g	Intenzivna crna čokolada 70% ...	300.0	Crna
4	Crna 85% kakao 100g	Gorka crna čokolada 85% kak...	350.0	Crna
5	Bela vanila 100g	Bela čokolada sa prirodnom va...	260.0	Bela

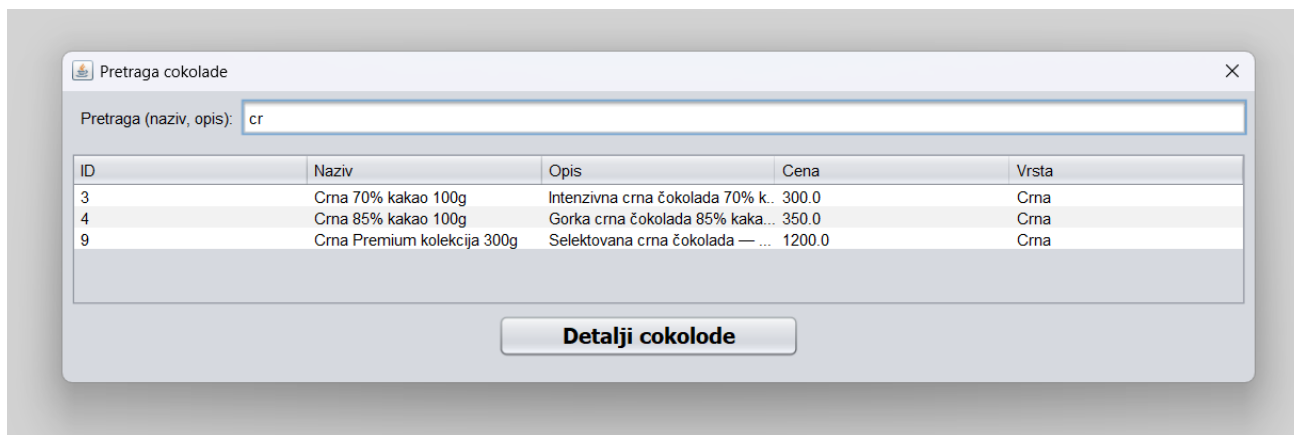
Слика 46: Форма за претрагу чоколаде

Основни сценарио СК

1. Администратор уноси вредност по којој претражује чоколаде.(АПУСО)
2. Администратор позива систем да нађе чоколаду по задатој вредности.(АПСО)

Опис акције: При самом отварању форме за претрагу чоколаде, иницијално се позива системска операција *ucitajListuCokolada(List<Cokolada>)*, а учитане чоколаде се приказују у табели. Администратор у поље претраге уноси критеријум по коме претражује чоколаде, при чему се позива системска операција *nadjiCokolade(String zadataVrednost, List<Cokolada>)*.

3. Систем тражи чоколаде по задатој вредности.(СО)
4. Систем приказује администратору податке о чоколадама и поруку: „Систем је нашао чоколаде по задатој вредности.”.(ИА)

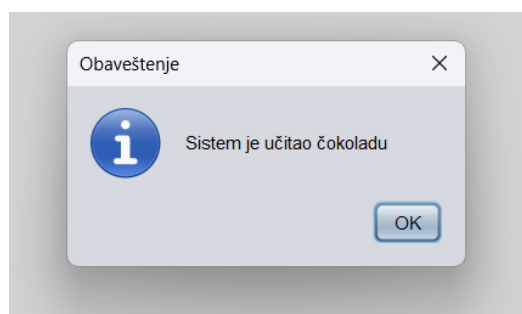


Слика 47: Систем тражи чоколаде по задатој вредности

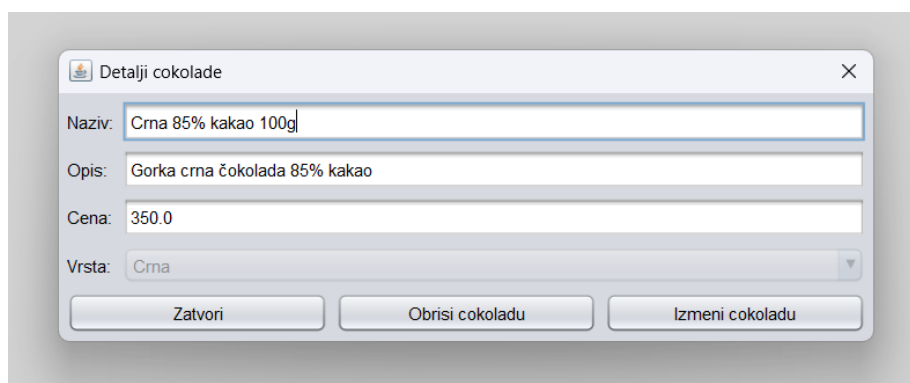
5. **Администратор** бира **чоколаду** коју жели да му **систем** прикаже. (АПУСО)
6. **Администратор** позива **систем** да учита **чоколаду**. (АПСО)

Опис акције: Администратор кликом на дугме „Detalji cokolade“ позива системску операцију *ucitajCokoladu(Cokolada)*.

7. **Систем** учитава **чоколаду**. (СО)
8. **Систем** приказује **администратору** податке о **чоколади** и поруку: „**Систем** је прочитао чоколаду“. (ИА)



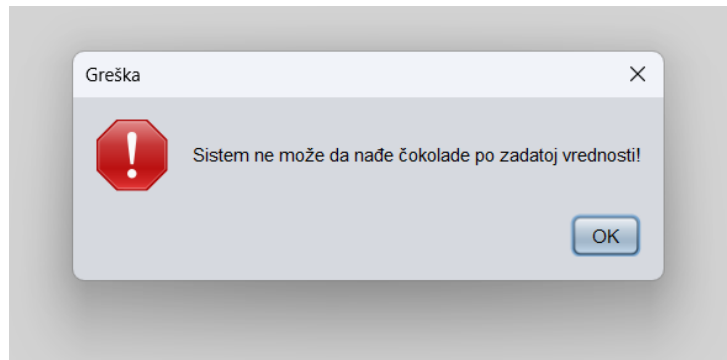
Слика 48: Порука о успешном учитавању чоколаде



Слика 49: Приказ података чоколаде

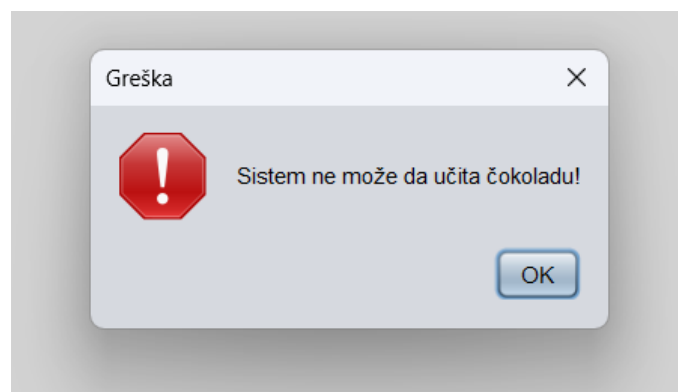
Алтернативна сценарија

- 4.1 Уколико **систем** не може да нађе **чоколаде** по задатој вредности, он приказује **администратору** поруку: „**Систем** не може да нађе **чоколаде** по задатој вредности.”. Прекида се извршење сценарија. (ИА)



Слика 50: Приказ поруке о неуспешном учитавању чоколада

- 8.1. Уколико **систем** не може да прочита **чоколаду** он приказује **администратору** поруку: „**Систем** не може да прочита **чоколаду**.”.(ИА)



Слика 51: Приказ поруке о неуспешном учитавању чоколаде

СК4: Случај коришћења - Измена чоколаде

Назив СК

Измена података **чоколаде**

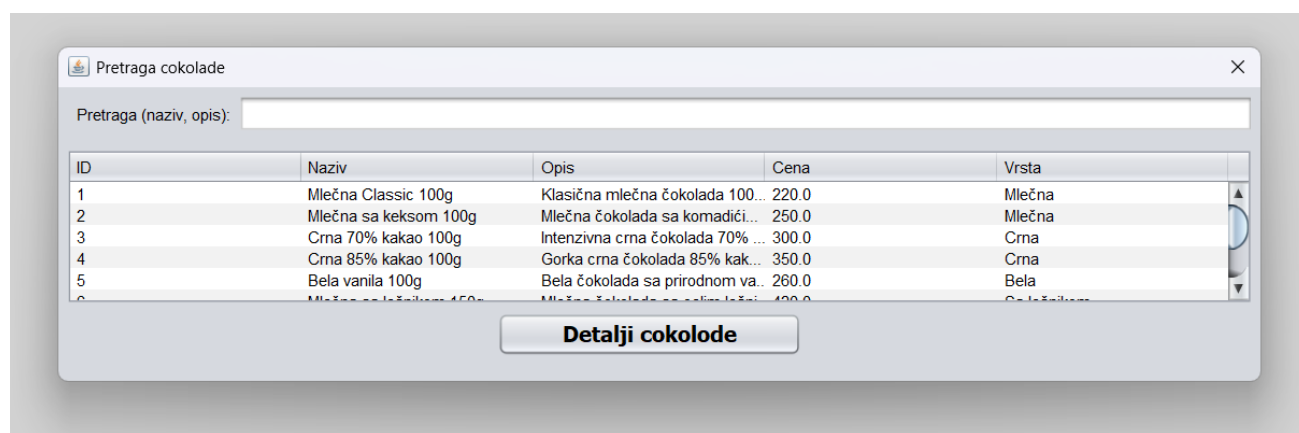
Актери СК

Администратор

Учесници СК

Администратор и **систем** (програм)

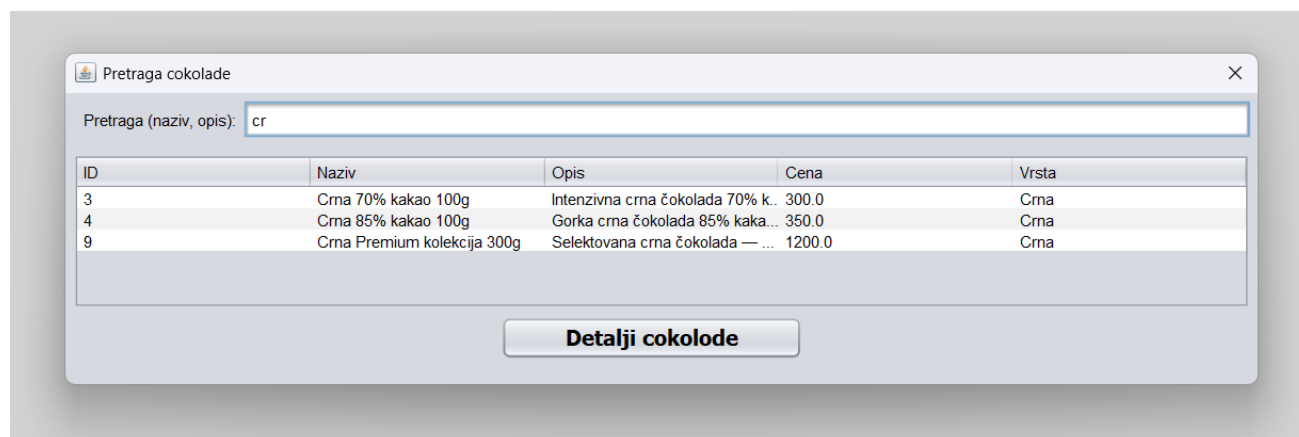
Предуслов: **Систем** је укључен и **администратор** је пријављен под својом шифром. **Систем** приказује форму за рад са **чоколадама**. Учитана је листа **чоколада**.



Слика 52: Форма за претрагу чоколаде

Основни сценарио СК

1. **Администратор** уноси вредност по којој претражује **чоколаде**. (АПУСО)

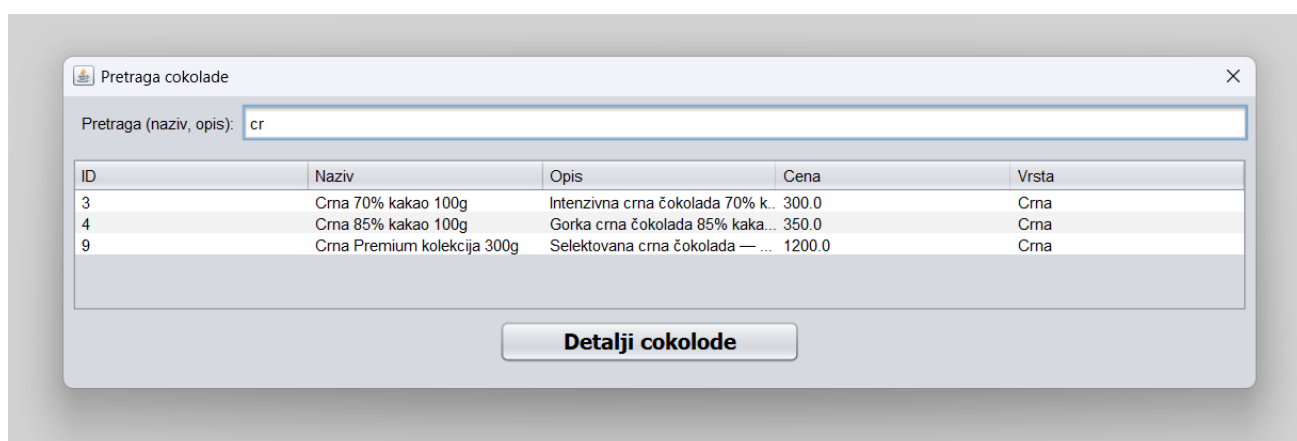


Слика 53: Унос вредности за претрагу чоколаде

2. **Администратор** контролише да ли је коректно унео податке за претрагу **чоколада**. (АНСО)
3. **Администратор** позива **систем** да пронађе **чоколаде** по задатој вредности.(АПСО)

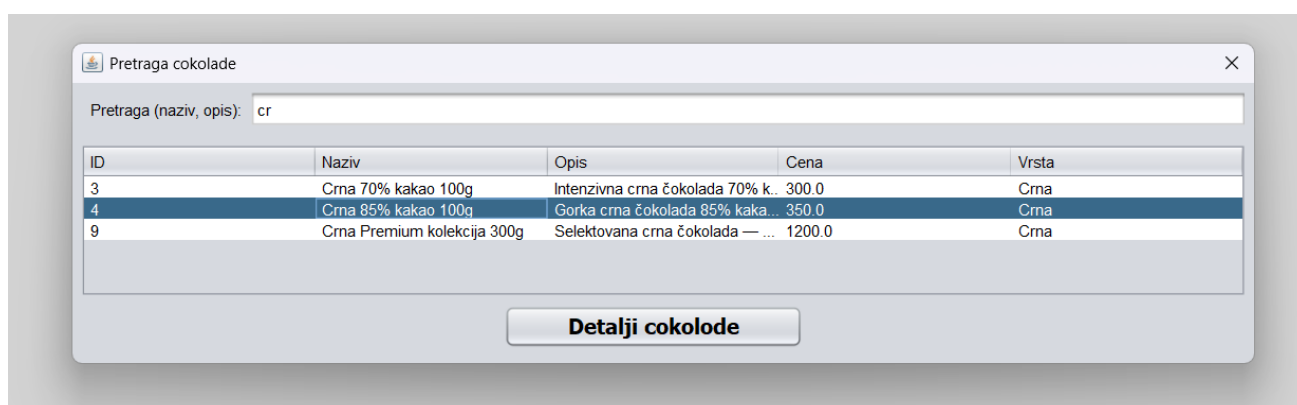
Опис акције: При самом отварању форме за претрагу чоколаде, иницијално се позива системска операција *ucitajListuCokolada(List<Cokolada>)*, а учитане чоколаде се приказују у табели. Администратор у поље претраге уноси критеријум по коме претражује чоколаде, при чему се позива системска операција *nadjiCokolade(String zadataVrednost, List<Cokolada>)*.

4. **Систем** тражи **чоколаде** по задатој вредности.(СО)
5. **Систем** приказује податке о траженим **чоколадама** уз поруку: „Подаци о траженим чоколадама су успешно учитани”.(ИА)



Слика 54: Приказ чоколада по задатој вредности

6. **Администратор** бира **чоколаду** коју жели да измени. (АПУСО)

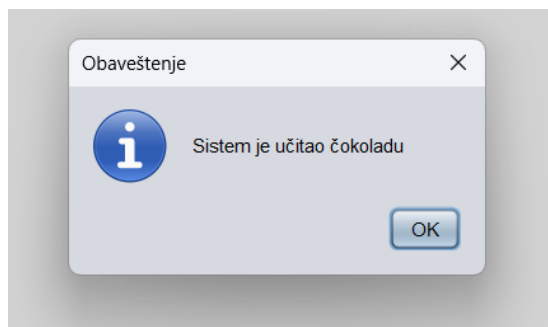


Слика 55: Одабир чоколаде за измену

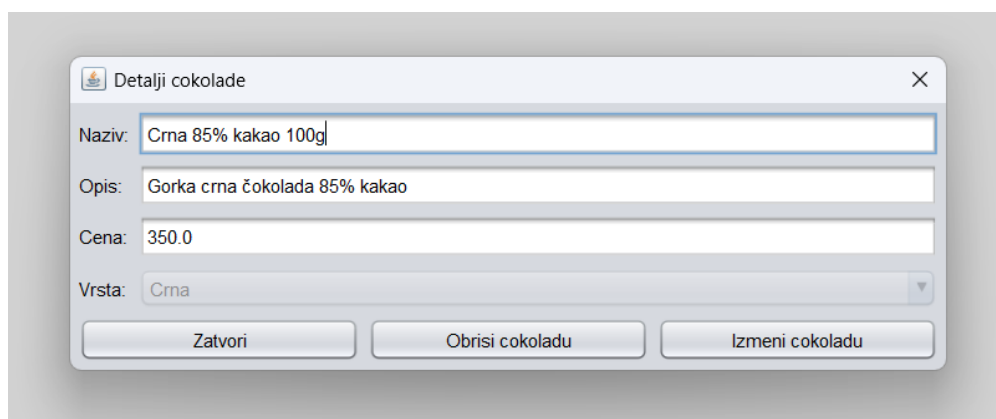
7. **Администратор** позива **систем** да пронађе изабрану **чоколаду**. (АПСО)

Опис акције: Администратор кликом на дугме „*Detalji cokolade*“ позива системску операцију *ucitajCokoladu(Cokolada)*.

8. **Систем** проналази изабрану **чоколаду**. (СО)
9. **Систем** приказује податке о траженој **чоколади** уз поруку: „**Систем** је учитао **чоколаду**“. (ИА)

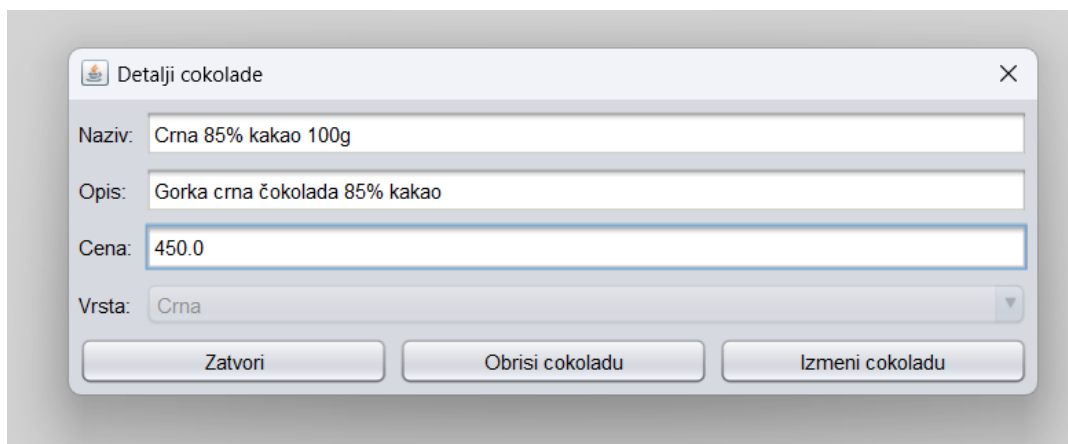


Слика 56: Порука о успешном учитавању чоколаде



Слика 57: Приказ података изабране чоколаде

10. **Администратор** мења податке о **чоколади**. (АПУСО)
11. **Администратор** контролише да ли је коректно унео податке о **чоколади**. (АНСО)



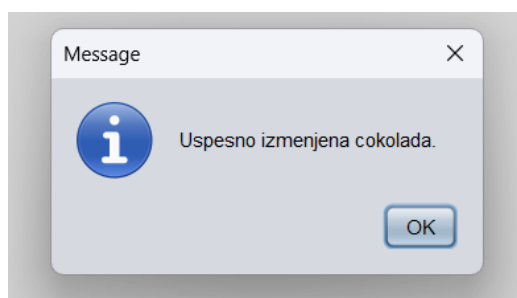
Слика 58: Администратор мења податке (цену) чоколаде

12. **Администратор** **позива** систем да запамти податке о **чоколади**.(АПСО)

Опис акције: Администратор кликом на дугме „Izmeni sokoladu“ позива системску операцију *izmeniCokoladu(Cokolada)*.

13. **Систем** **памти** податке о **чоколади**.(СО)

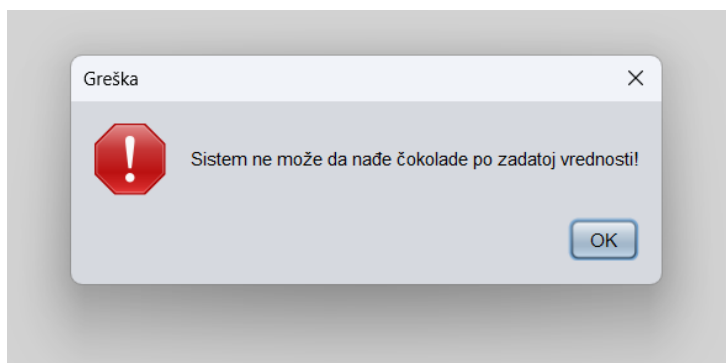
14. **Систем** **приказује администратору** измењену **чоколаду** уз поруку: „**Систем** је изменио податке **чоколаде**“. (ИА)



Слика 59: Порука о успешној измени чоколаде

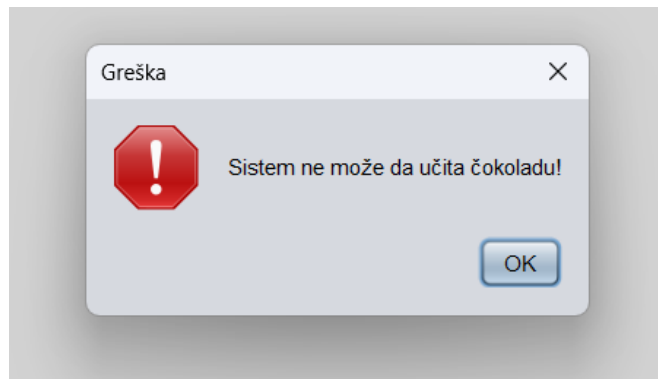
Алтернативна сценарија

5.1. Уколико **систем** не може да пронађе **чоколаде**, приказује следећу поруку **администратору**: „**Систем** не може да пронађе **чоколаде** на основу унетих података“. Прекида се извршење сценарија. (ИА)



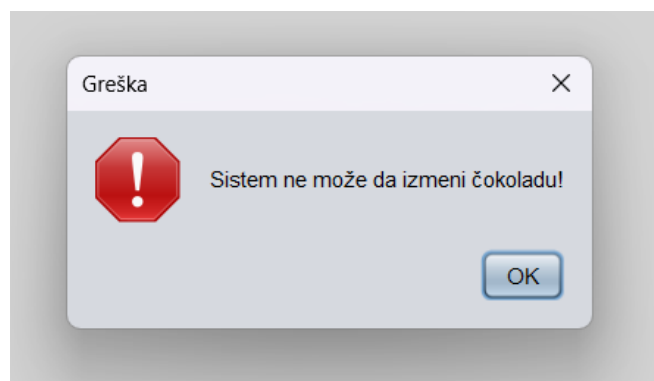
Слика 60: Приказ поруке о неуспешном учитавању чоколада

9.1 Уколико **систем** не може да прикаже податке о чоколади, приказује следећу поруку **администратору**: „**Систем** не може да пронађе **чоколаду**“. Прекида се извршење сценарија. (ИА)



Слика 61: Приказ поруке о неуспешном учитавању чоколаде

14.1. Уколико **систем** не може да измени **чоколаду**, он приказује следећу поруку **администратору**: „**Систем** не може да измени **чоколаду**“. (ИА)



Слика 62: Приказ поруке о неуспешној измени чоколаде

СК5: Случај коришћења - Брисање чоколаде

Назив СК

Брисање чоколаде

Актери СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за рад са чоколадама. Учитана је листа чоколада.

ID	Naziv	Opis	Cena	Vrsta
1	Mlečna Classic 100g	Klasična mlečna čokolada 100...	220.0	Mlečna
2	Mlečna sa keksom 100g	Mlečna čokolada sa komadići...	250.0	Mlečna
3	Crna 70% kakao 100g	Intenzivna crna čokolada 70% ...	300.0	Crna
4	Crna 85% kakao 100g	Gorka crna čokolada 85% kak...	350.0	Crna
5	Bela vanila 100g	Bela čokolada sa prirodnom va...	280.0	Bela

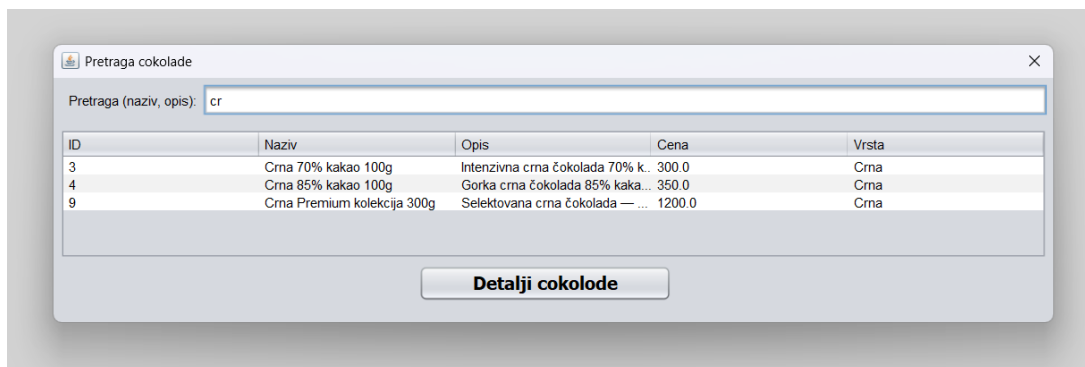
Слика 63: Форма за рад са чоколадама

Основни сценарио СК

1. Администратор уноси вредност по којој претражује чоколаде. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за претрагу чоколада. (АНСО)
3. Администратор позива систем да пронађе чоколаде по задатој вредности. (АПСО)

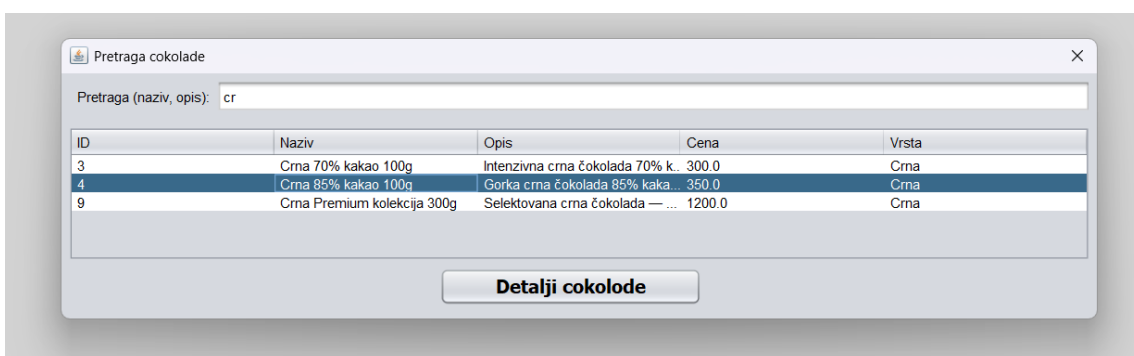
Опис акције: При самом отварању форме за претрагу чоколада, иницијално се позива системска операција *ucitajListuCokolada(List<Cokolada>)*, а учитане чоколаде се приказују у табели. Администратор у поље претраге уноси критеријум по коме претражује чоколаде, при чему се позива системска операција *nadjiCokolade(String zadataVrednost, List<Cokolada>)*.

4. Систем тражи чоколаде по задатој вредности. (СО)
5. Систем приказује чоколаде администратору и поруку: „Систем је пронашао чоколаде по задатој вредности“. (ИА)



Слика 64: Учитавање чоколада по задатој вредности

6. **Администратор бира чоколаду** коју жели да избрише. (АПУСО)

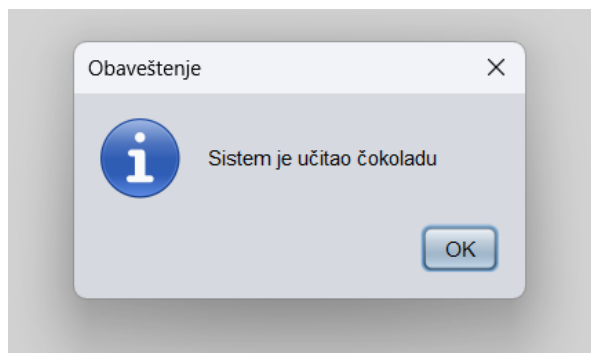


Слика 65: Администратор бира чоколаду за брисање

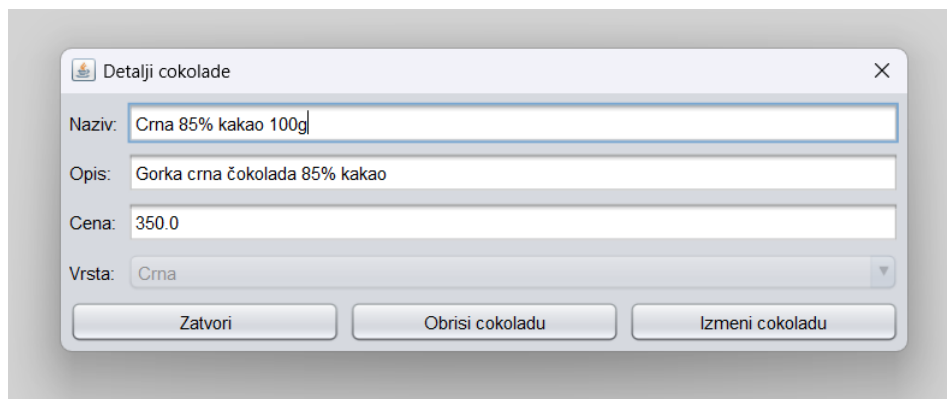
7. **Администратор позива систем** да нађе изабрану чоколаду. (АПСО)

Опис акције: Администратор кликом на дугме „*Detalji cokolade*“ позива системску операцију *ucitajCokoladu(Cokolada)*.

8. **Систем проналази** изабрану чоколаду. (СО)
 9. **Систем приказује** чоколаду и поруку: „Систем је пронашао чоколаду“. (ИА)



Слика 66: Успешно учитавање чоколаде



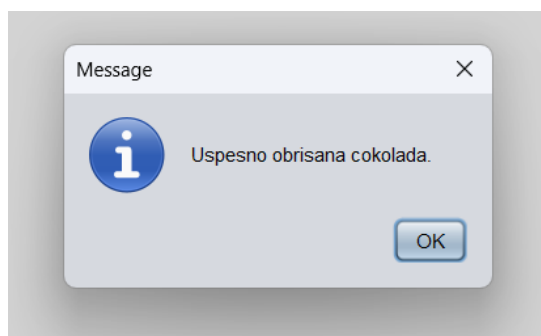
Слика 67: Детаљи изабране чоколаде

10. **Администратор** позива **систем** да обрише **чоколаду**. (АПСО)

Опис акције: Администратор кликом на дугме „Obrisi cokoladu“ позива системску операцију *obrisiCokoladu(Cokolada)*.

11. **Систем** брише **чоколаду**. (СО)

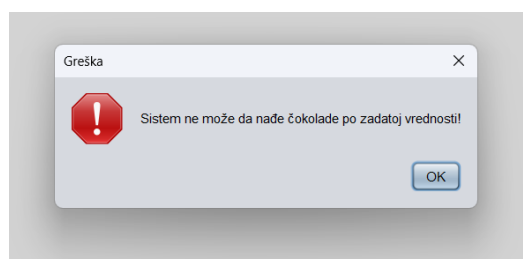
12. **Систем** приказује **администратору** поруку: „Систем је обрисао **чоколаду**“. (ИА)



Слика 68: Успешно брисање чоколаде

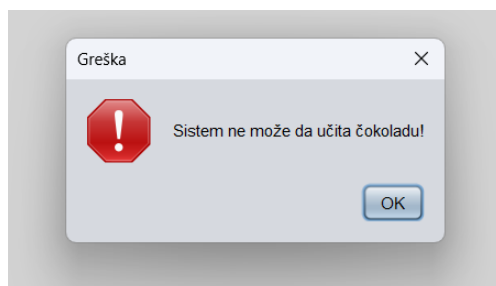
Алтернативна сценарија:

5.1. Уколико **систем** не може да нађе **чоколаде**, он приказује **администратору** поруку: „Систем не може да пронађе **чоколаде** по задатој вредности“. Прекида се извршење сценарија. (ИА)



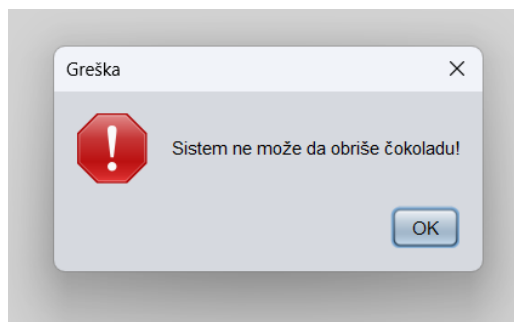
Слика 69: Порука о неуспешном учитавању чоколада

9.1. Уколико **систем** не може да нађе **чоколаду**, он приказује **администратору** поруку: „**Систем** не може да учита **чоколаду**.”. Прекида се извршење сценарија. (ИА)



Слика 70: Порука о неуспешном учитавању чоколаде

12.1. Уколико **систем** не може да обрише **чоколаду** он приказује **администратору** поруку: „**Систем** не може да обрише **чоколаду** “. (ИА)



Слика 71: Порука о неуспешном брисању чоколаде

СК6: Случај коришћења - Унос поруџбине (сложен СК)

Назив СК

Унос **поруџбине**

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и **администратор** је пријављен под својом шифром. Систем приказује форму за рад са **поруџбинама**. Учитане су листе купаца и чоколада.

Klijentska forma

Kupac: Cokolada Porudzbina Odjava

Ulogovani administrator: Jelena Cavić

Porudzbina

Kupac: Luka Lukić

Datum isporuke: 20.04.2026

Ukupani iznos:

Sacuvaj porudzbину

Stavke porudzbine

Cokolada: Mlečna Classic 100g (Cena: 220.0din)

Kolicina:

Iznos:

Dodaj stavku Obrisi stavku

Rb	Cokolada	Cena po komadu	Kolicina	Iznos
----	----------	----------------	----------	-------

Слика 72: Форма за рад са поруџбинама

Основни сценарио СК

1. **Администратор** уноси податке о **поруџбини**. (АПУСО)
2. **Администратор** контролише да ли је коректно унео податке о **поруџбини**. (АНСО)

Klijentska forma

Kupac: Cokolada Porudzbina Odjava

Ulogovani administrator: Jelena Cavić

Porudzbina

Kupac: Luka Lukić

Datum isporuke: 20.04.2026

Ukupani iznos: 1800.0din

Sacuvaj porudzbину

Stavke porudzbine

Cokolada: Crna 85% kakaо 100g (Cena: 450.0din)

Kolicina: 4

Iznos: 1800.0

Dodaj stavku Obrisi stavku

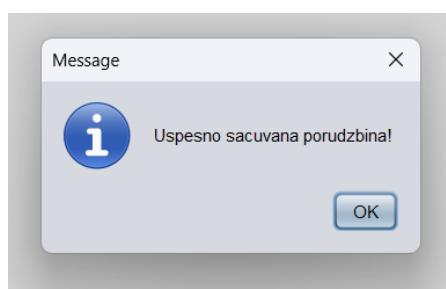
Rb	Cokolada	Cena po komadu	Kolicina	Iznos
1	Crna 85% kaka...	450.0din	4kom	1800.0

Слика 73: Унос података о поруџбини

3. **Администратор** позива систем да креира **поруџбину**. (АПСО)

Опис акције: Администратор кликом на дугме „*Sacuvaj porudzbini*“ позива системску операцију *kreirajPorudzbini(Porudzbina)*.

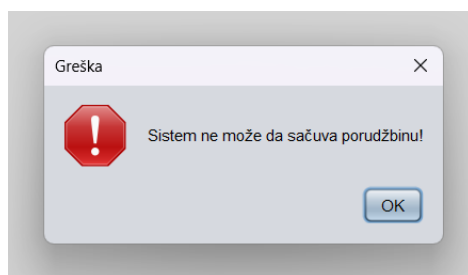
4. **Систем** креира **поруџбину**. (СО)
5. **Систем** приказује **администратору** креирану **поруџбину** и поруку: „**Систем** је креирао **поруџбину**.“. (ИА)



Слика 74: Успешно додавање поруџбине

Алтернативна сценарија:

- 5.1. Уколико **систем** не може да креира **поруџбину** он приказује **администратору** поруку: „**Систем** не може да креира **поруџбину**“. (ИА)



Слика 75: Порука о неуспешном додавању чоколаде

СК7: Случај коришћења - Претрага поруџбине

Назив СК

Претрага поруџбине

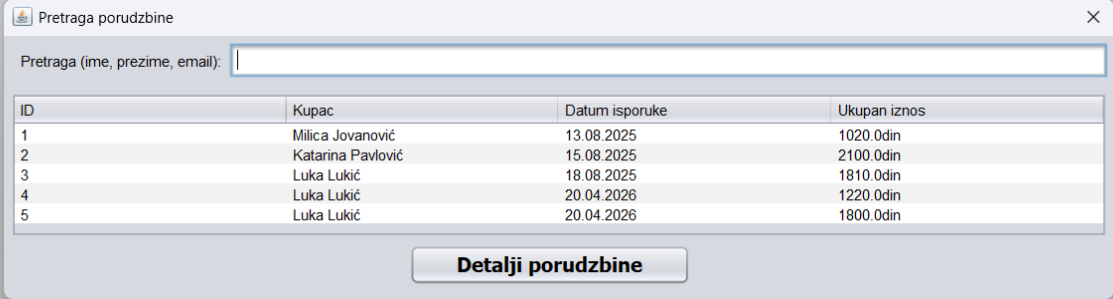
Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за претраживање поруџбина. Учитана је листа поруџбина.



ID	Kupac	Datum isporuke	Ukupan iznos
1	Milica Jovanović	13.08.2025	1020.0din
2	Katarina Pavlović	15.08.2025	2100.0din
3	Luka Lukić	18.08.2025	1810.0din
4	Luka Lukić	20.04.2026	1220.0din
5	Luka Lukić	20.04.2026	1800.0din

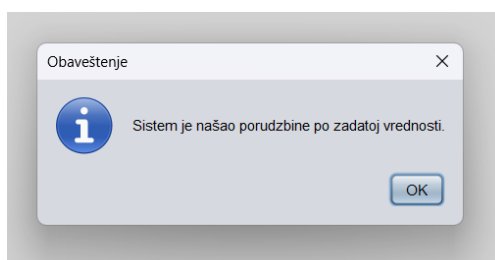
Слика 76: Форма за претрагу поруџбина

Основни сценарио СК

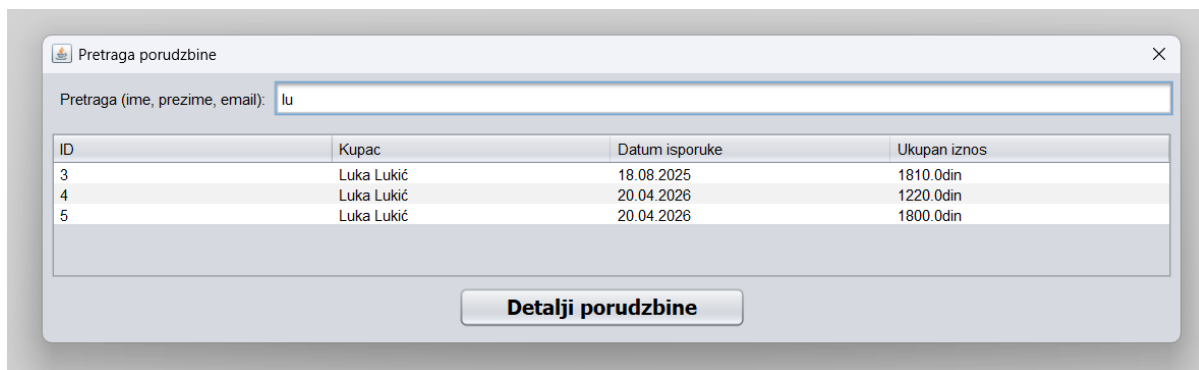
1. Администратор уноси вредност по којој претражује поруџбине. (АПУСО)
2. Администратор позива систем да нађе поруџбине по задатој вредности. (АПСО)

Опис акције: При самом отварању форме за претрагу чоколаде, иницијално се позива системска операција `ucitajListuPorudzbina(List<Porudzbina>)`, а учитане поруџбине се приказују у табели. Администратор у поље претраге уноси критеријум по коме претражује чоколаде, при чему се позива системска операција `nadjiPorudzbine(String zadataVrednost, List<Porudzbina>)`.

3. Систем тражи поруџбине по задатој вредности. (СО)
4. Систем приказује администратору податке о поруџбинама и поруку: „Систем је нашао поруџбине по задатој вредности.“. (ИА)



Слика 77: Успешно учитавање поруџбина

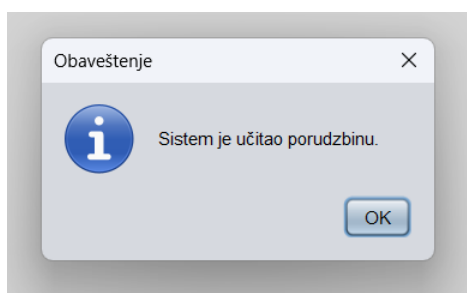


Слика 78: Систем проналази поруџбине по задатој вредности

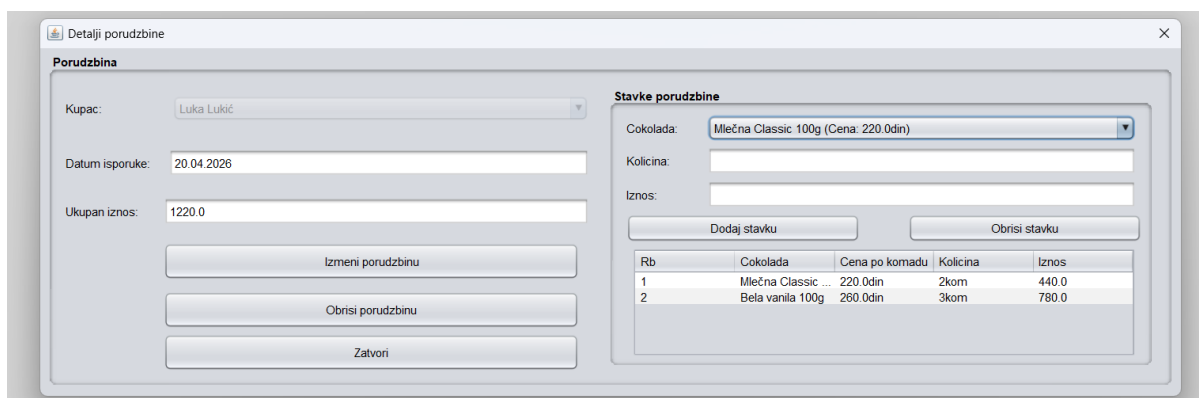
5. **Администратор** бира **поруџбину** коју жели да му систем прикаже. (АПУСО)
6. **Администратор** позива **систем** да учита **поруџбину**. (АПСО)

Опис акције: Администратор кликом на дугме „Detalji porudzbine“ позива системску операцију *ucitajPorudzbinu(Porudzbina)*.

7. **Систем** учитава **поруџбину**. (СО)
8. **Систем** приказује **администратору** податке о **поруџбини** и поруку: „Систем је прочитао **поруџбину**.“. (ИА)



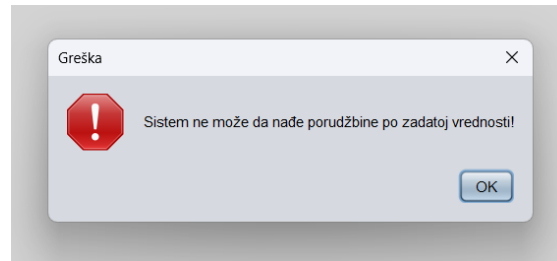
Слика 79: Успешно учитавање поруџбине



Слика 80: Детаљи поруџбине

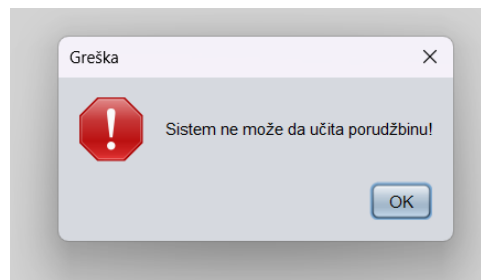
Алтернативна сценарија:

4.1 Уколико **систем** не може да нађе **поруџбине** по задатој вредности, он приказује **администратору** поруку: „**Систем** не може да нађе **поруџбине** по задатој вредности.“. Прекида се извршење сценарија. (ИА)



Слика 81: Порука о неуспешној претрази поруџбине

8.1 Уколико **систем** не може да учита **поруџбину** приказује **администратору** поруку: „**Систем** не може да учита **поруџбину**.“.(ИА)



Слика 82: Порука о неуспешном учитавању поруџбине

СК8: Случај коришћења - Измена поруџбине (сложен СК)

Назив СК

Измена поруџбине

Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за рад са поруџбинама. Учитане су листе поруџбина и чоколада.

ID	Kupac	Datum isporuke	Ukupan iznos
1	Milica Jovanović	13.08.2025	1020.0din
2	Katarina Pavlović	15.08.2025	2100.0din
3	Luka Lukić	18.08.2025	1810.0din
4	Luka Lukić	20.04.2026	1220.0din
5	Luka Lukić	20.04.2026	1800.0din

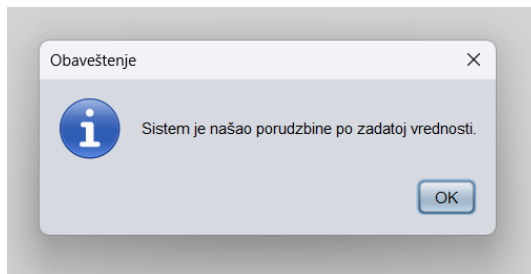
Слика 83: Форма за претрагу поруџбина

Основни сценарио СК

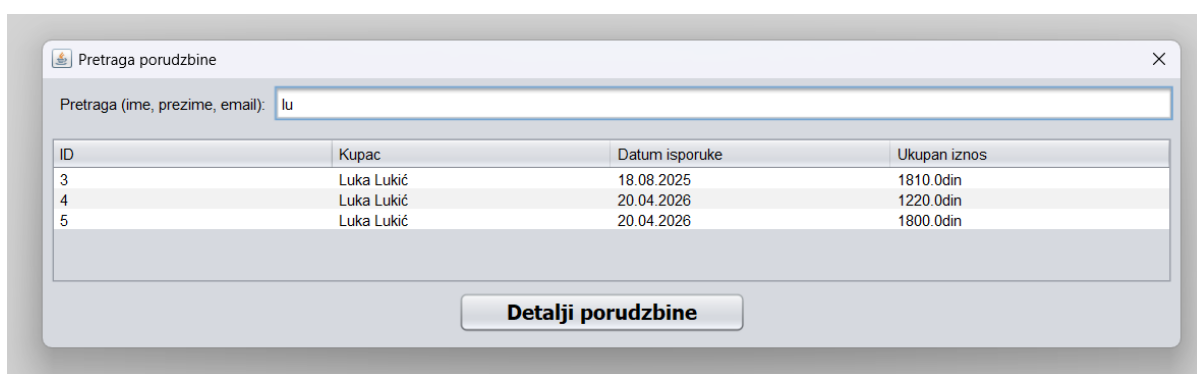
1. Администратор уноси вредност по којој претражује поруџбине. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за претрагу поруџбина. (АНСО)
3. Администратор позива систем да нађе поруџбине по задатој вредности. (АПСО)
4. Систем тражи поруџбине по задатој вредности. (СО)

Опис акције: При самом отварању форме за претрагу чоколада, иницијално се позива системска операција `ucitajListuPorudzbina(List<Porudzbina>)`, а учитане поруџбине се приказују у табели. Администратор у поље претраге уноси критеријум по коме претражује чоколаде, при чему се позива системска операција `nadjiPorudzbine(String zadataVrednost, List<Porudzbina>)`.

5. **Систем** приказује **администратору** поруџбине и поруку: „**Систем** је нашао поруџбине по задатој вредности.“. (ИА)



Слика 84: Успешно учитавање поруџбина

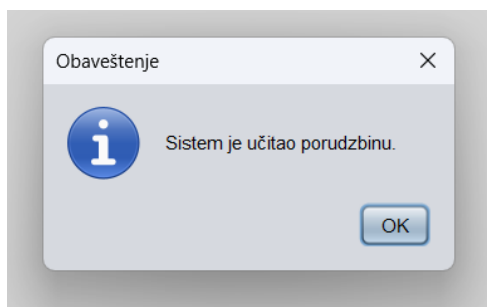


Слика 85: Систем проналази поруџбине по задатој вредности

6. **Администратор** бира **поруџбину** коју жели да измени. (АПУСО)
7. **Администратор** позива **систем** да нађе изабрану **поруџбину**. (АПСО)

Опис акције: Администратор кликом на дугме „*Detalji porudzbine*“ позива системску операцију *ucitajPorudzbinu(Porudzbina)*.

8. **Систем** проналази изабрану **поруџбину**. (СО)
9. **Систем** приказује **администратору** **поруџбину** и поруку: „**Систем** је прочитао **поруџбину**“. (ИА)



Слика 86: Успешно учитавање поруџбине

Detalji porudzbine

Porudzbina

Kupac: Luka Lukić

Datum isporuke: 20.04.2026

Ukupan iznos: 1220.0

Izmeni porudzbinu

Obrisi porudzbinu

Zatvori

Stavke porudzbine

Cokolada: Mlečna Classic 100g (Cena: 220.0din)

Kolicina:

Iznos:

Dodaj stavku

Obrisi stavku

Rb	Cokolada	Cena po komadu	Kolicina	Iznos
1	Mlečna Classic ...	220.0din	2kom	440.0
2	Bela vanila 100g	260.0din	3kom	780.0

Слика 87: Детаљи поруџбине

10. **Администратор** мења податке о поруџбини. (АПУСО)
11. **Администратор** контролише да ли је коректно унео податке о поруџбини. (АНСО)

Detalji porudzbine

Porudzbina

Kupac: Luka Lukić

Datum isporuke: 10.04.2026

Ukupan iznos: 2050.0din

Izmeni porudzbinu

Obrisi porudzbinu

Zatvori

Stavke porudzbine

Cokolada: Vočna jagoda 120g (Cena: 380.0din)

Kolicina: 1

Iznos: 380.0

Dodaj stavku

Obrisi stavku

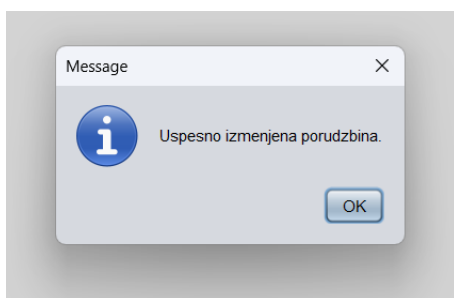
Rb	Cokolada	Cena po komadu	Kolicina	Iznos
1	Mlečna Classic ...	220.0din	2kom	440.0
2	Bela vanila 100g	260.0din	3kom	780.0
3	Crna 85% kaka...	450.0din	1kom	450.0
4	Vočna jagoda 12...	380.0din	1kom	380.0

Слика 88: Промена података поруџбине

12. **Администратор** позива_систем да запамти податке о поруџбини. (АПСО)

Опис акције: Администратор кликом на дугме „Izmeni porudzbinu“ позива системску операцију *izmeniPorudzbinu(Porudzbina)*.

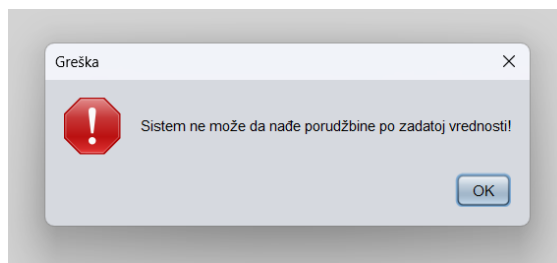
13. **Систем** памти податке о поруџбини. (СО)
14. **Систем** приказује **администратору** поруку: „Систем је изменио поруџбину.“. (ИА)



Слика 89: Порука о успешној измени поруџбине

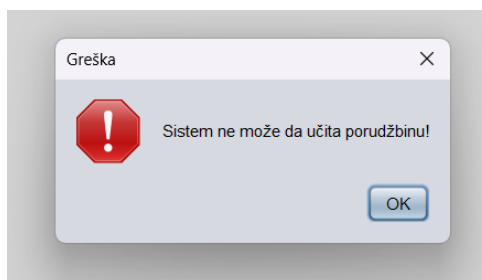
Алтернативна сценарија:

- 5.1 Уколико **систем** не може да пронађе **поруџбине** он приказује **администратору** поруку: „**Систем** не може да пронађе **поруџбине** по задатој вредности”. Прекида се извршење сценарија. (ИА)



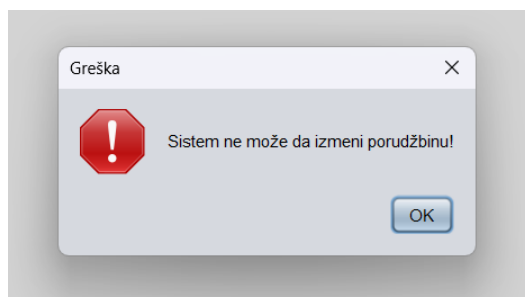
Слика 90: Порука о неуспешној претрази поруџбине

- 9.1. Уколико **систем** не може да прикаже податке о **поруџбини**, приказује следећу поруку **администратору**: „**Систем** не може учита **поруџбину**.”. Прекида се извршење сценарија. (ИА)



Слика 91: Порука о неуспешном учитавању поруџбине

- 14.1 Уколико **систем** не може да измени податке о **поруџбини** он приказује **администратору** поруку: „**Систем** не може да измени **поруџбину**”. (ИА)



Слика 92: Порука о неуспешној измени поруџбине

СК9: Случај коришћења - Брисање поруџбине

Назив СК

Брисање поруџбине

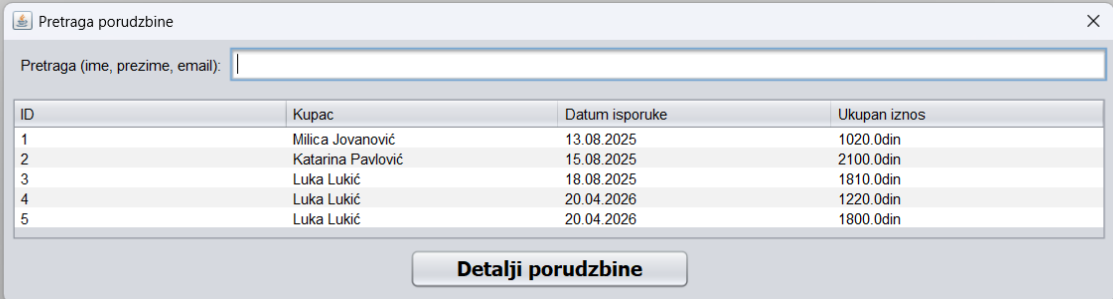
Актори СК

Администратор

Учесници СК

Администратор и систем (програм)

Предуслов: Систем је укључен и администратор је пријављен под својом шифром. Систем приказује форму за рад са поруџбинама. Учитана је листа поруџбина.



ID	Kupac	Datum isporuke	Ukupan iznos
1	Milica Jovanović	13.08.2025	1020.0din
2	Katarina Pavlović	15.08.2025	2100.0din
3	Luka Lukić	18.08.2025	1810.0din
4	Luka Lukić	20.04.2026	1220.0din
5	Luka Lukić	20.04.2026	1800.0din

Слика 93: Форма за претрагу поруџбина

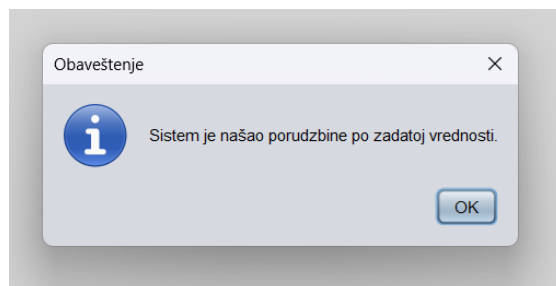
Основни сценарио СК

1. Администратор уноси вредност по којој претражује поруџбине. (АПУСО)
2. Администратор контролише да ли је коректно унео податке за претрагу поруџбина. (АНСО)
3. Администратор позива систем да нађе поруџбине по задатој вредности. (АПСО)

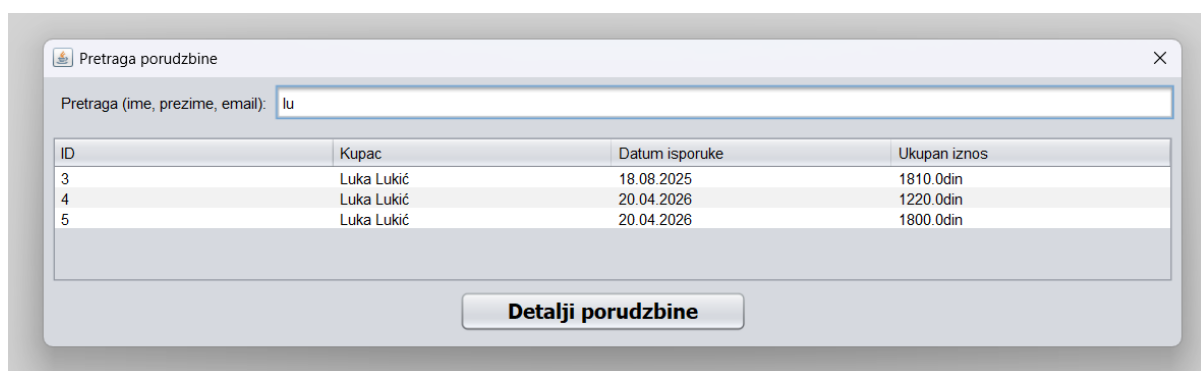
Опис акције: При самом отварању форме за претрагу чоколаде, иницијално се позива системска операција `ucitajListuPorudzbina(List<Porudzbina>)`, а учитане поруџбине се приказују у табели. Администратор у поље претраге уноси критеријум по коме претражује чоколаде, при чему се позива системска операција `nadjiPorudzbine(String zadataVrednost, List<Porudzbina>)`.

4. Систем тражи поруџбине по задатој вредности. (СО)

5. **Систем** приказује **поруџбине администратору** и поруку: „**Систем** је нашао **поруџбине** по задатој вредности“. (ИА)



Слика 94: Успешно учитавање поруџбина

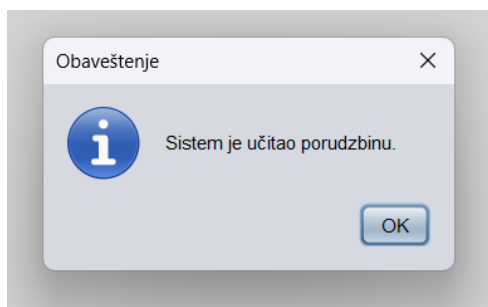


Слика 95: Систем проналази поруџбине по задатој вредности

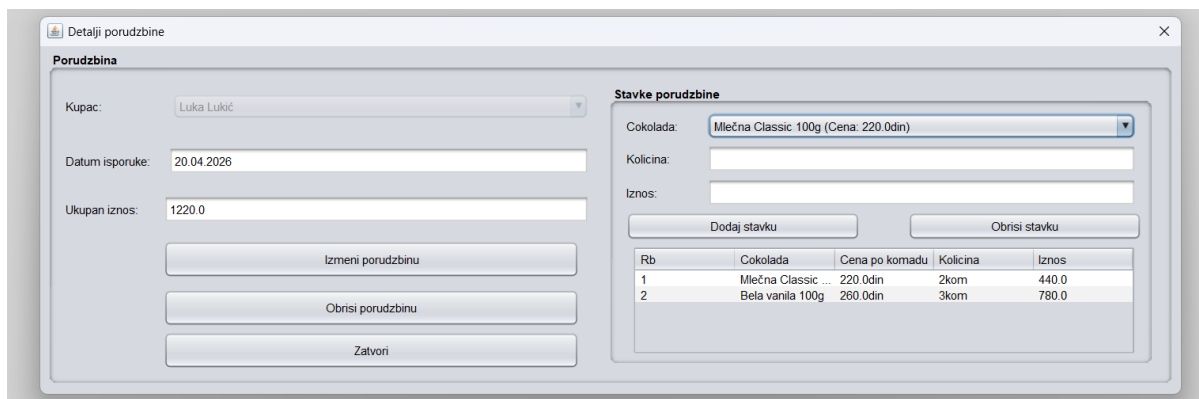
6. **Администратор** бира **поруџбину** који жели да избрише. (АПУСО)
7. **Администратор** позива **систем** да нађе изабрану **поруџбину**. (АПСО)

Опис акције: Администратор кликом на дугме „*Detalji porudzbine*“ позива системску операцију *ucitajPorudzbinu(Porudzbina)*.

8. **Систем** проналази изабрану **поруџбину**. (СО)
9. **Систем** приказује **администратору** **поруџбину** и поруку: „**Систем** је прочитао **поруџбину**“. (ИА)



Слика 96: Успешно учитавање поруџбине



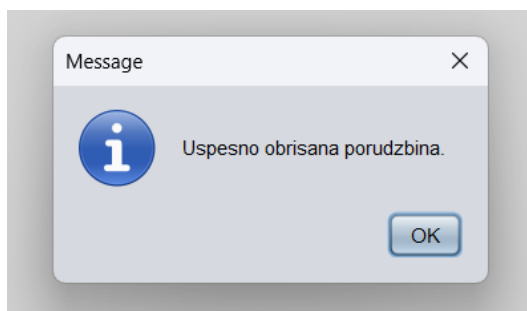
Слика 97: Детаљи поруџбине

10. **Администратор** позива **систем** да обрише **поруџбину**. (АПСО)

Опис акције: Администратор кликом на дугме „Obrisi porudzbinu“ позива системску операцију **obrisiPorudzbinu(Porudzbina)**.

11. **Систем** брише **поруџбину**. (СО)

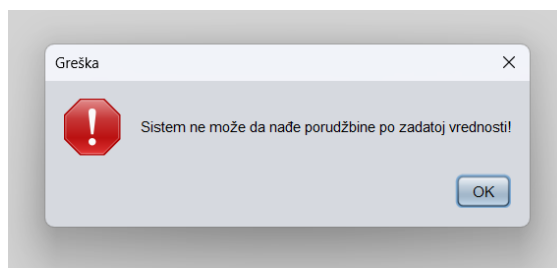
12. **Систем** приказује **администратору** поруку: „Систем је обрисао **поруџбину**“. (ИА)



Слика 98: Порука о успешном брисању поруџбине

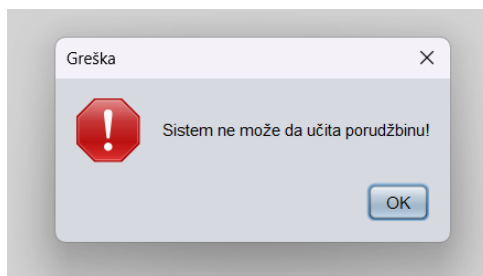
Алтернативна сценарија:

5.1. Уколико **систем** не може да нађе **поруџбине**, он приказује **администратору** поруку: „Систем не може да нађе **поруџбине** по задатој вредности“. Прекида се извршење сценарија. (ИА)



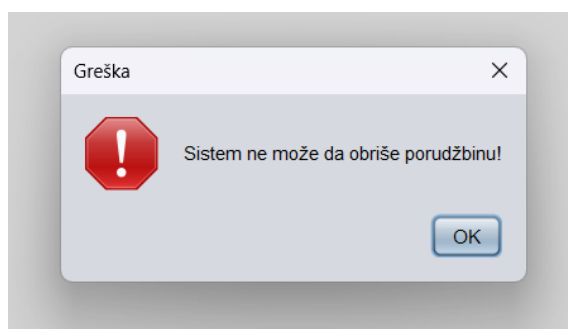
Слика 99: Порука о неуспешној претрази поруџбине

9.1 Уколико **систем** не може да прикаже податке о **поруџбини**, он приказује **администратору** поруку: „**Систем** не може да учита **поруџбину**.“. Прекида се извршење сценарија. (ИА)



Слика 100: Порука о неуспешном учитавању поруџбине

12.1 Уколико **систем** не може да обрише **поруџбину** он приказује **администратору** поруку: „**Систем** не може да обрише **поруџбину**.“. (ИА)



Слика 101: Порука о неуспешном брисању поруџбине

4.3.1.2 Пројектовање контролера корисничког интерфејса

Контролер корисничког интерфејса је одговоран да:

1. прихвати податке које шаље екранска форма
2. конвертује податке (који се налазе у графичким елементима) у објекат који представља улазни аргумент СО која ће бити позвана
3. шаље захтев за извршење системске операције до апликационог сервера (софтверског система)
4. прихвата објекат (излаз) софтверског система настао као резултат извршења системске операције
5. конвертује објекат у податке графичких елемената. [1]

4.3.2 Пројектовање апликационе логике

Апликациона логика описује структуру и понашање система. Апликациони сервер се састоји из:

1. **Контролера апликационе логике** – треба да подигне серверски сокет који ће да ослушкује мрежу. Служи за комуникацију са клијентом и одговоран је да прихвати захтев за извршење системске операције од клијента и проследи га до пословне логике која је одговорна за извршење СО
2. **Пословна логика** – описана је структуром (доменске класе) и понашањем (системске операције)
3. **Брокер базе података** – служи за комуникацију између пословне логике и базе података [1]

4.3.2.1 Контрорлер апликационе логике

Део за комуникацију подиже серверски сокет који ослушкује мрежу. Када клијентски сокет успостави конекцију са серверским сокетом, тада сервер генерише нит која ће успоставити двосмерну комуникацију са клијентом.

Софтверски систем реализован је као клијент-сервер апликација. На серверској страни је нит *ThreadServer* која садржи објекат класе *ServerSocket*. Нит константно позива методу *accept* која чека да се покрене клијентска апликација која, кад се то деси, ће покушати да се повеже на сервер. Слање и примање података од клијента се остварује преко сокета, метода *accept* креира објекат класе *Socket*. Клијент шаље захтев за извршење неке од СО до одговарајуће нити (коју смо назвали *ThreadClient*), која је повезана са тим клијентом. *ThreadClient* прима захтев и даље га преусмерава до класа које су одговорне за извршење СО. Након извршења СО резултат се враћа до апликационе логике, односно до класе *ThreadClient* на серверској страни која тај резултат шаље назад до клијента путем сокета.

4.3.2.2 Пословна логика

4.3.2.2.1 Пројектовање понашања софтверског система (системске операције)

За сваки од претходно дефинисаних уговора правимо системску операцију, што заправо представља пројектовање понашања. Класа *AbstractSO* која представља апстрактну класу која садржи методу *templateExecute(izvrsiOpstuOpreaciju)*, која представља шаблон извршавања сваке операције над базом података, а као параметар прима објекат класе *AbstractDomainObject*. У тој методи се позивају методе *validate* и *execute*, које су апстрактне и које ће свака класа системске операције имплементирати. Након тога се позива метода *commit* која узима објекат *Connection*, класе *DBBroker* и позива њену методу *commit*. Провера предуслова се извршава на клијентској страни уколико постоји, а постуслови се читавају у оквиру *Response* објекта који шаље сервер клијенту и на основу кога клијент закључује да ли је операција успешно извршена на серверској страни или је дошло до грешке.

За сваку системску операцију треба направити концептуална решења која су директно повезана са логиком проблема.

За сваки уговор пројектује се концептуално решење.

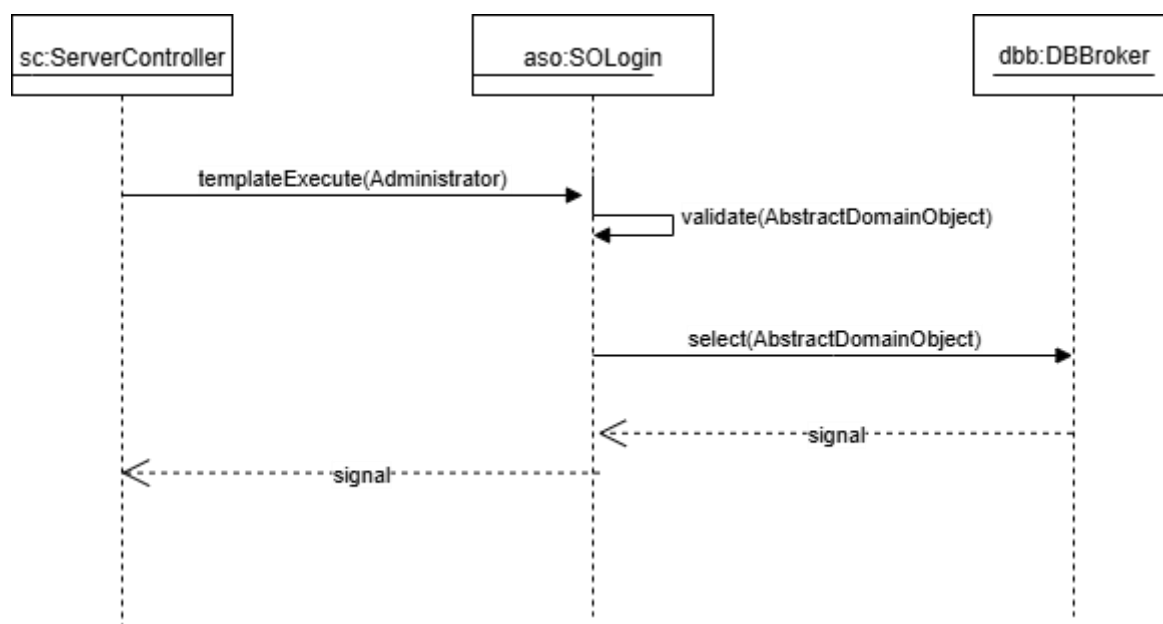
Уговор UG1: Login

Операција: **login(Administrator):** signal;

Веза са СК: СК1

Предуслови: /

Постуслови: Администратор је улоган.



Слика 102: Дијаграм секвенци за уговор login

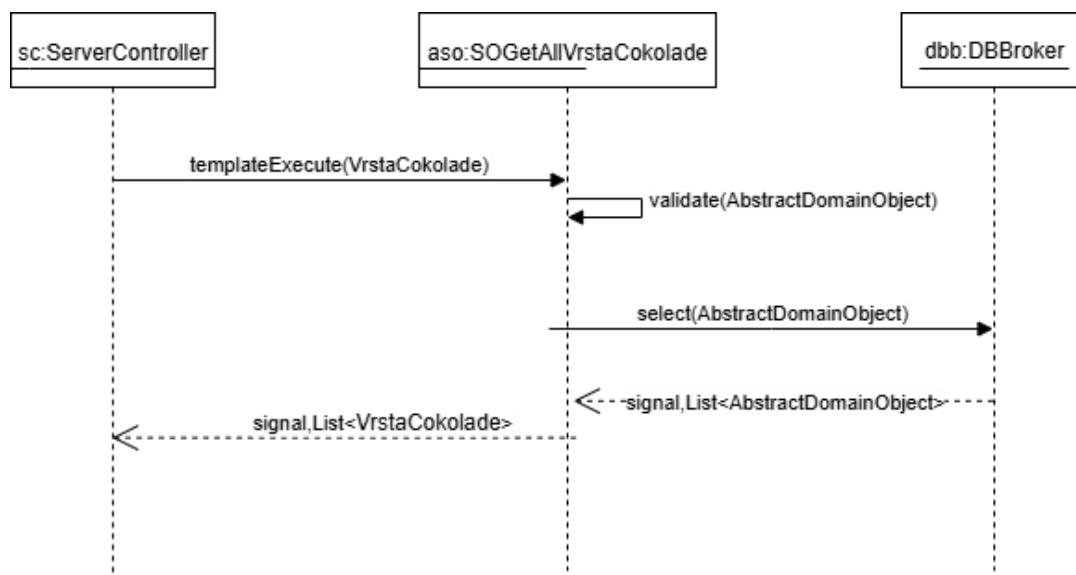
Уговор UG2: ucitajListuVrstaCokolade

Операција: ucitajListuVrstaCokolade(List<VrstaCokolade>): signal;

Веза са СК: СК2

Предуслови: /

Постуслови: Учитана је листа врсти чоколада.



Слика 103: Дијаграм секвенци за уговор ucitajListuVrstaCokolade

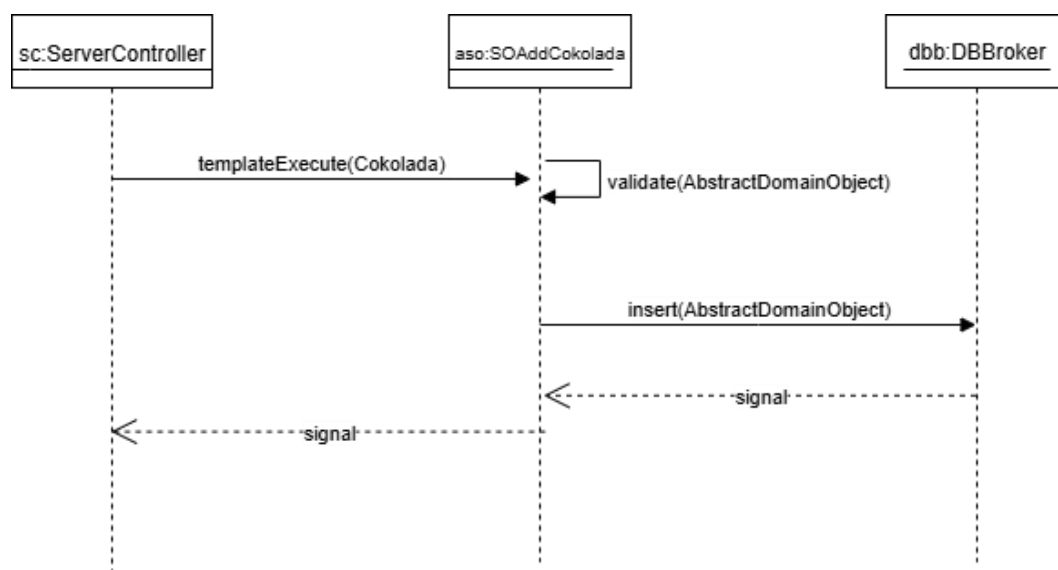
Уговор UG3: kreirajCokoladu

Операција: kreirajCokoladu(Cokolada): signal;

Веза са СК: СК2

Предуслови: Вредносна и структурна ограничења над објектом Чоколада морају бити задовољена.

Постуслови: Креирана је нова чоколада.



Слика 104: Дијаграм секвенци за уговор kreirajCokoladu

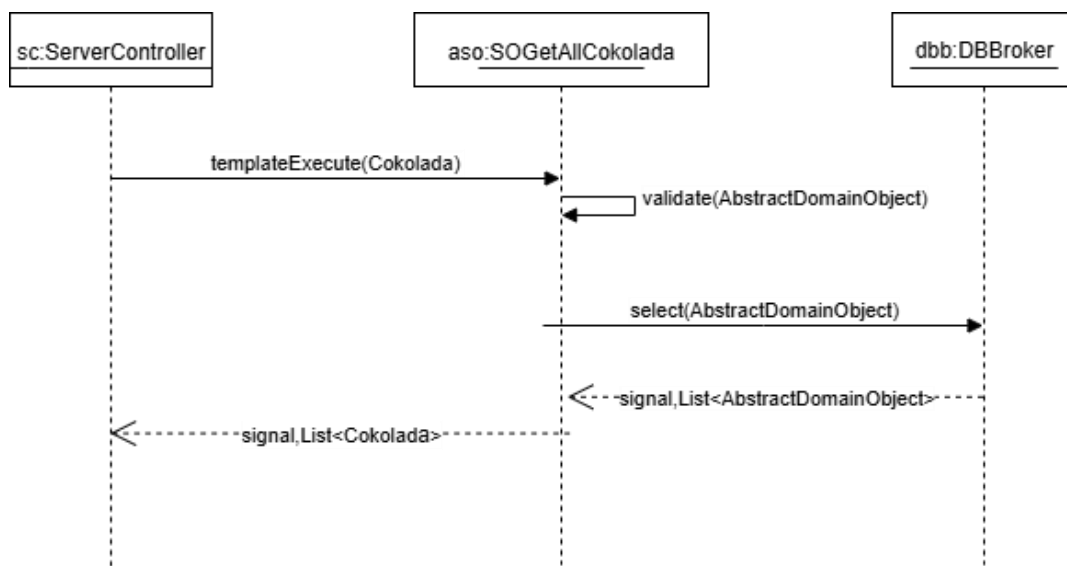
Уговор UG4: *ucitajListuCokolada*

Операција: **ucitajListuCokolada(List< Cokolada >): signal;**

Веза са СК: СК3, СК4, СК5, СК6, СК8

Предуслови: /

Постуслови: Учитана је листа чоколада.



Слика 105: Дијаграм секвенци за уговор *ucitajListuCokolada*

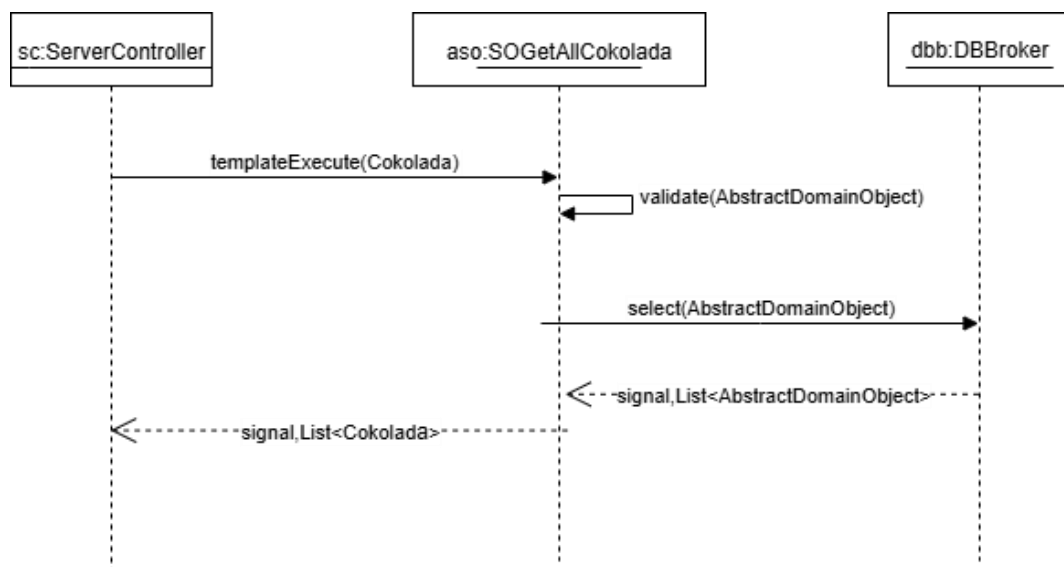
Уговор UG5: *nadjiCokolade*

Операција: **nadjiCokolade(ZadataVrednost, List<Cokolada>): signal;**

Веза са СК: СК3, СК4, СК5

Предуслови: /

Постуслови: Пронађене су тражене чоколаде.



Слика 106: Дијаграм секвенци за уговор *nadjiCokolade*

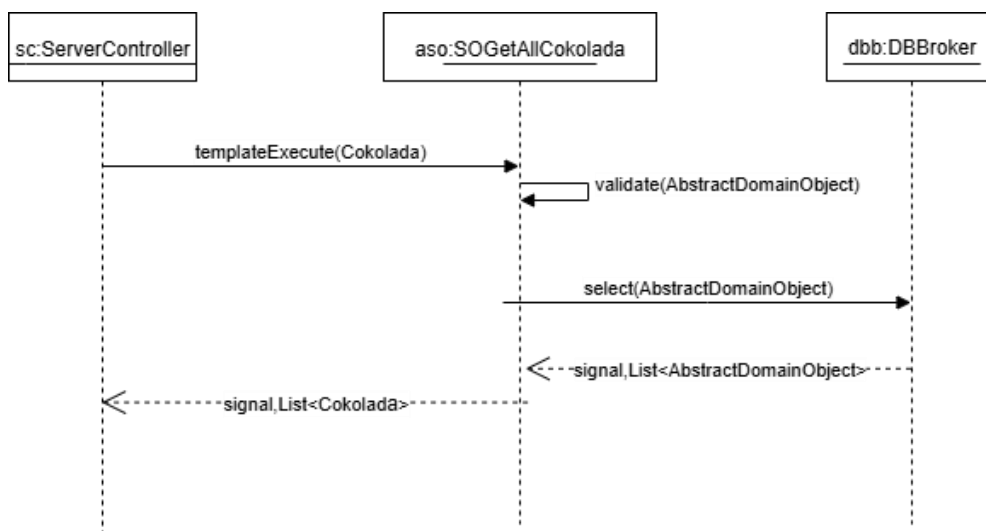
Уговор UG6: *ucitajCokoladu*

Операција: *ucitajCokoladu(Cokolada)*: signal;

Веза са СК: СК3, СК4, СК5

Предуслови: /

Постуслови: *Учитана је тражена чоколада.*



Слика 107: Дијаграм секвенци за уговор *ucitajCokoladu*

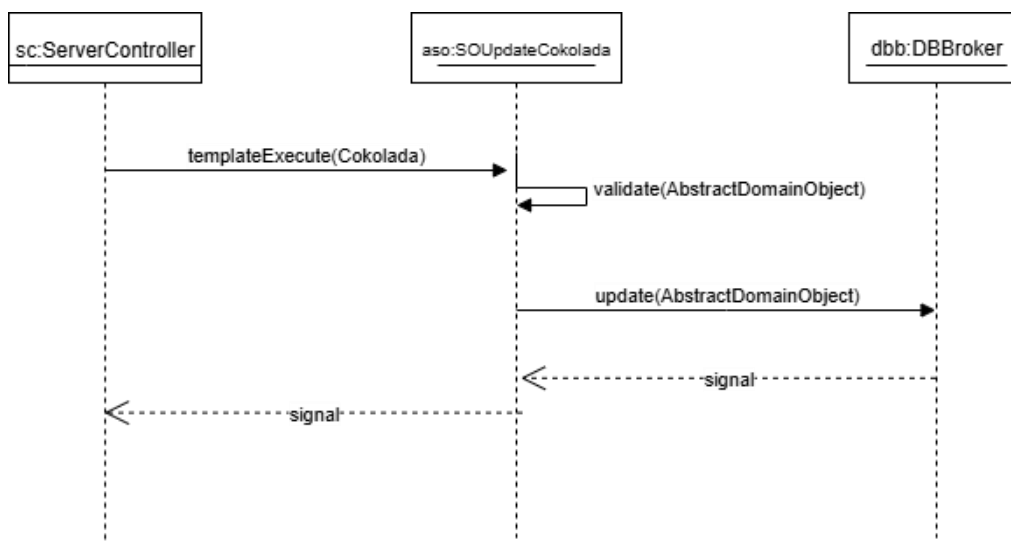
Уговор UG7: *izmeniCokoladu*

Операција: *izmeniCokoladu(Cokolada)*: signal;

Веза са СК: СК4

Предуслови: *Вредносна и структурна ограничења над објектом Чоколада морају бити задовољена.*

Постуслови: *Подаци о чоколади су измењени.*



Слика 108: Дијаграм секвенци за уговор *izmeniCokoladu*

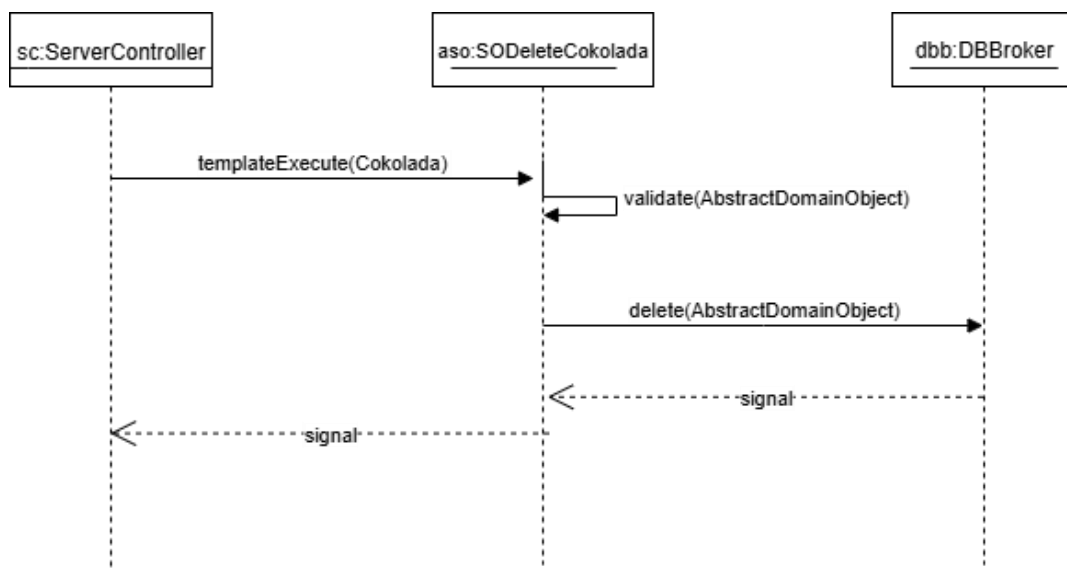
Уговор UG8: *obrisiCokoladu*

Операција: **obrisiCokoladu(Cokolada)**: signal;

Веза са СК: СК5

Предуслови: Структурна ограничења над објектом Чоколада морају бити задовољена.

Постуслови: Чоколада је обрисана.



Слика 109: Дијаграм секвенци за уговор *obrisiCokoladu*

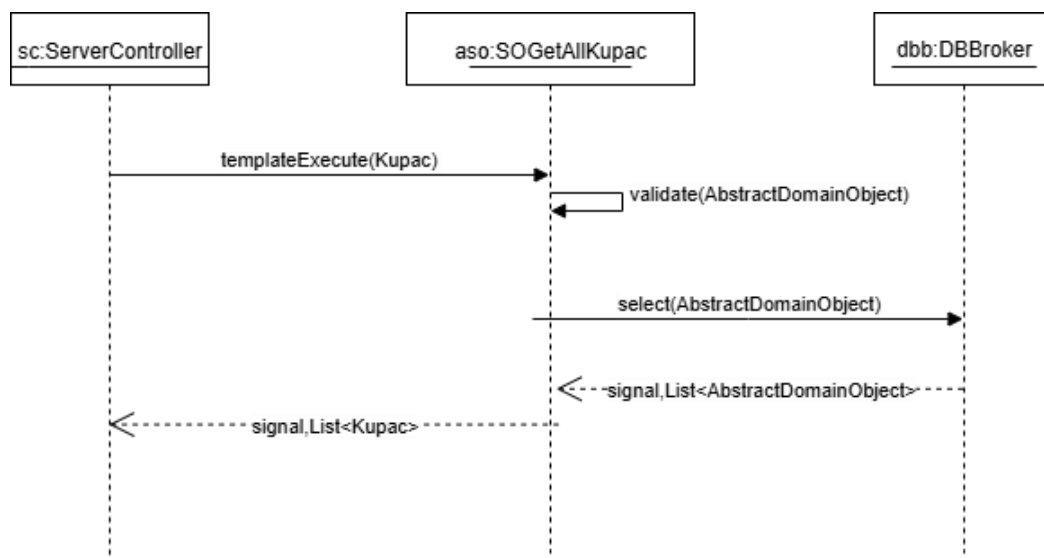
Уговор UG9: *ucitajListuKupaca*

Операција: **ucitajListuKupaca(List<Kupac>)**: signal;

Веза са СК: СК6

Предуслови: /

Постуслови: Листа купаца је учитана.



Слика 110: Дијаграм секвенци за уговор *ucitajListuKupaca*

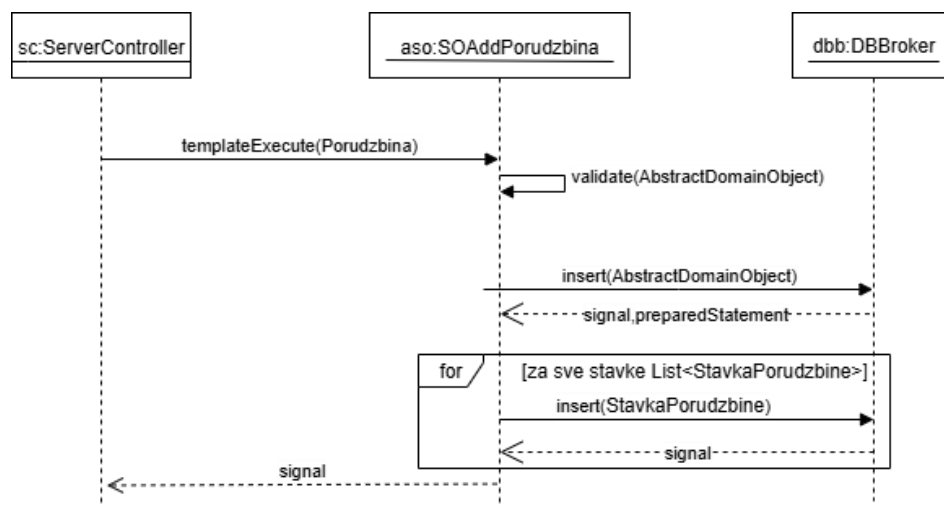
Уговор UG10: kreirajPorudzbину

Операција: kreirajPorudzbину(Porudzbina): signal;

Веза са СК: СК6

Предуслови: Вредносна и структурна ограничења над објектима Поруџбина и СтавкаПоруџбине морају бити задовољена.

Постуслови: Унети подаци о поруџбини су запамћени.



Слика 111: Дијаграм секвенци за уговор kreirajPorudzbину

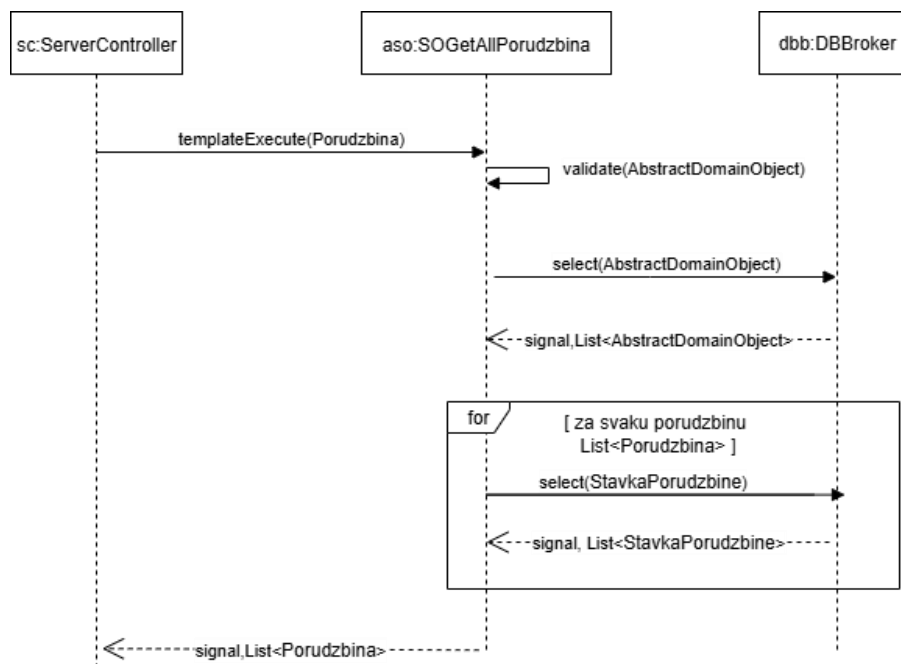
Уговор UG11: ucitajListuPorudzbina

Операција: ucitajListuPorudzbina (List<Porudzbina>): signal;

Веза са СК: СК7, СК8, СК9

Предуслови: /

Постуслови: Учитана је листа поруџбина.



Слика 112: Дијаграм секвенци за уговор ucitajListuPorudzbina

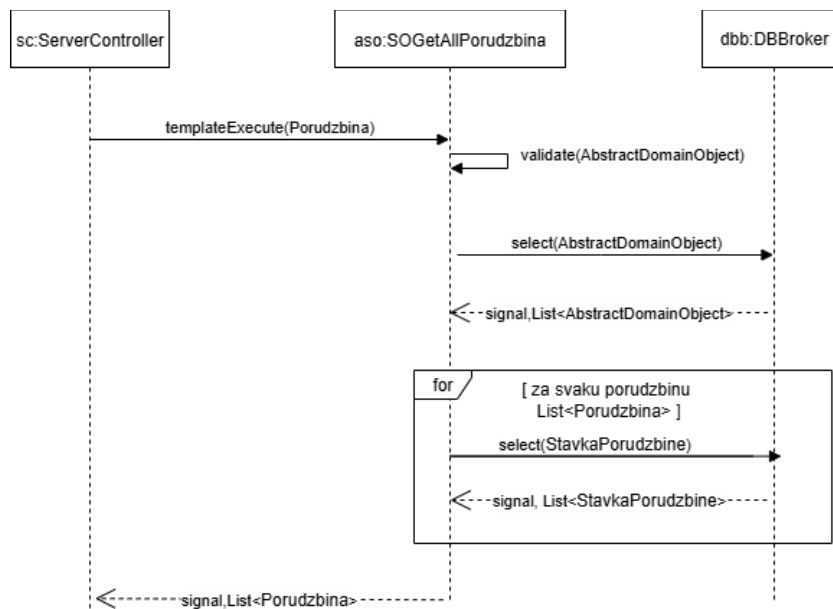
Уговор UG12: *nadjiPorudzbine*

Операција: **nadjiPorudzbine (ZadataVrednost, List<Porudzbina>): signal;**

Веза са СК: СК7, СК8, СК9

Предуслови: /

Постуслови: Пронађене су тражене поруџбине.



Слика 113: Дијаграм секвенци за уговор *nadjiPorudzbine*

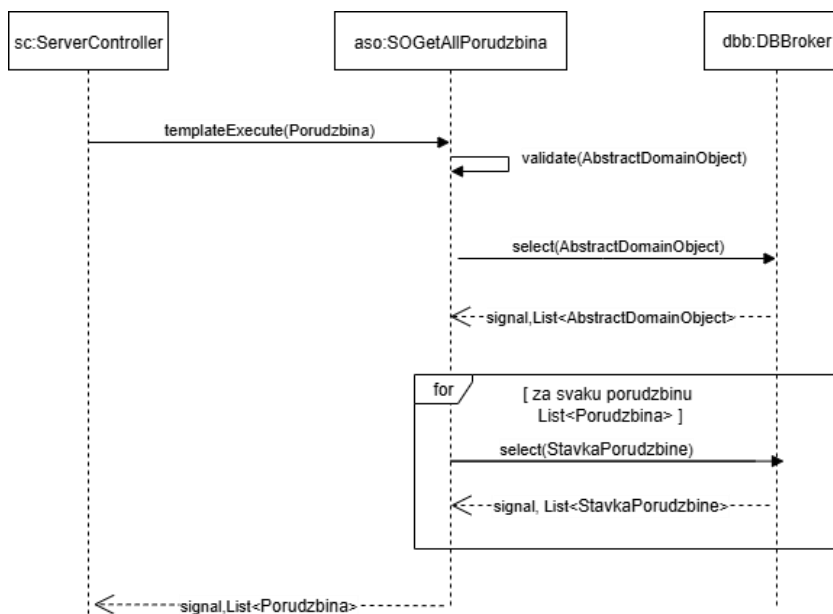
Уговор UG13: *ucitajPorudzbinu*

Операција: **ucitajPorudzbinu(Porudzbina): signal;**

Веза са СК: СК7, СК8, СК9

Предуслови: /

Постуслови: Учитана је тражена поруџбина.



Слика 114: Дијаграм секвенци за уговор *ucitajPorudzbinu*

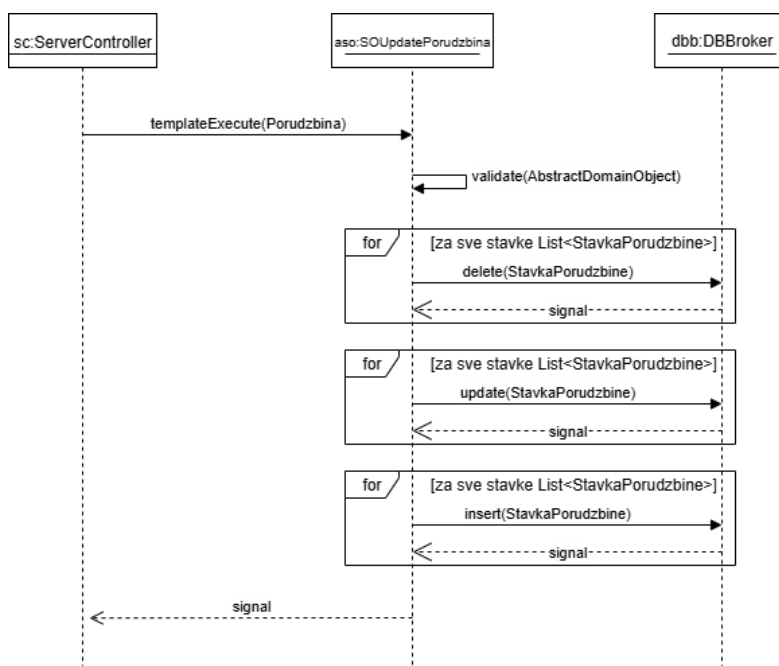
Уговор UG14: izmeniPorudzbinu

Операција: **izmeniPorudzbinu(Porudzbina): signal;**

Веза са СК: СК8

Предуслови: Вредносна и структурна ограничења над објектима Поруџбина и СтавкаПоруџбине морају бити задовољена.

Постуслови: Поруџбина је измењена.



Слика 115: Дијаграм секвенци за уговор *izmeniPorudzbinu*

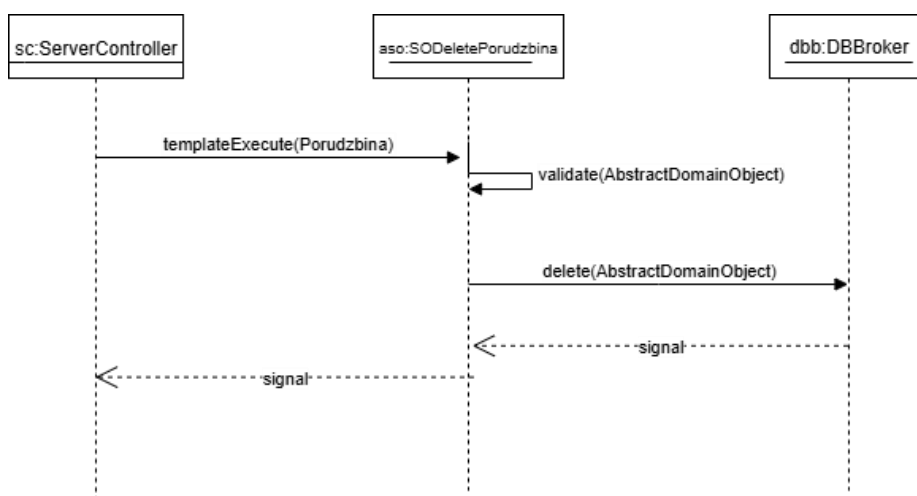
Уговор UG15: obrisiPorudzbinu

Операција: **obrisiPorudzbinu (Porudzbina): signal;**

Веза са СК: СК9

Предуслови: Структурна ограничења над објектом Поруџбина морају бити задовољена.

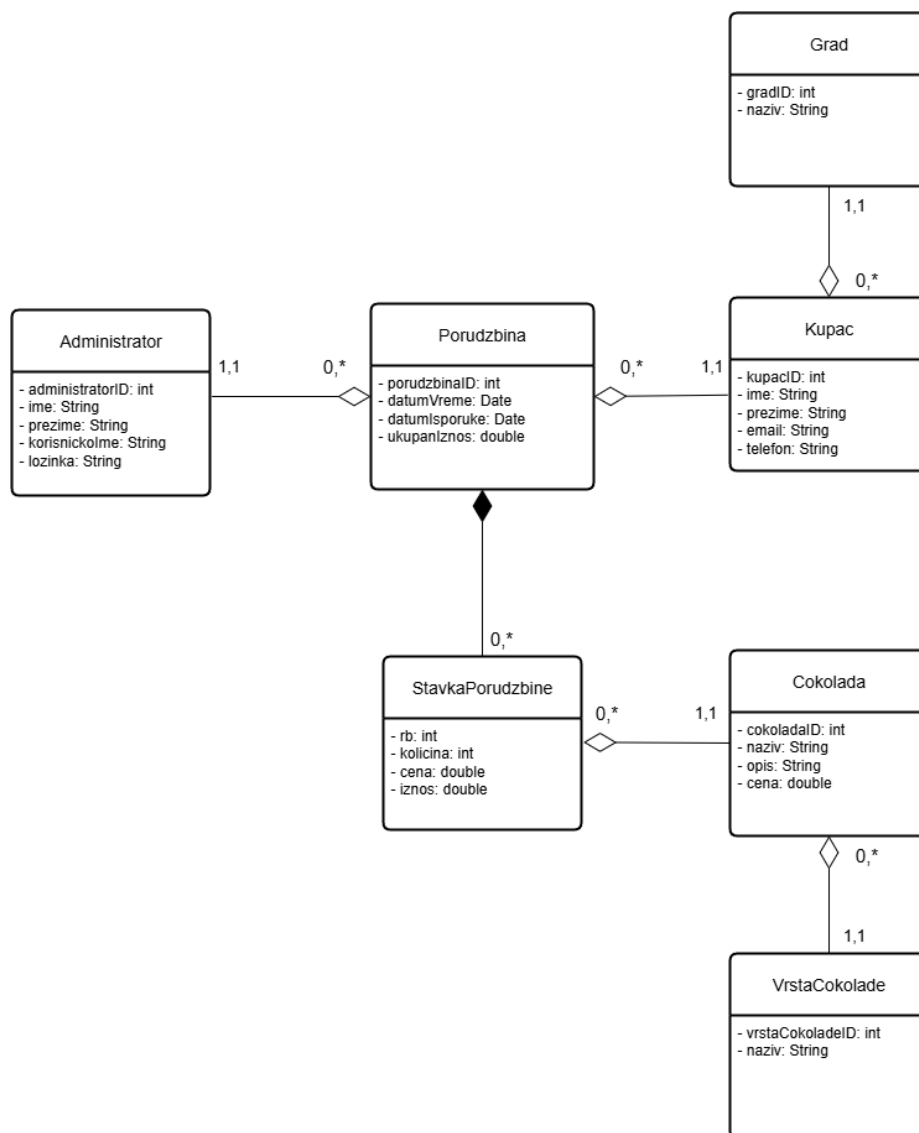
Постуслови: Поруџбина је обрисана.



Слика 116: Дијаграм секвенци за уговор *obrisiPorudzbinu*

4.3.2.2 Пројектовање структуре софтверског система (доменске класе)

На основу концептуалних класа креирају се софтверске класе.



Слика 117: Концептуални дијаграм класа

Идентификоване су следеће класе:

- Администратор

```
public class Administrator extends AbstractDomainObject {  
  
    private int administratorID;  
    private String ime;  
    private String prezime;  
    private String korisnickoIme;  
    private String lozinka;  
  
    public Administrator() {  
    }  
  
    public Administrator(int administratorID, String ime, String prezime, String korisnickoIme, String lozinka) {  
        this.administratorID = administratorID;  
        this.ime = ime;  
        this.prezime = prezime;  
        this.korisnickoIme = korisnickoIme;  
        this.lozinka = lozinka;  
    }  
}
```

Слика 118: Класа Администратор

- Врста чоколаде

```
public class VrstaCokolade extends AbstractDomainObject {  
  
    private int vrstaCokoladeID;  
    private String naziv;  
  
    @Override  
    public String toString() {  
        return naziv;  
    }  
  
    public VrstaCokolade(int vrstaCokoladeID, String naziv) {  
        this.vrstaCokoladeID = vrstaCokoladeID;  
        this.naziv = naziv;  
    }  
  
    public VrstaCokolade() {  
    }  
}
```

Слика 119: Класа Врста чоколаде

- Чоколада

```
public class Cokolada extends AbstractDomainObject {  
  
    private int cokoladaID;  
    private String naziv;  
    private String opis;  
    private double cena;  
    private VrstaCokolade vrstaCokolade;  
  
    @Override  
    public String toString() {  
        return naziv + " (Cena: " + cena + "din)";  
    }  
  
    public Cokolada(int cokoladaID, String naziv, String opis, double cena, VrstaCokolade vrstaCokolade) {  
        this.cokoladaID = cokoladaID;  
        this.naziv = naziv;  
        this.opis = opis;  
        this.cena = cena;  
        this.vrstaCokolade = vrstaCokolade;  
    }  
}
```

Слика 120: Класа Чоколада

- Град

```
public class Grad extends AbstractDomainObject {  
  
    private int gradID;  
    private String naziv;  
  
    @Override  
    public String toString() {  
        return naziv;  
    }  
  
    public Grad(int gradID, String naziv) {  
        this.gradID = gradID;  
        this.naziv = naziv;  
    }  
  
    public Grad() {  
    }  
}
```

Слика 121: Класа Град

- Купац

```
public class Kupac extends AbstractDomainObject {  
  
    private int kupacID;  
    private String ime;  
    private String prezime;  
    private String email;  
    private String telefon;  
    private Grad grad;  
  
    @Override  
    public String toString() {  
        return ime + " " + prezime;  
    }  
  
    public Kupac(int kupacID, String ime, String prezime, String email, String telefon, Grad grad) {  
        this.kupacID = kupacID;  
        this.ime = ime;  
        this.prezime = prezime;  
        this.email = email;  
        this.telefon = telefon;  
        this.grad = grad;  
    }  
  
    public Kupac() {  
    }  
}
```

Слика 122: Класа Купац

- Поруџбина

```
public class Porudzbina extends AbstractDomainObject {  
  
    private int porudzbinaID;  
    private Date datumVreme;  
    private Date datumIsporuke;  
    private double ukupanIznos;  
    private Kupac kupac;  
    private Administrator administrator;  
    private ArrayList<StavkaPorudzbine> stavkePorudzbine;  
  
    public Porudzbina(int porudzbinaID, Date datumVreme, Date datumIsporuke, double ukupanIznos, Kupac kupac,  
        Administrator administrator, ArrayList<StavkaPorudzbine> stavkePorudzbine) {  
        this.porudzbinaID = porudzbinaID;  
        this.datumVreme = datumVreme;  
        this.datumIsporuke = datumIsporuke;  
        this.ukupanIznos = ukupanIznos;  
        this.kupac = kupac;  
        this.administrator = administrator;  
        this.stavkePorudzbine = stavkePorudzbine;  
    }  
  
    public Porudzbina() {  
    }  
}
```

Слика 123: Класа Поруџбина

- Ставка поруџбине

```
public class StavkaPorudzbine extends AbstractDomainObject {

    private Porudzbina porudzbina;
    private int rb;
    private int kolicina;
    private double cena;
    private double iznos;
    private Cokolada cokolada;

    public StavkaPorudzbine(Porudzbina porudzbina, int rb, int kolicina, double cena, double iznos, Cokolada cokolada) {
        this.porudzbina = porudzbina;
        this.rb = rb;
        this.kolicina = kolicina;
        this.cena = cena;
        this.iznos = iznos;
        this.cokolada = cokolada;
    }

    public StavkaPorudzbine() {
    }
}
```

Слика 124: Класа Ставка поруџбине

Поред њих додате су и следеће класе:

- AbstractDomainObject - апстрактна класа које све доменске класе наслеђују

```
public abstract class AbstractDomainObject implements Serializable {

    public abstract String nazivTabele();
    public abstract String alias();
    public abstract String join();
    public abstract ArrayList<AbstractDomainObject> vratiListu(ResultSet rs) throws SQLException;
    public abstract String koloneZaInsert();
    public abstract String vrednostiZaInsert();
    public abstract String vrednostiZaUpdate();
    public abstract String uslov();
    public abstract String uslovZaSelect();
}
```

Слика 125: Класа AbstractDomainObject

- Operation – интерфејс који садржи све операције које се шаљу од клијента серверу

```
public interface Operation {

    public static final int LOGIN = 0;
    public static final int LOGOUT = 1;

    public static final int ADD_KUPAC = 2;
    public static final int DELETE_KUPAC = 3;
    public static final int UPDATE_KUPAC = 4;
    public static final int GET_ALL_KUPAC = 5;

    public static final int ADD_COKOLADA = 6;
    public static final int DELETE_COKOLADA = 7;
    public static final int UPDATE_COKOLADA = 8;
    public static final int GET_ALL_COKOLADA = 9;

    public static final int ADD_PORUDZBINA = 10;
    public static final int DELETE_PORUDZBINA = 11;
    public static final int UPDATE_PORUDZBINA = 12;
    public static final int GET_ALL_PORUDZBINA = 13;

    public static final int GET_ALL_VRSTA_COKOLADE = 14;

    public static final int GET_ALL_GRAD = 15;
}
```

Слика 126: Интерфејс Operation

- Request – служи за слање објекта од клијента ка серверу. Садржи један *Object* атрибут који представља објекат над којим треба извршити захтевану операцију и један *int* атрибут који представља операцију која треба да се изврши.

```
public class Request implements Serializable {

    private int operation;
    private Object data;

    public Request() {
    }

    public Request(int operation, Object data) {
        this.operation = operation;
        this.data = data;
    }

    public Object getData() {
        return data;
    }

    public int getOperation() {
        return operation;
    }

    public void setData(Object data) {
        this.data = data;
    }

    public void setOperation(int operation) {
        this.operation = operation;
    }

}
```

Слика 127: Класа Request

- Response – служи за слање објекта од сервера ка клијенту. Садржи један *Object* атрибут који представља резултат извршене операције, један *Exception* атрибут који представља изузетак који се можда десио.

```
public class Response implements Serializable {

    private Object data;
    private Exception exc;
    private ResponseStatus responseStatus;

    public Response() {
    }

    public Response(Object data, Exception exc, ResponseStatus responseStatus) {
        this.data = data;
        this.exc = exc;
        this.responseStatus = responseStatus;
    }

    public Object getData() {
        return data;
    }

    public void setData(Object data) {
        this.data = data;
    }

    public Exception getException() {
        return exc;
    }

    public void setException(Exception exc) {
        this.exc = exc;
    }

    public ResponseStatus getResponseStatus() {
        return responseStatus;
    }

    public void setResponseStatus(ResponseStatus responseStatus) {
        this.responseStatus = responseStatus;
    }

}
```

Слика 128: Класа Response

4.3.2.3 Пројектовање складишта података

На основу релационог модела и ограничења пројектоване су табеле базе података које користи наш софтверски систем:

Table Name: administrator Engine: InnoDB Database: cokolada Character Set: utf8mb4 Collation: utf8mb4_general_ci

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
AdministratorID	bigint	10		☒	☒	☒	☒	☐	☐		(none)		
Ime	varchar	50		☐	☒	☐	☐	☐	☐		(none)		
Prezime	varchar	50		☐	☒	☐	☐	☐	☐		(none)		
KorisnickoIme	varchar	30		☐	☒	☐	☐	☐	☐		(none)		
Lozinka	varchar	30		☐	☒	☐	☐	☐	☐		(none)		

Слика 129: Табела Администратор

Table Name: vrstacokolade Engine: InnoDB Database: cokolada Character Set: utf8mb4 Collation: utf8mb4_general_ci

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
VrstaCokoladeID	bigint	10		☒	☒	☒	☒	☐	☐		(none)		
Naziv	varchar	50		☐	☒	☐	☐	☐	☐		(none)		

Слика 130: Табела Врста чоколаде

Table Name: cokolada Engine: InnoDB Database: cokolada Character Set: utf8mb4 Collation: utf8mb4_general_ci

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
CokoladeID	bigint	10		☒	☒	☒	☒	☐	☐		(none)		
Naziv	varchar	50		☐	☒	☐	☐	☐	☐		(none)		
Opis	varchar	300		☐	☒	☐	☐	☐	☐		(none)		
Cena	decimal	10,2		☐	☒	☐	☐	☐	☐		(none)		
VrstaCokoladeID	bigint	10		☐	☒	☒	☐	☐	☐		(none)		

Слика 131: Табела Чоколада

Table Name: grad Engine: InnoDB Database: cokolada Character Set: utf8mb4 Collation: utf8mb4_general_ci

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
GradID	bigint	10		☒	☒	☒	☒	☐	☐		(none)		
Naziv	varchar	30		☐	☒	☐	☐	☐	☐		(none)		

Слика 132: Табела Град

Table Name Engine

Database Character Set

Collation

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
☐ KupacID	bigint	10		☒	☒	☒	☒	☐	☐		(none)		
☐ Ime	varchar	30		☐	☒	☐	☐	☐	☐		(none)		
☐ Prezime	varchar	30		☐	☒	☐	☐	☐	☐		(none)		
☐ Email	varchar	50		☐	☒	☐	☐	☐	☐		(none)		
☐ Telefon	varchar	30		☐	☒	☐	☐	☐	☐		(none)		
☐ GradID	bigint	10		☐	☒	☒	☐	☐	☐		(none)		

Слика 133: Табела Купац

Table Name Engine

Database Character Set

Collation

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
☐ PorudzbinaID	bigint	10		☒	☒	☒	☒	☐	☐		(none)		
☐ DatumVreme	datetime			☐	☒	☐	☐	☐	☐		(none)		
☐ DatumIspruke	date			☐	☒	☐	☐	☐	☐		(none)		
☐ Ukupaniznos	decimal	10,2		☐	☒	☐	☐	☐	☐		(none)		
☐ KupacID	bigint	10		☐	☒	☒	☐	☐	☐		(none)		
☐ AdministratorID	bigint	10		☐	☒	☐	☐	☐	☐		(none)		

Слика 134: Табела Поруџбина

Table Name Engine

Database Character Set

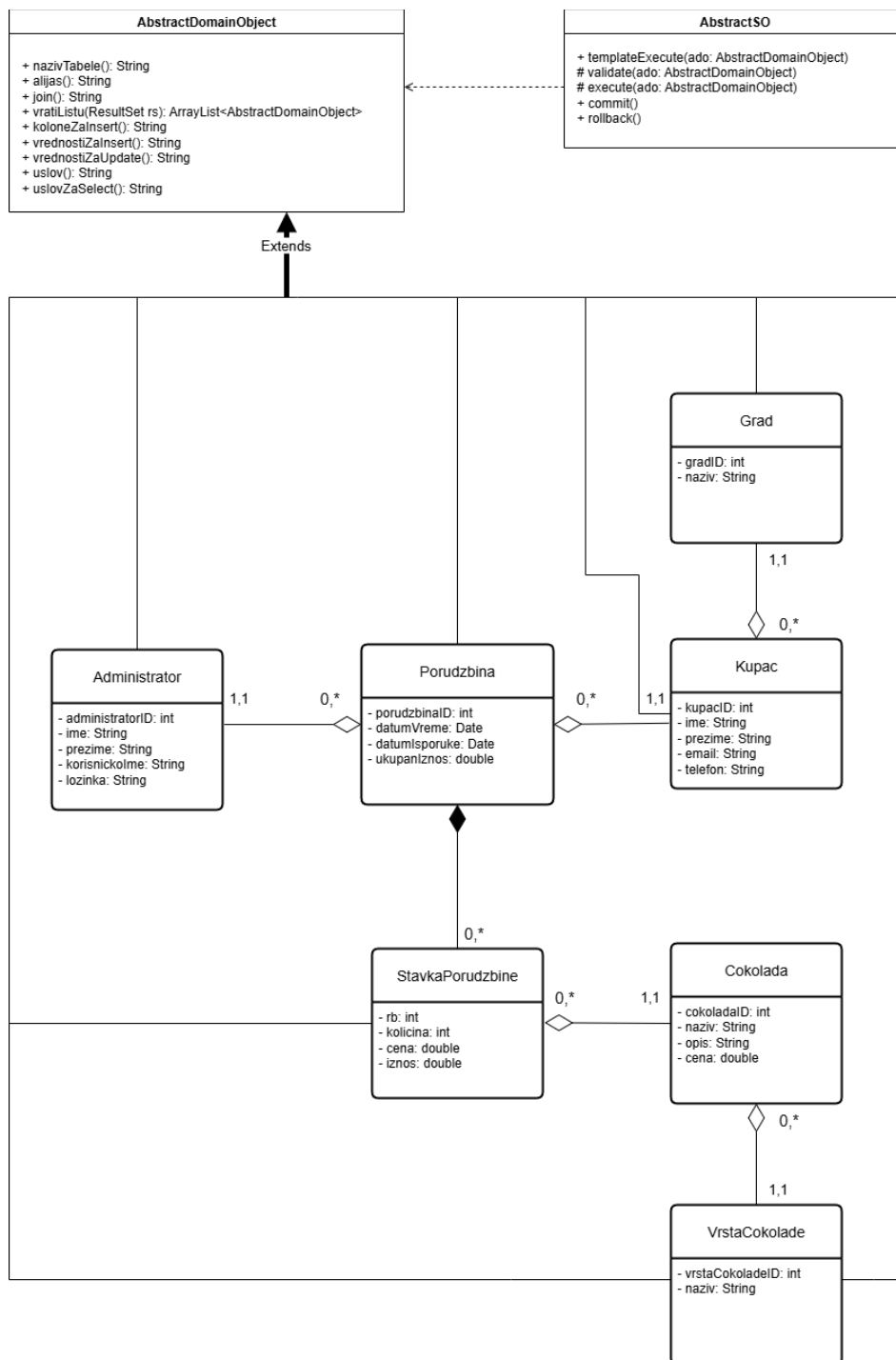
Collation

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment	Virtuality	Expression	Check Constraint
☐ PorudzbinaID	bigint	10		☒	☒	☒	☐	☐	☐		(none)		
☐ Rb	int	7		☒	☒	☐	☐	☐	☐		(none)		
☐ Kolicina	int	7		☐	☒	☐	☐	☐	☐		(none)		
☐ Cena	decimal	10,2		☐	☒	☐	☐	☐	☐		(none)		
☐ Inos	decimal	10,2		☐	☒	☐	☐	☐	☐		(none)		
☐ CokoladaID	bigint	10		☐	☒	☒	☐	☐	☐		(none)		

Слика 135: Табела Ставка поруџбине

За комуникацију са базом података правимо генеричку класу *AbstractSO* која има методе за валидацију и извршавања трансакције, које су апстрактне и које ће имплементирати свака класа системске операције која ће се извршавати, као и методе за потврђивање и поништавање трансакције. Она се служи класом *DBBroker* која је имплементирана помоћу *Singleton* патерна и која параметре за повезивање на базу података чита из текстуалне датотеке која садржи све потребне параметре и помоћу ње наша генеричка класа добија конекцију на базу података.

Као резултат пројектовања класе *AbstractSO* и доменских објеката добијамо следећи дијаграм класа:

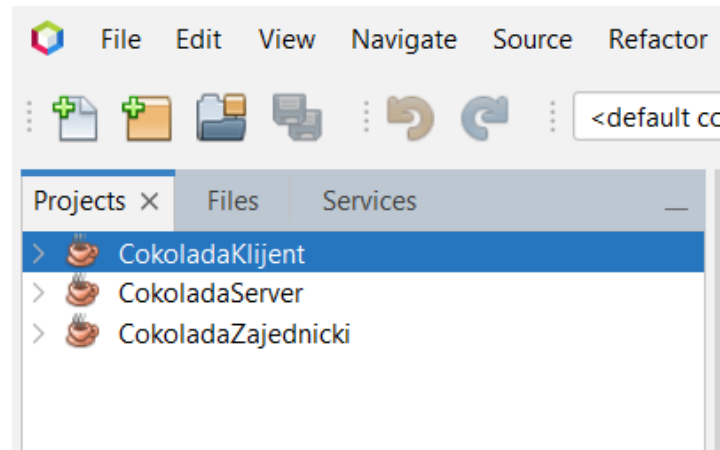


Слика 136: Дијаграм класа

4.4. Фаза имплементације

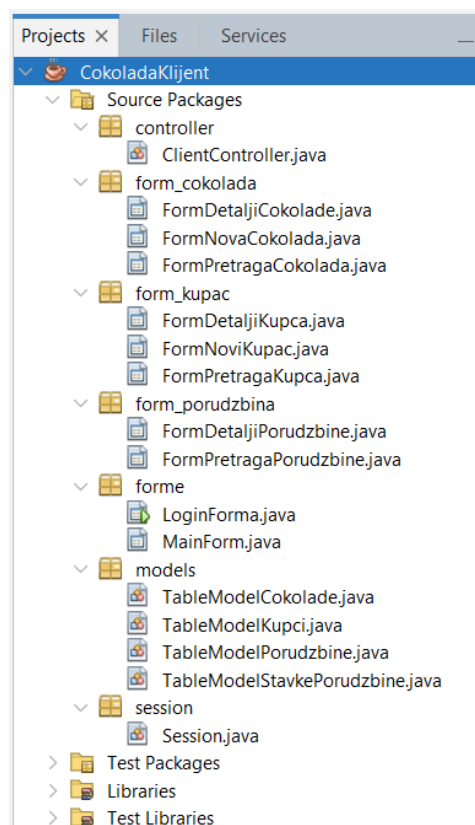
Софтверски систем је развијан у програмском језику Java 20, развојно окружење ApacheNetBeans IDE 18. Као систем за управљање базом података коришћен је MySQL 13.2.1.

Организација пројеката је приказана на следећој слици.



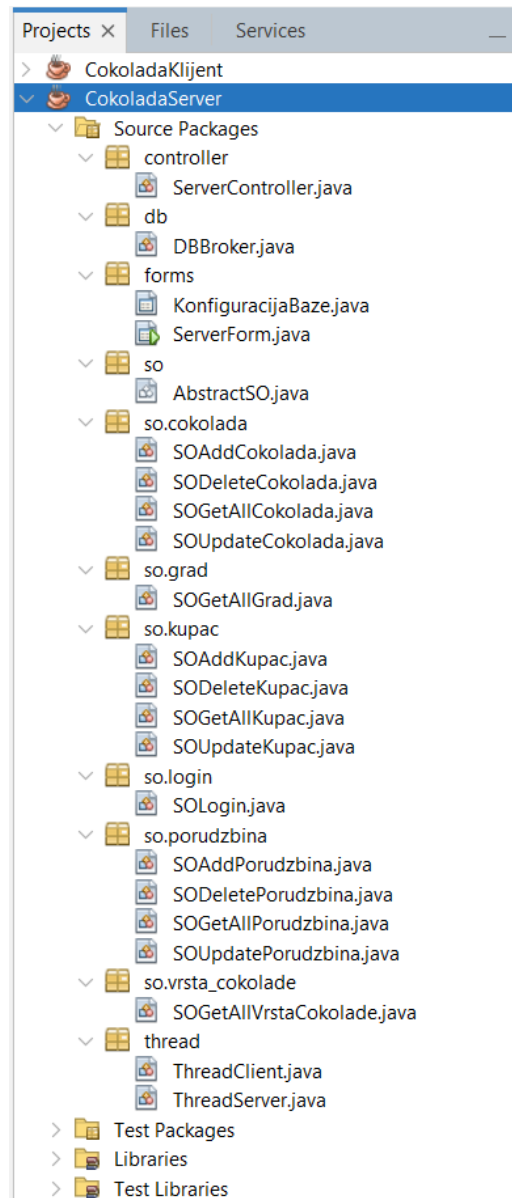
Слика 138: Организација пројекта

Пројекат *CokoladaKlijent* садржи форме на којима запослени ради и сокет који служи за комуникацију са сервером.



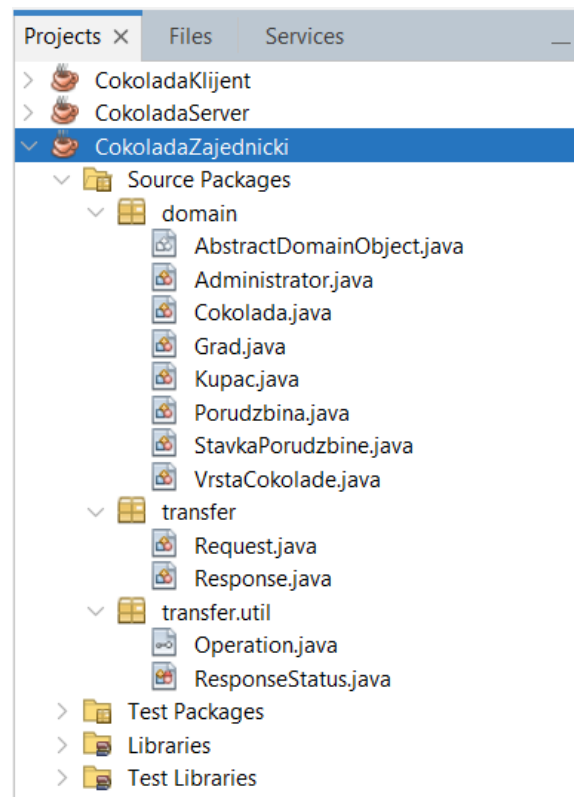
Слика 139: Клијентски део

Пројекат *CokoladaServer* садржи нити за покретање комуникације са клијентом, контролера, генеричку класу за базу података и генерисање конекције на базу података и серверску форму неопходну за покретање сервера.



Слика 140: Серверски део

Пројекат *CokoladaZajednicki* садржи заједничке класе које користе клијент и сервер пројекти. То су доменске класе, класе *Request* и *Response*, интерфејс *Operation* и енум *ResponseStatus*.



Слика 141: Заједнички део

4.5 Тестирање

Тестирање представља једну од најважнијих фаза процеса развоја софтвера, јер омогућава откривање и исправљање грешака пре пуштања система у рад. У оквиру овог пројекта примењено је јединично тестирање (unit testing), са циљем да се провери исправност појединачних системских операција и њихово правилно понашање у различитим сценаријима. Фокус је стављен на тестирање системских операција на страни сервера, будући да су оне одговорне за имплементацију пословне логике, очување интегритета података и валидацију улазних информација.

4.5.1 JUnit оквир за јединично тестирање

JUnit представља најкоришћенији оквир за јединично тестирање у Java окружењу. Омогућава аутоматизацију тестова, једноставан начин за креирање тестних случајева и могућност брзог покретања великог броја тестова без потребе за ручном провером резултата. У овом пројекту коришћена је верзија **JUnit 4.13.2**, која се одликује употребом анотација као што су:

- **@Test** — означава да метода представља тестни случај.
- **@Before** — метода која се извршава пре сваког теста и служи за иницијално подешавање тест окружења.
- **@After** — метода која се извршава након сваког теста и користи се за чишћење ресурса.
- **@Test(expected = Exception.class)** — означава да се у оквиру теста очекује бацање изузетка, што је корисно при тестирању валидирања података.

Једна од предности JUnit-а јесте могућност потпуне аутоматизације тестирања. Када се тест покрене, JUnit сам пријављује да ли је тест успешно извршен или је дошло до грешке, уз детаљно навођење разлога. Ово значајно убрзава процес развоја и спречава уношење неочекиваних грешака приликом измене кода.

Уз JUnit је развијен и механизам за симулацију рада базе података без потребе за стварним коришћењем MySQL сервера током тестирања. Пошто системске операције у пројекту користе шаблон *templateExecute()* који обухвата рад над базом (insert, update, delete), било је неопходно омогућити контролисано окружење у коме ће се ова функционалност тестирати.

Из тог разлога креиране су помоћне класе:

- **FakeConnection** — симулира понашање `java.sql.Connection` објекта, прихвата позиве `commit()`, `rollback()` и креира лажне `Statement/PreparedStatement` објекте.
- **FakePreparedStatement** и **FakeStatement** — симулирају извршавање SQL наредби без стварног приступа бази.
- **setSelectResult()** у `DBBroker` класи — омогућава да се резултати `SELECT` упита унапред дефинишу, тако да системска операција може да се тестира у жељеним сценаријима.

На овај начин остварено је изоловано окружење у коме се може тестирати логика системских операција без зависности од инфраструктуре базе података.

4.5.2 Тестирање системске операције SOAddCokolada

Системска операција **SOAddCokolada** задужена је за додавање нове чоколаде у систем. Операција обухвата два основна корака:

1. **validate()** — провера исправности унетих података (тип објекта, опсег цене, јединственост назива).
2. **execute()** — уписивање нове чоколаде у базу.

Тестови су осмишљени тако да покрију све кључне сценарије:

1) Валидација исправног објекта

Тест проверава да ли метода `validate()` пролази без грешке када се проследи објекат са коректним вредностима. Очекује се да операција не баца изузетак.

2) Валидација погрешног типа

Прослеђује се објекат који није појављивање класе `Cokolada`. Очекивани резултат је бацање изузетка.

3) Валидација неисправне цене

Тестира се да ли систем исправно реагује када је цена чоколаде ван дозвољеног опсега.

4) Валидација дуплираног назива

Симулира се резултат `SELECT` упита који указује на постојање чоколаде истог назива, након чега се позива метода `validate()` и очекује изузетак.

5) Успешно извршавање операције

Са лажним DB окружењем тестира се позив `templateExecute()`, који у себи позива `validate()`, `execute()` и `commit()`. Очекује се да операција буде успешно завршена.

Овакви тестови покривају све гране кода и обезбеђују исправност логике пре додавања чоколаде у систем. Резултати тестирања показали су да се операција понаша у складу са очекивањима у свим сценаријима.

```
22 public class SOAddCokoladaTest {
23
24     private SOAddCokolada so;
25     private Cokolada cokolada;
26
27     private class PogresanObjekat extends AbstractDomainObject {
28         @Override public String nazivTabele() { return ""; }
29         @Override public String alias() { return ""; }
30         @Override public String join() { return ""; }
31         @Override public ArrayList<AbstractDomainObject> vratiListu(java.sql.ResultSet rs) { return null; }
32         @Override public String koloneZaInsert() { return ""; }
33         @Override public String vrednostiZaInsert() { return ""; }
34         @Override public String vrednostiZaUpdate() { return ""; }
35         @Override public String uslov() { return ""; }
36         @Override public String uslovZaSelect() { return ""; }
37     }
38
39     @Before
40     public void setUp() throws Exception {
41         so = new SOAddCokolada();
42
43         DBBroker.getInstance().setConnection(new FakeConnection());
44
45         cokolada = new Cokolada();
46         cokolada.setNaziv(naziv: "Crna 70%");
47         cokolada.setOpis(opis: "Gorka čokolada sa 70% kaka");
48         cokolada.setCena(cena: 300);
49         cokolada.setVrstaCokolade(new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna"));
50
51         DBBroker.getInstance().setSelectResult(new ArrayList<>());
52     }
53
54     @After
55     public void tearDown() {
56         so = null;
57         cokolada = null;
58     }
59
60     @Test
61     public void testExecuteOperationSuccess() {
62         try {
63             so.templateExecute(adso: cokolada);
64         } catch (Exception ex) {
65             fail("Operacija nije smela da baci izuzetak! Poruka: " + ex.getMessage());
66         }
67     }
68 }
```

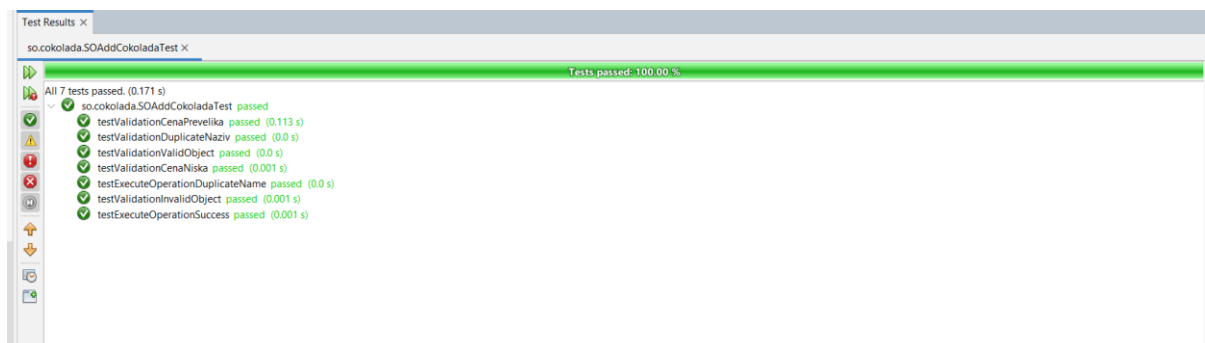
Слика 142: Тестирање системске операције SOAddCokolada

```

68
69
70 @Test(expected = Exception.class)
71 public void testExecuteOperationDuplicateName() throws Exception {
72     ArrayList<Cokolada> lista = new ArrayList<>();
73     lista.add(new Cokolada(cokoladaID: 5, naziv: "Crna 70%", opis: "opis", cena: 300, new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna")));
74     DBBroker.getInstance().setSelectResult(new ArrayList<AbstractDomainObject>>(c: lista));
75     so.templateExecute(ado: cokolada);
76 }
77
78
79
80 @Test
81 public void testValidationValidObject() {
82     try {
83         so.validate(ado: cokolada);
84     } catch (Exception ex) {
85         fail("Validacija nije smela da padne! Poruka: " + ex.getMessage());
86     }
87 }
88
89
90 @Test(expected = Exception.class)
91 public void testValidationInvalidObject() throws Exception {
92     so.validate(new PogresanObjekat());
93 }
94
95
96 @Test(expected = Exception.class)
97 public void testValidationCenaNiska() throws Exception {
98     cokolada.setCena(cena: 20);
99     so.validate(ado: cokolada);
100 }
101
102
103 @Test(expected = Exception.class)
104 public void testValidationCenaPrevelika() throws Exception {
105     cokolada.setCena(cena: 60000);
106     so.validate(ado: cokolada);
107 }
108
109
110 @Test(expected = Exception.class)
111 public void testValidationDuplicateNaziv() throws Exception {
112     ArrayList<Cokolada> lista = new ArrayList<>();
113     lista.add(new Cokolada(cokoladaID: 2, naziv: "Crna 70%", opis: "opis", cena: 300, new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna")));
114     DBBroker.getInstance().setSelectResult(new ArrayList<AbstractDomainObject>>(c: lista));
115     so.validate(ado: cokolada);
116 }

```

Слика 143: Тестирање системске операције SOAddCokolada



Слика 144: Тестирање системске операције SOAddCokolada

4.5.3 Тестирање системске операције SOUpdatePorudzbina

Операција SOUpdatePorudzbina сложенија је од претходне јер обрађује више врста измена на постојећој поруџбини:

- ажурирање података о поруџбини,
- брисање ставки које више не постоје,
- измену ставки које су задржане,
- уметање нових ставки.

1) Тестирање исправних података

Прослеђује се објекат са важећим датумом испоруке и најмање једном ставком. Очекује се да validate() не баци изузетак.

2) Тестирање погрешног типа

Као и код SOAddCokolada, проверава се реакција на прослеђивање објекта погрешне класе.

3) Тестирање неважећег датума испоруке

Ако је датум испоруке пре текућег датума, метод validate() треба да баци изузетак.

4) Тестирање празне листе ставки

Као и у реалном систему, поруџбина мора садржати бар једну ставку. Ако листа ставки буде празна — очекује се изузетак.

5) Тестирање успешног ажурирања поруџбине

Помоћу setSelectResult() симулира се да у бази већ постоје одређене ставке. Поруџбина се затим прослеђује са истим ставкама, а templateExecute() треба да се изврши без грешке. Ово проверава да ли систем исправно обрађује сценарио у коме нема промена на нивоу ставки.

6) Додавање нових ставки

У овом тесту стара листа ставки има, на пример, ставку са RB=1, док нова листа садржи и RB=1 и RB=2. Очекује се:

- update постојеће ставке (RB=1)
- insert нове ставке (RB=2)
- успешно завршавање операције

7) Брисање ставки које више не постоје

Ово је најинтересантнији тест. Стара листа има ставку RB=1, али нова има само RB=2. Очекује се да:

- систем позове delete() над ставком RB=1
- извршавање templateExecute() прође без бацања изузетка

FakeStatement омогућава да се ови позиви симулирају без стварне базе.

```

27 public class SOUpdatePorudzbinaTest {
28
29     private SOUpdatePorudzbina so;
30     private Porudzbina porudzbina;
31
32     private class PogresanObjekat extends AbstractDomainObject {
33
34         @Override
35         public String nazivTabele() {
36             return "";
37         }
38
39         @Override
40         public String alias() {
41             return "";
42         }
43
44         @Override
45         public String join() {
46             return "";
47         }
48
49         @Override
50         public ArrayList<AbstractDomainObject> vratiListu(java.sql.ResultSet rs) {
51             return null;
52         }
53
54         @Override
55         public String koloneZaInsert() {
56             return "";
57         }
58
59         @Override
60         public String vrednostiZaInsert() {
61             return "";
62         }
63
64         @Override
65         public String vrednostiZaUpdate() {
66             return "";
67         }
68
69         @Override
70         public String uslov() {
71             return "";
72         }
73
74         @Override
75         public String uslovZaSelect() {
76             return "";
77         }
78     }

```

Слика 145: Тестирање системске операције SOUpdatePorudzbina

```

79
80 @Before
81 public void setUp() throws Exception {
82
83     so = new SOUpdatePorudzbina();
84
85     DBBroker.getInstance().setConnection(new FakeConnection());
86
87     porudzbina = new Porudzbina();
88     porudzbina.setPorudzbinaID(porudzbinaID);
89     porudzbina.setDatumVreme(new Date());
90     porudzbina.setDatumIsporuke(new Date(System.currentTimeMillis() + 86400000)); // sutra
91     porudzbina.setKupac(new Kupac());
92     porudzbina.setAdministrator(new Administrator());
93
94     ArrayList<StavkaPorudzbine> stavke = new ArrayList<>();
95     stavke.add(new StavkaPorudzbine(porudzbina, sbr: 1, količina: 2, cena: 200, iznos: 400,
96         new Cokolada(cokoladaID: 1, naziv: "Crna", opis: "Opis", cena: 200, new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna"))));
97
98     porudzbina.setStavkePorudzbine(stavke);
99
100     DBBroker.getInstance().setSelectResult(new ArrayList<>());
101
102 }
103
104 @After
105 public void tearDown() {
106     so = null;
107     porudzbina = null;
108 }
109
110 @Test
111 public void testValidationValidObject() {
112     try {
113         so.validate(so);
114     } catch (Exception e) {
115         fail("Validan objekat NE SME da padne! -> " + e.getMessage());
116     }
117 }
118
119 @Test(expected = Exception.class)
120 public void testValidationInvalidObject() throws Exception {
121     so.validate(new PogresanObjekat());
122 }
123
124 @Test(expected = Exception.class)
125 public void testValidationDateInvalid() throws Exception {
126     porudzbina.setDatumIsporuke(new Date()); // danas -> neispravno
127     so.validate(so);
128 }

```

Слика 146: Тестирање системске операције SOUpdatePorudzbina

```

129     @Test(expected = Exception.class)
130     public void testValidationNoItems() throws Exception {
131         porudzbina.setStavkePorudzbine(new ArrayList<>());
132         so.validate(ado:porudzbina);
133     }
134
135     @Test
136     public void testExecuteOperationSuccess() {
137         try {
138             ArrayList<AbstractDomainObject> stare = new ArrayList<>();
139             stare.add(new StavkaPorudzbine(porudzbina, rb: 1, kolicina: 2, cena: 200, iznos: 400,
140                 new Cokolada(cokoladaID: 1, naziv: "Crna", opis: "opis", cena: 200, new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna"))));
141
142             DBBroker.getInstance().setSelectResult(result:stare);
143
144             so.templateExecute(ado:porudzbina);
145
146         } catch (Exception e) {
147             fail("Operacija nije smela da baci izuzetak! -> " + e.getMessage());
148         }
149     }
150
151     @Test
152     public void testExecuteOperationInsertNewItem() {
153         try {
154             ArrayList<AbstractDomainObject> stare = new ArrayList<>();
155             stare.add(new StavkaPorudzbine(porudzbina, rb: 1, kolicina: 2, cena: 200, iznos: 400,
156                 new Cokolada(cokoladaID: 1, naziv: "Crna", opis: "opis", cena: 200, new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna"))));
157
158             DBBroker.getInstance().setSelectResult(result:stare);
159
160             porudzbina.getStavkePorudzbine().add(
161                 new StavkaPorudzbine(porudzbina, rb: 2, kolicina: 1, cena: 300, iznos: 300,
162                     new Cokolada(cokoladaID: 2, naziv: "Mlecna", opis: "opis", cena: 300, new VrstaCokolade(vrstaCokoladeID: 2, naziv: "Mlecna"))));
163
164             so.templateExecute(ado:porudzbina);
165
166         } catch (Exception e) {
167             fail("Insert nove stavke NE SME da padne! -> " + e.getMessage());
168         }
169     }
170
171 }

```

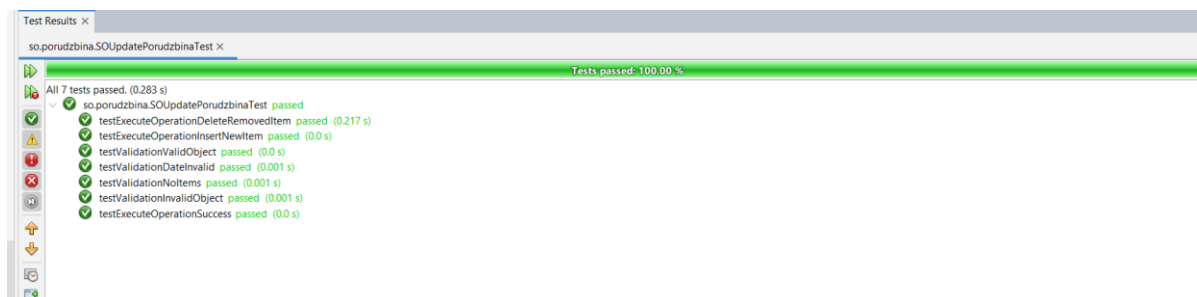
Слика 147: Тестирање системске операције SOUpdatePorudzbina

```

172     @Test
173     public void testExecuteOperationDeleteRemovedItem() {
174         try {
175             ArrayList<AbstractDomainObject> stare = new ArrayList<>();
176             stare.add(new StavkaPorudzbine(porudzbina, rb: 1, kolicina: 2, cena: 200, iznos: 400,
177                 new Cokolada(cokoladaID: 1, naziv: "Crna", opis: "opis", cena: 200, new VrstaCokolade(vrstaCokoladeID: 1, naziv: "Crna"))));
178
179             DBBroker.getInstance().setSelectResult(result:stare);
180
181             ArrayList<StavkaPorudzbine> nove = new ArrayList<>();
182             nove.add(new StavkaPorudzbine(porudzbina, rb: 2, kolicina: 1, cena: 300, iznos: 300,
183                 new Cokolada(cokoladaID: 2, naziv: "Mlecna", opis: "opis", cena: 300, new VrstaCokolade(vrstaCokoladeID: 2, naziv: "Mlecna"))));
184
185             porudzbina.setStavkePorudzbine(stavkePorudzbine:nove);
186
187             so.templateExecute(ado:porudzbina);
188
189         } catch (Exception e) {
190             fail("Delete stare stavke NE SME da padne! -> " + e.getMessage());
191         }
192     }
193
194 }
195

```

Слика 148: Тестирање системске операције SOUpdatePorudzbina



Слика 149: Тестирање системске операције SOUpdatePorudzbina

5. Закључак

Развој софтверског система за продавницу чоколаде представља пример примене упрошћене Ларманове методе развоја софтвера у пракси, од иницијалне формулације корисничких захтева, преко анализе и пројектовања, па све до имплементације и тестирања комплетног решења. Ова метода показала се као изузетно погодна за академске и мање пројекте јер на структуриран и јасан начин води студента кроз све фазе развоја, уз задржавање објектно-оријентисаних принципа и фокус на раздвајање структуре и понашања система.

Током рада развијено је софтверско решење које омогућава ефикасну и интуитивну подршку пословним процесима продавнице чоколаде: управљању чоколадама и њиховим врстама, купцима, поруџбинама и ставкама поруџбина. Анализом захтева дефинисани су случајеви коришћења који су затим послужили као основ за идентификацију системских операција, израду концептуалног модела и релационе базе података, као и логичку структуру пословне логике.

У фази пројектовања, примена тронивојске архитектуре омогућила је доследно раздвајање корисничког интерфејса, апликационе логике и складишта података. Овакав приступ омогућава лакше одржавање, већи степен модуларности и боље разумевање међузависности унутар система. Имплементација је спроведена у програмском језику Java, уз коришћење Swing-а за графички интерфејс и JDBC-а за приступ бази података. Пословна логика реализована је кроз шаблон системских операција `templateExecute()`, који омогућава конзистентан и сигуран рад са базом.

Посебан допринос овог рада јесте изведена стратегија тестирања, при чему је изолован рад логике од инфраструктуре базе података коришћењем `FakeConnection` механизма. На тај начин тестиране су комплексне системске операције као што су `SOAddCokolada` и `SUUpdatePorudzbina`, уз проверу валидности улазних података, правилног понашања `SELECT/INSERT/UPDATE/DELETE` логике и исправног руковања изузецима. Овакав приступ обезбеђује већу поузданост система и значајно олакшава будуће одржавање.

Иако развијени систем у потпуности задовољава дефинисане функционалне захтеве, постоји простор за надоградњу. Потенцијална побољшања укључују прелазак на савременије графичке библиотеке (JavaFX), увођење напреднијих техника валидације, развој веб интерфејса или интеграцију са системима за електронску трговину. Додатно, увођење ORM алата попут `Hibernate`-а могло би поједноставити рад са базом и омогућити лакше проширивање модела.

У целини гледано, реализација овог пројекта показује да правилна примена методологије, уз разумевање архитектонских и програмских принципа, омогућава развој стабилног, логички конзистентног и употребљивог софтверског решења. Рад представља успешну симулацију реалног процеса развоја софтверског система и добру основу за даље истраживање и професионално усавршавање.

6. Коришћена литература

1. Влајић, С. (2020). **Пројектовање софтвера – радни материјал (вер. 1.3)**. Факултет организационих наука, Универзитет у Београду.
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). **Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley.
3. Oracle. (2024). **Java Platform, Standard Edition Documentation**. <https://docs.oracle.com/javase/>
4. Oracle. **JDBC API Guide**. <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
5. Beck, K. (2001). **Test Driven Development: By Example**. Addison-Wesley.
6. JUnit Team. **JUnit 4 Documentation**. <https://junit.org/junit4/>